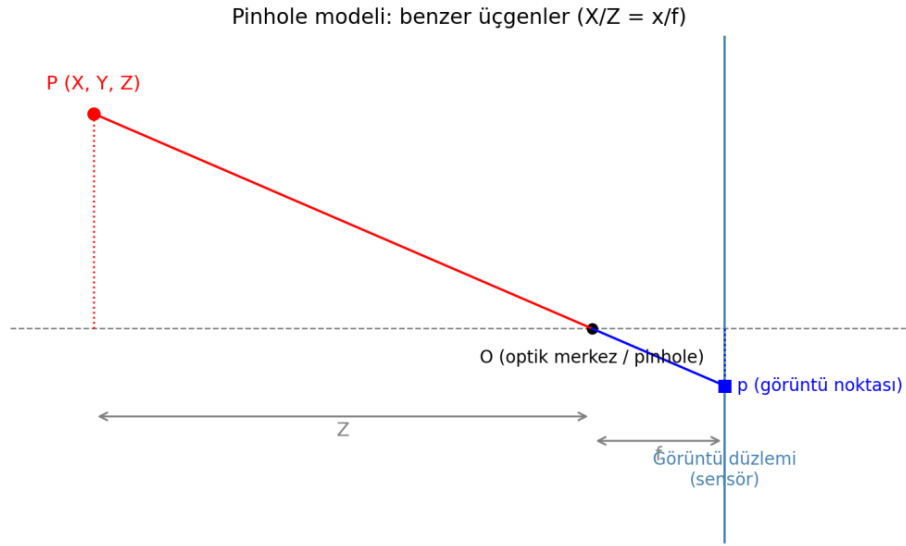


Gimbal–Lazer Tabanlı Kamera Distorsiyon Kalibrasyonu

*Star Tracker Optik Sistemi için Kamera İçsel Parametre ve Distorsiyon Karakterizasyonu —
DeneySEL Çalışma Raporu*



TÜBİTAK UZAY

Star Tracker Optik Kalibrasyon Çalışması

Hazırlayan: Ekin & Ece & İdil & Yakup

Ağustos 2026

1. Giriş ve Amaç

Gerçek bir kamera, ders kitaplarındaki ideal pinhole (iğne deliği) modelinden iki temel noktada sapar: (i) lensin odak uzaklığı ve optik merkezi üretim ve montaj toleransları nedeniyle nominal değerlerinden kayabilir, (ii) lens elemanları görüntüyü doğrusal olmayan biçimde bükerek (**distorsiyon**). Bu çalışmanın nihai amacı, uzayda seyir hâlindeki araçların yönelimini (attitude) yıldız görüntüleri üzerinden belirleyen bir star tracker sisteminde kullanılacak kameranın optik sisteminin distorsiyon katsayılarını yüksek doğrulukla belirleyebilen bir kalibrasyon algoritması ve düzeneği geliştirmektir. Bir star tracker'ın yönelim tahmini, yıldız görüntülerindeki piksel konumlarının, kameranın matematiksel modeli (içsel parametreler ve distorsiyon katsayıları) aracılığıyla gerçek açısal konumlara doğru biçimde dönüştürülebilmesine dayanır; bu dönüşümün doğruluğu, doğrudan kameranın lensine ait distorsiyon karakterizasyonunun kalitesiyle sınırlıdır.

Bu çalışmada asıl hedeflenen, tek bir lens için nokta atışı bir ölçüm yapmak değil, sürdürülebilir ve genelleştirilebilir bir yöntem ortaya koymaktır. Geliştirilecek düzeneğin ve algoritmanın her ayrıntısıyla raporlanması, ileride kullanılan lens değiştiğinde sıfırdan yeni bir kalibrasyon algoritması tasarlamak yerine, aynı algoritmanın yalnızca kameranın içsel parametrelerine (f_x , f_y , c_x , c_y) ilişkin küçük güncellemelerle farklı lenslerin distorsiyon katsayılarını da bulabilmesini mümkün kılmayı amaçlamaktadır.

Bu doğrultuda, çalışmanın şu anki aşaması geliştirilmiş, nihai bir sistemden ziyade, tek bir lens üzerinde yürütülen bir deneme ve geliştirme sürecidir; amaç, en az hatayla (en düşük RMS ile) sonuç veren ölçüm yöntemini ve düzenek parametrelerini (tarama deseni, çalışma mesafesi, kolimasyon, merkez tespit algoritması vb.) belirlemektir. Optimum ölçüm koşulları tespit edildiğinde, düzenek ve algoritma nihai hâline getirilerek yukarıda tarif edilen sürdürülebilir ve genelleştirilebilir kalibrasyon sistemine dönüştürülecektir.

Bunun için birbirini tamamlayan iki bağımsız yöntem izlenmiştir:

- Aşama 1 — Chessboard (satranç tahtası) tabanlı klasik OpenCV kalibrasyonu: Bilinen geometri bir düzlemsel görüntü farklı açı ve mesafelerden görüntülenerek kameranın ilk kaba içsel parametre kestirimi elde edilmiştir.
- Aşama 2 — Gimbal + lazer tabanlı kontrollü grid taraması: Bilinen açısal konumlara hassas biçimde konumlandırılan bir lazer kaynağının kameradaki izdüşümü sistematik olarak taranarak, distorsiyon alanı doğrudan ve yoğun biçimde örneklenmiştir.

Bu iki yöntem birbirinin yerine geçmez; chessboard yöntemi hızlı ve standart bir başlangıç noktası sağlarken, gimbal-lazer yöntemi kameranın görüş alanının tamamında, bilinen ve çok hassas açısal referanslarla distorsiyonun uzaysal davranışını doğrudan haritalandırma imkânı verir. Aşağıdaki bölümlerde önce sistemin donanımı, ardından kullanılan optik ve matematiksel altyapı, sonrasında deneysel yöntem ve son olarak elde edilen ölçüm sonuçları sunulmaktadır.

Bu rapor iki ayrı çalışma kaydının birleştirilmesiyle, çalışmanın gerçekte izlediği kronolojik sırayla oluşturulmuştur: önce kameranın içsel parametrelerini bulmak için denenen checkerboard tabanlı klasik kalibrasyon (Bölüm 2), ardından bu yöntemin yetersiz kaldığı noktada devreye giren gimbal donanımı ve gimbal-lazer tabanlı distorsiyon kalibrasyonu (Bölüm 3 ve sonrası) tek bir akış hâlinde sunulmaktadır.

2. Kamera Kalibrasyonu İhtiyacı: Checkerboard Tabanlı İnterinsik Kalibrasyon (Zhang Yöntemi)

İşe başlarken elimizde kameranın matematiksel modeli yoktu; star tracker için piksel-açı dönüşümünü kurabilmemizin ilk şartı ise kameranın içsel parametrelerini (intrinsic parameters) f_x , f_y , c_x , c_y ve distorsiyon katsayılarını -- bilmektir. Bu yüzden ilk elimizi attığımız şey, literatürde en yaygın ve en hızlı kurulabilen yöntem oldu: Zhang'in checkerboard (satranç tahtası) tabanlı klasik kalibrasyon tekniği. Fikir basitti, kameranın karşısına geometrisini bildiğimiz düzlemsel bir desen koyup farklı açı ve mesafelerden fotoğraflarsak, köşe noktalarının piksel konumlarından yola çıkarak kameranın "kimlik kartını" geriye doğru çözebilirdik. Aşağıdaki bölüm bu ilk denememizin (yöntem, 11 kalibrasyonluk ölçüm kampanyası ve çıkardığımız derslerin) tam dökümüdür.

Bu çalışma, kullanılan kameranın iç parametrelerini belirlemek ve optik eksenin görüntü düzlemini kestiği ana noktayı (principal point) piksel cinsinden hesaplamak amacıyla gerçekleştirilmiştir. OpenCV kalibrasyon modelinde bu nokta c_x ve c_y koordinatlarıyla ifade edilir. Kalibrasyon boyunca lens odağının, kamera çözünürlüğünün ve kırpm/ölçekleme ayarlarının **değiştirilmemesi** gerekir.

Kamera İç Parametre Matrisi

$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$

f_x ve f_y : piksel cinsinden odak parametreleridir.

c_x ve c_y : optik merkezin görüntü üzerindeki piksel koordinatlarıdır.

Bu kod f_x ve f_y değerlerini `camera_matrix` dizisinin içinde, c_x ve c_y değerlerini ise ayrıca `npz` dosyasında saklar.

2.1 Kalibrasyon Kodunun İşleyişi

- **Kamera bağlantısı:** VmbPy aracılığıyla bilgisayara bağlı ilk kamera açılır ve mümkünse Mono8 piksel formatı seçilir.
- **Köşe tespiti:** CHECKERBOARD = (8, 6) tanımı, satranç tahtasındaki kare sayısını değil yatay ve dikey iç köşe sayısını belirtir.
- **Görüntü kaydı:** Köşeler algılandığında SPACE tuşuna her basışta bir kalibrasyon karesi kaydedilir; hedef sayı 20'dir ama değiştirilebilir.
- **Alt piksel iyileştirme:** `cornerSubPix` fonksiyonu, bulunan köşelerin koordinatlarını piksel altı hassasiyetle düzeltir.
- **Kalibrasyon hesabı:** `calibrateCamera`; kamera matrisi, distorsiyon katsayıları ve RMS yeniden izdüşüm hatasını hesaplar.
- **Dosya çıktısı:** `camera_matrix`, `distortion`, görüntü boyutu, c_x , c_y ve rms değerleri `camera_calibration2.npz` dosyasına kaydedilir.

Kalibrasyon tamamlandıktan sonra, sonuç `npz` dosyasından f_x , f_y , c_x , c_y ve RMS değerleri şu şekilde okunabilir:

```
import numpy as np

data = np.load("camera_calibration2.npz")
K = data["camera_matrix"]

fx = K[0, 0]
fy = K[1, 1]
cx = K[0, 2]
cy = K[1, 2]

print(f"fx = {fx:.3f} px")
print(f"fy = {fy:.3f} px")
print(f"cx = {cx:.3f} px")
print(f"cy = {cy:.3f} px")
print(f"RMS = {float(data['rms']):.3f} px")
```

2.2 Uygulama Yöntemi

İlk kalibrasyonlarda 8×6 iç köşeli checkerboard kullanılmıştır. Köşe noktaları kamera görüntüsünde algılandığında SPACE tuşuna basılarak görüntü kaydedilmiş, her kayıttan sonra checkerboard'un konumu, açısı veya kameraya uzaklığı değiştirilmiştir. Yirmi geçerli görüntü tamamlandığında kod, optik merkez koordinatlarını ve RMS yeniden izdüşüm hatasını hesaplamıştır.

Checkerboard boyutunun etkisini gözlemek amacıyla üçüncü ölçümde 6×6 iç köşeli farklı bir desen kullanılmış, son ölçümde tekrar 8×6 desene dönmüştür. Karşılaştırmada 2048×2048 çözünürlüklü görüntünün geometrik merkezi olan (1024, 1024) referans alınmıştır.

2.3 Ölçüm Sonuçları: 11 Kalibrasyon, 3 Ortam (24.07.2026)

Optik merkezin tek bir kalibrasyona bakılarak belirlenmesi güvenilir değildir: her kalibrasyon, o anki poz dağılımına, ışığa ve desenin kalitesine göre biraz farklı bir sonuç üretir. Bu nedenle checkerboard üç ayrı kaynaktan gösterilerek toplam 11 bağımsız kalibrasyon yapılmıştır. Aşağıdaki tablolarda her ölçümün sonucu, 2048×2048 görüntünün geometrik merkezi olan (1024, 1024) noktasına göre sapmasıyla birlikte verilmektedir.

Merkezden Sapma Hesabı

$$\Delta x = c_x - 1024$$

$$\Delta y = c_y - 1024$$

$$d = \sqrt{(\Delta x)^2 + (\Delta y)^2}$$

Buradaki d değeri, hesaplanan optik merkezin nominal görüntü merkezine piksel cinsinden uzaklığıdır; tek başına kalibrasyon doğruluğu anlamına gelmez.

Tablolardaki okuma notu: “±” ile verilen değer, o kalibrasyonun kendi iç belirsizliğidir (calibrateCameraExtended fonksiyonunun döndürdüğü standart sapma). RMS ise modelin köşeleri ne kadar iyi açıkladığını gösteren yeniden izdüşüm hatasıdır. Bilgisayar ekranı setindeki ilk üç ölçümde konsol çıktısının yalnızca özet satırları kaydedildiği için ± değerleri mevcut değildir.

Bilgisayar Ekranından Yansıtılan Checkerboard

Deney #	c_x (px)	c_y (px)	Δx (px)	Δy (px)	d (px)	RMS (px)
1	1028.839	918.711	+4.839	-105.289	105.40	0.2348
2	1031.135	972.087	+7.135	-51.913	52.40	0.3080
3	1037.676	939.852	+13.676	-84.148	85.25	0.7660
4	1031.414 ±1.365	948.439 ±1.263	+7.414	-75.561	75.92	0.3942

(Tablo 9.1. Bilgisayar ekranı hedefiyle yapılan 4 kalibrasyon)

A4 Kâğıda Basılı Checkerboard

Deney #	c_x (px)	c_y (px)	Δx (px)	Δy (px)	d (px)	RMS (px)
1	1033.507 ±3.662	932.442 ±2.875	+9.507	-91.558	92.05	0.8934
2	1052.799 ±3.031	965.008 ±2.803	+28.799	-58.992	65.65	0.8781
3	1029.264 ±2.924	967.249 ±2.584	+5.264	-56.751	56.99	0.9183
4	998.251 ±8.528	991.578 ±3.389	-25.749	-32.422	41.40	0.8487

(Tablo 9.2. A4 kâğıt hedefiyle yapılan 4 kalibrasyon)

Telefon Ekranından Yansıtılan Checkerboard

Deney #	cx (px)	cy (px)	Δx (px)	Δy (px)	d (px)	RMS (px)
1	1034.322 $\pm$ 2.502	942.834 $\pm$ 2.154	+10.322	-81.166	81.82	0.5173
2	1033.896 $\pm$ 2.980	939.117 $\pm$ 2.739	+9.896	-84.883	85.46	0.5350
3	1026.415 $\pm$ 2.328	949.412 $\pm$ 2.052	+2.415	-74.588	74.63	0.4725

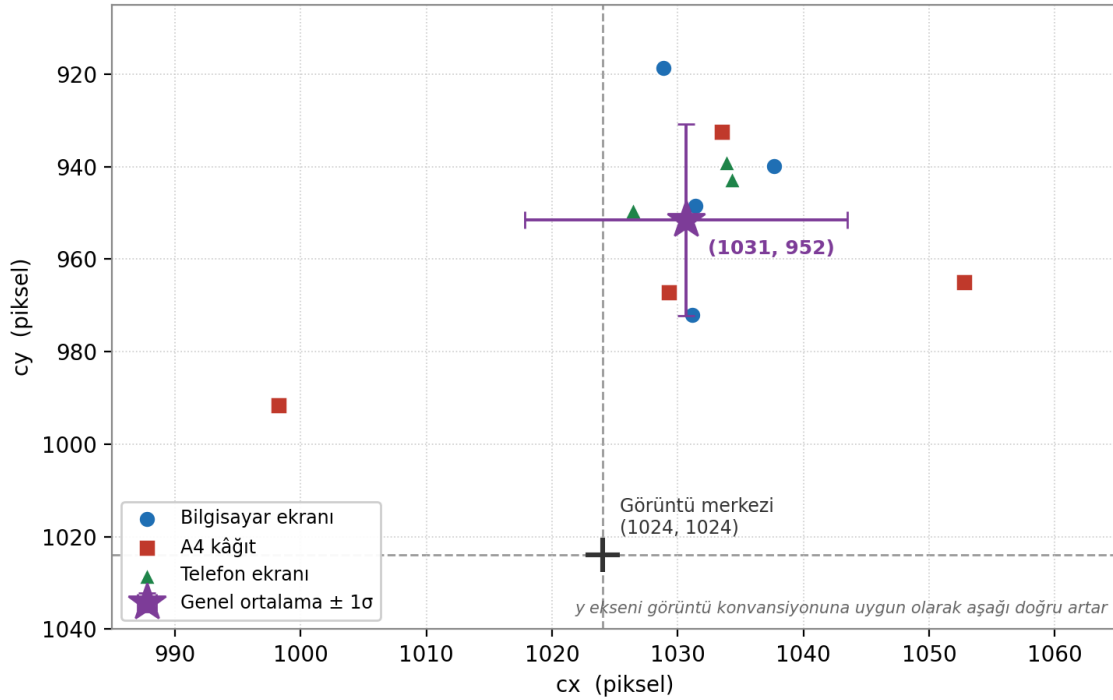
(Tablo 9.3. Telefon ekranı hedefiyle yapılan 3 kalibrasyon)

Ortamların Karşılaştırması

Ortam	n	cx ortalama $\pm \sigma$	cy ortalama $\pm \sigma$	Ortalama Δy	Ortalama RMS
Bilgisayar ekranı	4	1032.27 $\pm$ 3.28	944.77 $\pm$ 19.12	-79.23	0.426
A4 kâğıt	4	1028.46 $\pm$ 19.56	964.07 $\pm$ 21.02	-59.93	0.885
Telefon ekranı	3	1031.54 $\pm$ 3.63	943.79 $\pm$ 4.26	-80.21	0.508
TÜMÜ (genel)	11	1030.68 $\pm$ 12.83	951.52 $\pm$ 20.68	-72.48	0.615

(Tablo 9.4. Ortam bazında özet — σ , o gruptaki ölçümlerin birbirine göre yayılımıdır)

11 kalibrasyonda bulunan optik merkez konumları



(Grafik 9.1. 11 kalibrasyonda bulunan optik merkez konumları. Yatay eksen cx, dikey eksen cy; kesikli çizgiler görüntü merkezini gösterir.)

2.4 Tabloların Yorumu ve Önerilen Optik Merkez

Tablolar ve grafik birlikte okunduğunda dört net eğilim ortaya çıkmaktadır:

- **cy her ölçümde merkezin üstünde kalıyor.** On bir ölçümün on biri de 1024'ün altında bir cy değeri vermiştir (en yüksek 991.6 px, en düşük 918.7 px). Ortalama $\Delta y = -72.5$ px'dir. Bir ölçüm hatası bu kadar tek yönlü olamaz; rastgele bir gürültü olsaydı değerlerin yaklaşık yarısının merkezin altında, yarısının üstünde çıkması beklenirdi.

- **cx merkeze yakın ve rastgele dağılıyor.** cx değerleri 998.3 ile 1052.8 px arasında, merkezin bir sağında bir solunda yer almaktadır. Ortalaması 1030.7 px, yani merkezden yalnızca ≈ 7 px uzaktadır ve bu fark grubun kendi yayılımının (± 12.8 px) içinde kalır. Yatay eksenle anlamlı bir kayma yoktur.
- **Yayılım, tek bir ölçümün iç hatasından çok daha büyük.** Tek bir kalibrasyonun kendi RMS hatası 0.23–0.92 px aralığındadır; buna karşılık ölçümler arası yayılım cx için ± 12.8 px, cy için ± 20.7 px'dir. Aradaki bu büyüklük farkı, sonucu belirleyen şeyin algoritmanın fit kalitesi değil, kalibrasyon oturumları arasındaki poz/hedef farklılıkları olduğunu gösterir. Yani “RMS düşük çıktı, öyleyse cy doğrudur” denemez.
- **Hedefin türü RMS'i etkiliyor, ancak kaymayı ortadan kaldırmıyor.** Ekran tabanlı hedefler belirgin şekilde daha temiz sonuç vermiştir (bilgisayar ekranı ortalama RMS 0.43 px, telefon 0.51 px, A4 kâğıt 0.89 px). Bu beklenen bir sonuçtur: ekran kusursuz düzlemdir ve kontrastı sabittir, kâğıt ise kıvrılabilir ve baskı kalitesi değişkendir. Buna rağmen cy kayması her üç ortamda da aynı yönde ve benzer büyüklükte kalmıştır (ortam ortalamaları –60 ile –80 px arası). Kayma hedeften kaynaklansaydı, ortamlar arasında yön değiştirmesi gerekirdi.

Bu Ne Anlama Geliyor?

Görüntü merkezi (1024, 1024) ile optik merkez (≈ 1031 , ≈ 952) aynı nokta değildir. Aradaki ≈ 73 piksellik fark, ağırlıklı olarak düşey eksendedir ve lens ile sensörün birbirine göre hafif kaçık monte edilmiş olmasıyla açıklanabilir. Bu, kameranın bozuk olduğu anlamına gelmez. Düzeltmesi değil, hesaba katılması gereken sabit bir donanım özelliğidir.

Pratik sonuç: gimbal hedefleme hesaplarında görüntünün geometrik merkezi değil, kalibrasyonla bulunan optik merkez referans alınmalıdır.

Kamera optik merkezinin belirlenmesi amacıyla üç farklı ortamda (A4 kâğıt, bilgisayar ekranı, telefon ekranı) toplam 11 bağımsız checkerboard kalibrasyonu gerçekleştirilmiştir. Sonuçlar incelendiğinde optik merkezin x bileşeni (cx) genel olarak görüntü merkezine yakın ve nispeten rastgele dağılırken (998–1053 px, ortalama ≈ 1031 px), y bileşeninin (cy) hemen hemen tüm ölçümlerde görüntü merkezinin (1024 px) belirgin şekilde altında kaldığı görülmüştür (919–992 px, ortalama ≈ 952 px).

Bu eğilimin hem baskılı hem ekran tabanlı hedeflerle yapılan ölçümlerde tutarlı şekilde tekrarlanması ve ölçümler arası yayılımın (cx için ± 13 px, cy için ± 21 px) tekil bir kalibrasyonun kendi iç fit hatasından (RMS ≈ 0.23 – 0.92 px) çok daha büyük olması, bu kaymanın rastgele ölçüm gürültüsünden değil, lens–sensör hizalamasındaki gerçek bir dikey ofsetten kaynaklandığına işaret etmektedir. Buna karşın hedef boyutu, baskı kalitesi ve aydınlatma gibi etkenlerin de belirli bir belirsizlik payı katabileceği göz önünde bulundurulmalıdır.

Bu doğrultuda kod içerisinde kullanılacak optimal optik merkez değeri aşağıdaki gibi önerilmektedir:

Önerilen Optik Merkez Değeri

$$(cx, cy) = (1031, 952) \pm (13, 21) \text{ px}$$

Bu değer, üç ortamdaki 11 kalibrasyonun ortalamasıdır; $\pm$ değerleri ölçümler arası yayılımı (1σ) ifade eder.

Bulgunun bağımsız bir lazer/otokollimasyon ölçümüyle doğrulanması tavsiye edilmektedir. Ayrıca ölçüm sayısı artırılırken checkerboard'un görüntünün dört köşesine ve kenarlarına daha sık taşınması, cy tahminindeki yayılımı azaltacaktır.

Not: Bu değerler lens odağı, kamera çözünürlüğü veya kırpm/ölçekleme ayarları değiştirilmediği sürece geçerlidir. Bunlardan herhangi biri değişirse kalibrasyon tekrarlanmalıdır.

2.5 Olası Hata Kaynakları

- **Yetersiz poz çeşitliliği:** Checkerboard görüntülerin çoğunda aynı uzaklıkta, benzer açıyla veya yalnızca görüntünün merkezinde kalmış olabilir.
- **Kenar bölgelerinin az örneklenmesi:** Distorsiyon ve optik merkez tahmini için checkerboard'un görüntünün dört köşesine ve kenarlarına da taşınması gerekir.
- **Odak değişimi:** Lens odağının ölçümler arasında oynatılması, kamera iç parametrelerini değiştirir ve farklı veri setlerinin karşılaştırılmasını bozar. Bundan şüpheleniyoruz.

- **Hareket bulanıklığı veya netlik kaybı:** SPACE tuşuna basıldığı anda checkerboard hareketliyse ya da görüntü net değilse köşe koordinatları hatalı bulunabilir.
- **Checkerboard düzlemselliği:** Kâğıdın bükülmesi, dalgalanması veya baskı ölçeğinin düzensiz olması, düzlemsel model varsayımını bozar. (A4 setinin en yüksek RMS değerlerini vermesi bu etkiyle uyumludur.)
- **Işık ve yansıma:** Parlama, düşük kontrast veya doygun bölgeler köşe tespitini ve alt piksel iyileştirmesini olumsuz etkileyebilir.
- **Yanlış iç köşe tanımı:** Kodda verilen (8, 6) veya (6, 6) değerleri kullanılan desenin gerçek iç köşe sayısıyla birebir uyuşmalıdır.
- **Çözünürlük/kırpma değişikliği:** Kalibrasyon görüntülerinin farklı boyutta alınması, sonradan ölçeklenmesi veya kırılması cx ve cy değerlerini doğrudan değiştirir.

2.6 Köşe Tespiti Sorunu ve Çözümü (26–27.07.2026)

26–27.07.2026 tarihlerinde yapılan denemelerde, checkerboard kameranın tam karşısında ve gözle bakıldığında net görünmesine rağmen findChessboardCorners fonksiyonu köşeleri çoğu karede bulamamıştır. Bulduğu karelerde ise sonuçlar oturum oturum tutarsız çıkmıştır. Yapılan incelemede sorunun tek bir sebebi olmadığı, birkaç etkenin üst üste bindiği görülmüştür:

- **Otomatik pozlama:** Kamera her karede pozlamayı yeniden ayarladığı için siyah-beyaz kare kontrastı sürekli değişiyor, köşe arama algoritması kararsız kalıyordu.
- **Yüksek çözünürlükte arama:** 2048×2048 görüntüde köşe araması hem yavaştı hem de sensör gürültüsü yanlış kenarlar üretiyordu.
- **Sabit alt piksel penceresi:** cornerSubPix için kullanılan sabit pencere, tahta kameraya yaklaştığında komşu köşeye taşarak koordinatı bozuyordu.
- **Poz tekrarı:** Arka arkaya çekilen kareler birbirine çok benziyordu; kalibrasyona yeni bilgi katmayan, sadece sayıyı şişiren kareler birikiyordu.
- **Tek bozuk karenin etkisi:** Bulanık ya da hatalı tespit edilmiş tek bir kare, ortak çözülen $fx/fy/cx/cy$ setini gözle görülür şekilde kaydırabiliyordu.

2.7 Kalibrasyon Algoritmasının Matematiği: Zhang Yöntemi

Bu bölüm, önceki bölümlerde kullanılan cx , cy , fx ve fy değerlerinin checkerboard fotoğraflarından nasıl çıktığını adım adım anlatır. Amaç, kodun içinde tek satırda geçen calibrateCamera çağrısının arka planda ne yaptığını görünür kılmaktır.

Kalibrasyon Neyi Arıyor?

Kamera, üç boyutlu dünyayı iki boyutlu bir piksel ızgarasına düşürür. Bu düşürme işlemi rastgele değildir; belirli bir matematiksel kurala uyar ve o kural dört sayıyla tanımlanır:

- **fx ve fy** — Büyütme çarpanı. Kameranın önündeki bir nesnenin, görüntüde kaç piksele karşılık geldiğini belirler. Odak uzaklığının piksel cinsinden ifadesidir; fx yatay, fy düşey yöndedir.
- **cx ve cy** — Optik merkez. Merceğin tam ortasından geçen hayali eksenin sensöre çarptığı piksel noktasıdır. Görüntünün geometrik ortası ile aynı nokta olmak zorunda değildir — bu raporun 9. bölümü zaten tam olarak bu farkı ölçmüştür.

Bu dört sayı kameraya aittir; hangi fotoğrafı çekersek çekelim **değişmezler**. İşte kalibrasyonun tek amacı bunları bulmaktır. Ancak bunları doğrudan ölçebileceğimiz bir alet yoktur; dolaylı yoldan, geometrisi zaten bilinen bir nesnenin fotoğraflarına bakarak geri hesaplarız. O nesne de checkerboard'dur: kare boyutlarını biz belirlediğimiz için, üzerindeki her köşenin birbirine göre konumunu daha en baştan biliriz.

Sembol Sözlüğü

Aşağıdaki tablo, bu bölümde geçen her sembolün ne olduğunu tanımlar.

Sembol	Ne demek?
X, Y, Z	Bir checkerboard köşesinin, checkerboard'un kendi üzerindeki konumu. Sol-üst köşe (0, 0) kabul edilir. Tahta düz bir yüzey olduğu için Z her zaman 0'dır.
x, y, z	Aynı köşenin, bu sefer kameraya göre konumu. Kameranin merceği orijin kabul edilir; z, köşenin kameraya olan uzaklığıdır (derinlik).
x', y'	Perspektif bölmesi yapılmış, birimsiz koordinat. "Bu nokta optik ekseninden ne kadar yana kaymış" sorusunun oransal cevabıdır.
xu, yu	Köşenin fotoğraftaki gerçek piksel koordinatı. Görüntüde gördüğümüz, ölçebildiğimiz sayı budur.
fx, fy	Piksel cinsinden odak uzaklığı — büyütme çarpanı.
cx, cy	Optik merkez — optik eksenin sensörü kestiği piksel noktası.
K	Yukarıdaki dört sayıyı tek bir 3×3 matriste toplayan iç parametre matrisi. Kameranin "kimlik kartı"dır.
R	3×3 döndürme matrisi. Bir fotoğraf çekilirken kameranin tahtaya göre hangi açıda durduğunu tarif eder. Her fotoğrafta farklıdır.
T	3 elemanlı öteleme vektörü. Aynı anda kameranin tahtaya göre nerede durduğunu tarif eder. Bu da her fotoğrafta farklıdır.
H	Homografi matrisi (3×3). Düz bir yüzeydeki (X, Y) noktalarını, fotoğraftaki (xu, yu) piksellerine doğrudan bağlayan dönüşümdür.
k ₁ , k ₂ , p ₁ , p ₂	Distorsiyon katsayıları. Merceğin görüntüyü kenarlara doğru bükmesini tarif eden düzeltme terimleridir.

(Tablo 11.1. Kalibrasyonda geçen semboller)

Kullanılan Formüller

Aşağıdaki formüller, sonraki aşamalarda geçen tüm matematiği kapsar. Aşamaları okurken buraya dönüp bakmak yeterlidir.

Formül Listesi

- 1) Kameraya göre konum: $[x \ y \ z]^T = R \cdot [X \ Y \ Z]^T + T$
- 2) Perspektif bölmesi: $x' = x / z, \ y' = y / z$
- 3) Piksele geçiş: $xu = fx \cdot x' + cx, \ yu = fy \cdot y' + cy$
- 4) İç parametre matrisi: $K = \begin{bmatrix} fx & 0 & cx \\ 0 & fy & cy \\ 0 & 0 & 1 \end{bmatrix}$
- 5) Homografi: $s \cdot [xu \ yu \ 1]^T = H \cdot [X \ Y \ 1]^T$
- 6) Homografinin açılımı: $H = K \cdot \begin{bmatrix} r_1 & r_2 & t \end{bmatrix}$
- 7) R ve T'nin geri çıkarılması: $K^{-1} \cdot H = \begin{bmatrix} r_1 & r_2 & t \end{bmatrix}, \ r_3 = r_1 \times r_2$
- 8) Son optimizasyon: $\min \sum_i \sum_j \| (xu, yu)_{ij} - f(fx, fy, cx, cy, k_1, k_2, \dots, R_i, T_i, X_j, Y_j) \|^2$

Aşama Aşama Nasıl Bulunuyor?

1. Aşama — Checkerboard'un geometrisini yazıyoruz

Kod ilk iş olarak kameraya hiç bakmadan bir tablo hazırlar: tahtadaki her iç köşenin (X, Y, 0) koordinatı. Kare boyutu bizim verdiğimiz sabittir (kodda SQUARE_SIZE), dolayısıyla köşeler bu sabitin katları olarak sırayla yazılır. Bu tablo bütün fotoğraflar için ortaktır ve hiç değişmez. Buradaki tek varsayım şudur: tahta gerçekten düzdür, yani Z = 0.

2. Aşama — Tahtayı farklı pozlarda fotoğraflıyoruz

Checkerboard kameranın önünde tutulur ve her karede konumu değiştirilir: sağa-sola eğilir, yukarı-aşağı döndürülür, yaklaştırılıp uzaklaştırılır, görüntünün merkezine ve kenarlarına taşınır. Kodda hedef 25 karedir. Bu çeşitlilik keyfi değildir; 4. Aşamadan itibaren her poz denklem sistemine farklı bir “bakış açısı” ekler ve belirsizlikleri kırar. Aynı pozun tekrarı hiçbir şey katmaz — kod bu yüzden birbirine çok benzeyen pozları reddeder (is_new_pose).

3. Aşama — Her fotoğrafta köşelerin piksel konumunu ölçüyoruz

Her fotoğraf tek tek işlenir. Görüntü işleme algoritması siyah ve beyaz karelerin kesiştiği noktaları bulur, ardından cornerSubPix bu noktaları piksel altı hassasiyetle düzeltir. Çıktı, her köşe için bir (xu, yu) çiftidir. Dikkat: bu aşamada fx, fy, cx, cy’ye hiç ihtiyaç yoktur — yapılan iş tamamen görüntüdeki parlaklık geçişlerine bakmaktır. Yani elimizde artık iki liste var: bilinen (X, Y) listesi ve ölçülen (xu, yu) listesi.

4. Aşama — Her fotoğraf için homografi H’yi çözüyoruz

Tahta düz olduğu ($Z = 0$) için, (X, Y) ile (xu, yu) arasında tek bir 3×3 matris kurulabilir — bu H’dir (Formül 5). H’nin 9 elemanı vardır ama ölçek serbestliği nedeniyle 8 bağımsız bilinmeyi vardır. Kullandığımız 8×6 tahtada 48 köşe bulunur; her köşe 2 denklem verir, yani fotoğraf başına 96 denklem. 8 bilinmeyen için fazlasıyla yeterlidir, dolayısıyla H en küçük kareler (least squares) ile çözülür. Bu aşamada fx, fy, cx, cy, R ve T hâlâ ayrı ayrı bilinmiyor; hepsi H’nin içinde birbirine karışmış hâlde duruyor.

5. Aşama — Tüm fotoğrafları birleştirip K’yı çıkarıyoruz

Kilit adım budur. H, aslında K ile R’nin ilk iki sütunu ve T’nin çarpımıdır (Formül 6). R bir döndürme matrisi olduğu için sütunları birbirine dik ve birim uzunlukta olmak zorundadır. Bu geometrik zorunluluk, K hakkında her fotoğraftan 2 denklem üretir. 25 fotoğraf $\times 2 = 50$ denklem elde edilir ve bunlar K’daki dört bilinmeyene karşı yine en küçük karelerle çözülür. Burada anlaşılması gereken şey: fx, fy, cx, cy bütün fotoğraflarda ORTAKTIR. Her fotoğraf kendi fx’ini üretmez; 50 denklemin hepsi birlikte, tek bir dördlüyü en iyi açıklayacak şekilde çözülür.

6. Aşama — Her fotoğrafın kendi R ve T’sini geri çıkarıyoruz

K artık bilindiğine göre, bir önceki aşamada bulduğumuz H’ler tersine çevrilebilir: $K^{-1} \cdot H$ bize doğrudan r_1, r_2 ve t’yi verir (Formül 7). R’nin üçüncü sütunu r_3 ise r_1 ile r_2 ’nin çapraz çarpımından gelir, çünkü döndürme matrisinin üç sütunu birbirine diktir. Böylece 25 fotoğrafın her biri için ayrı bir (R, T) çifti elde edilir — bu, o fotoğrafın çekildiği andaki kamera-tahta açısı ve mesafesidir.

7. Aşama — Hepsini birlikte inceltiyoruz

5. ve 6. Aşamalardaki çözüm kapalı formdur ve distorsiyonu hiç hesaba katmaz; iyi bir başlangıç tahminidir, son cevap değildir. Son adımda Levenberg–Marquardt algoritması devreye girer ve tek bir hedefi küçültmeye çalışır: modelin öngördüğü piksel ile gerçekten ölçülen piksel arasındaki fark (Formül 8). Buna yeniden izdüşüm hatası (reprojection error) denir — tablolarda RMS olarak raporlanan sayı tam olarak budur. Bu optimizasyonda fx, fy, cx, cy, distorsiyon katsayıları ve her fotoğrafın kendi R, T’si hep birlikte, aynı anda ayarlanır. 25 fotoğraf $\times 48$ köşe = 1200 nokta bu tek hesapta birlikte kullanılır. Çıkan sonuç, tüm fotoğrafları en az hatayla açıklayan nihai fx, fy, cx, cy değerleridir.

Burada özellikle vurgulanması gereken bir nokta, 4. ve 6. aşamalarda ortaya çıkan R (döndürme) ve T (öteleme) çiftidir: checkerboard yöntemi, kameranın içsel parametrelerini (fx, fy, cx, cy) bulabilmek için -- istemeden de olsa -- her fotoğrafın çekildiği andaki kameranın tahtaya göre üç boyutlu konumunu (dışsal/extrinsic parametreler) da çözmek zorunda kalır; R ve T tam olarak bu yan üründür ve her fotoğrafta yeniden hesaplanır. İlerleyen bölümlerde anlatılan gimbal-lazer yönteminde ise böyle bir dışsal çözüme hiç ihtiyaç yoktur, çünkü lazerin açısal konumu (θ_x, θ_y) zaten gimbal tarafından doğrudan ve hassas biçimde bilinmektedir. İki yöntem arasındaki en temel yapısal fark budur: biri konumu görüntüden çıkarsar, diğeri konumu baştan bilir.

Neden Tek Fotoğraf Yetmiyor?

Formül düzeyinde bakıldığında tek bir fotoğraftaki 48 köşe bile yeterli denklem sayısını sağlar ($48 \times 2 = 96$ denklem, karşısında 4 bilinmeyen). Buna rağmen tek pozdan gelen veriyle güvenilir bir sonuç alınamaz. Sebepleri şunlardır:

- **Ölçek belirsizliği:** Tahtanın kameraya uzaklığı (T 'nin derinlik bileşeni) ile f_x birbirini telafi edebilir. "Tahta biraz daha yakın + f_x biraz daha küçük" ile "tahta biraz daha uzak + f_x biraz daha büyük" neredeyse aynı görüntüyü üretir; tek fotoğrafa bakarak hangisinin doğru olduğunu ayırt etmek mümkün değildir.
- **Sınırlı kapsama:** Tahta tek bir açı ve mesafede tutulduğunda görüntünün yalnızca bir bölgesini (genelde merkeze yakın kısmı) kaplar. Kenarlardaki distorsiyon davranışı hakkında hiçbir bilgi vermez — oysa c_x ve c_y tam olarak o bölgelerden gelen bilgiyle keskinleşir.
- **Gürültüye dayanıklılık:** Tek pozdaki ölçüm hataları (kâğıt kıvrılması, köşe tespiti hassasiyeti, hareket bulanıklığı) doğrudan sonuca yansır. Farklı pozlardan gelen veri bu hataları istatistiksel olarak dengeler.

Tahtayı farklı açılarda, farklı mesafelerde ve görüntünün farklı bölgelerinde (kenarlar dâhil) göstermek, her poza farklı bir geometri kazandırır. "Uzaklık-odak telafisi" gibi bir hile tek pozda işe yarası da başka geometrideki pozlarda tutmaz. Bu nedenle 25 pozun HEPSİNİ aynı anda tatmin eden tek bir f_x , f_y , c_x , c_y seti, artık bu belirsizliklere düşmeyen güvenilir bir sonuçtur.

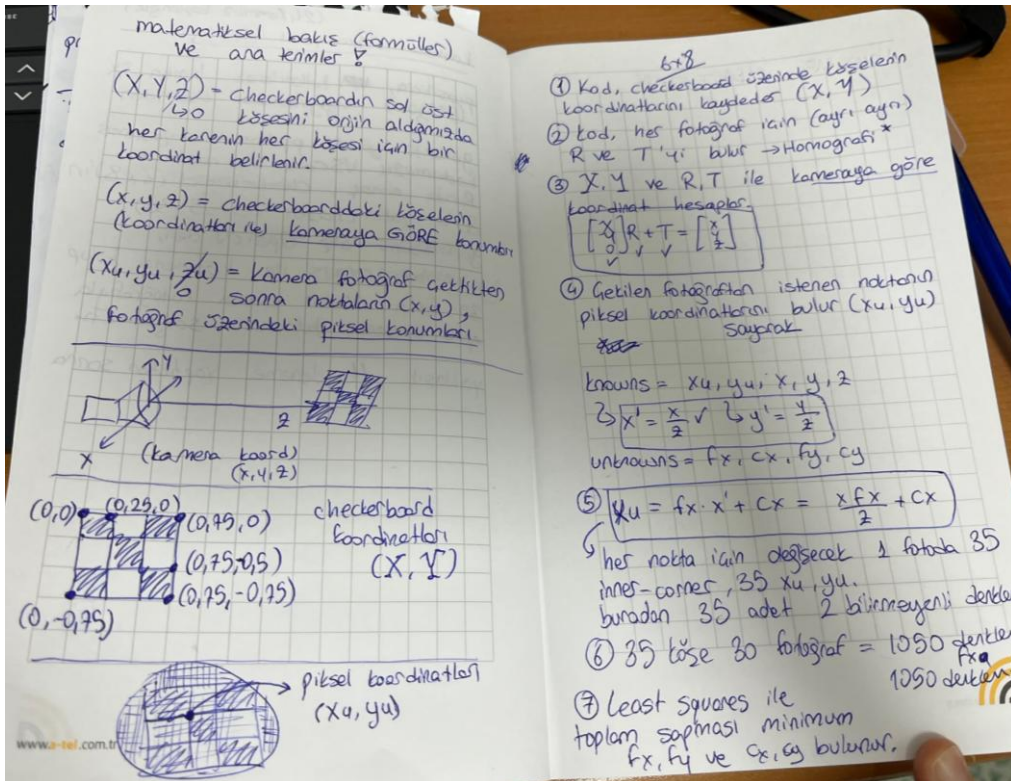
Özet Akış

Checkerboard geometrisi (X, Y, Z) biliniyor → farklı pozlarda fotoğraflar çekiliyor → her fotoğrafta köşe pikselleri (x_u, y_u) ölçülüyor → her fotoğraf için homografi H çözülüyor → tüm H 'lerden ortak K (f_x, f_y, c_x, c_y) çözülüyor → her fotoğrafın kendi R, T 'si geri çıkarılıyor → tüm veri birlikte, distorsiyon dâhil nonlinear optimizasyonla inceltiliyor → nihai $f_x, f_y, c_x, c_y, k_1, k_2 \dots$ elde ediliyor.

Çalışma Notları (El Notları)

Aşağıdaki sayfa, yukarıdaki aşamaların çalışılırken tutulan el notlarıdır.

(Şekil



Kalibrasyon aşamalarına ait el notları)

Erken dönemde, farklı çekim setleri arasında c_x , c_y değerlerinin belirli bir aralıkta salınım gösterdiğine dair genel bir gözlem not edilmişti; yukarıdaki 11 kalibrasyonluk sistematik çalışma bu gözlemi kesin verilerle doğrulamış ve salınımın rastgele değil, c_y ekseninde tutarlı yönlü bir kaymadan kaynaklandığını ortaya koymuştur (bkz. Bölüm 2.4).

Checkerboard kalibrasyonu bize kabaca bir f_x , f_y , c_x , c_y tahmini verdi; ancak yukarıda görüldüğü gibi, farklı çekim setleri arasında sonuçlar (özellikle c_y) belirgin, tek yönlü bir sapma gösteriyordu ve tek başına yeterince güvenilir ya da tekrarlanabilir bulunmadı. Bu noktada, kameranın görüş alanını çok daha kontrollü biçimde ve bilinen, hassas açısal referanslarla doğrudan taramamıza imkân verecek bir düzeneğe ihtiyaç duyduk: lazerimiz geldi. Lazeri kameranın önünde hassas biçimde konumlandırabilmek için de iki ekseninde hareket edebilen bir gimbal gerekiyordu -- aşağıdaki bölüm önce bu gimbalin ne olduğunu, nasıl çalıştığını ve nasıl seçildiğini anlatmaktadır.

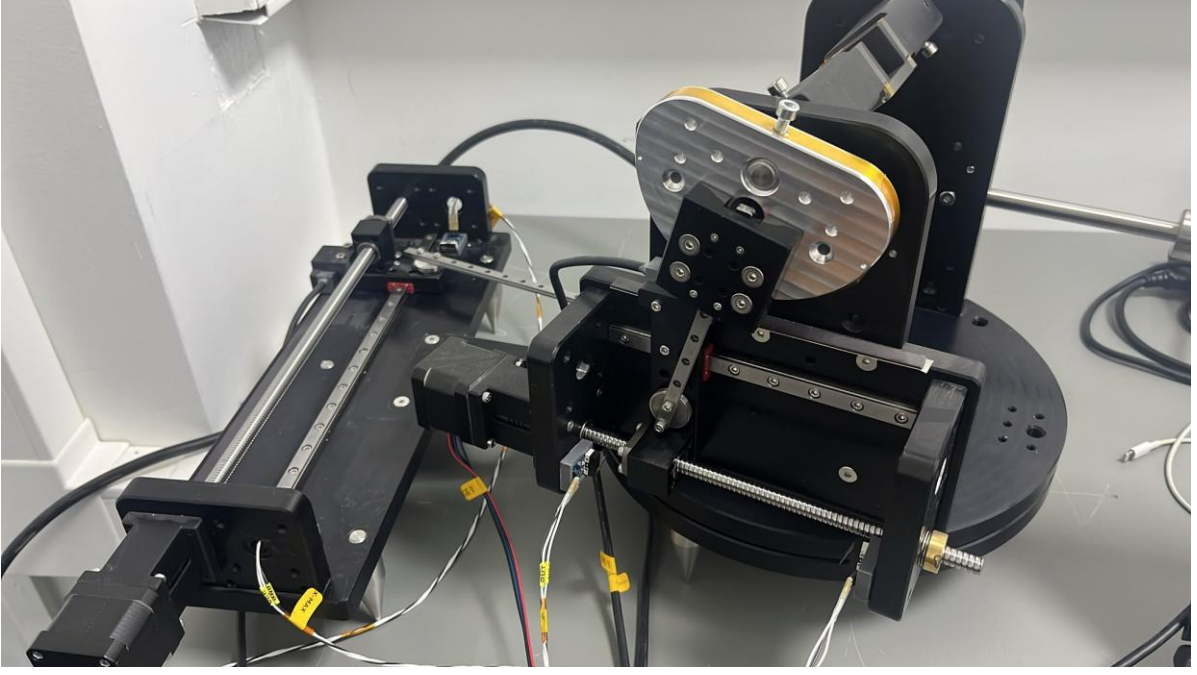
3. Gimbal: Çalışma Prensibi ve Donanım Seçimi

3.1 Gimbal Nedir? Temel Çalışma Prensibi

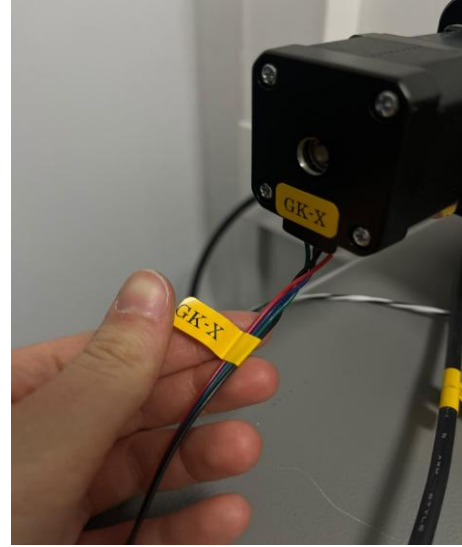
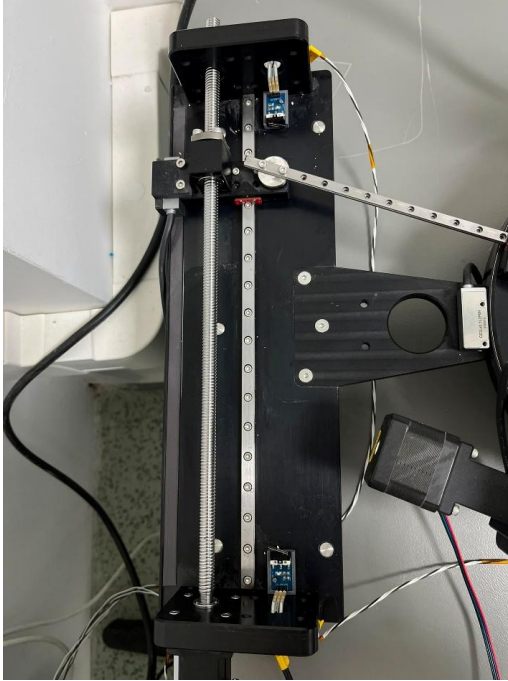
Laboratuvarımızda iki adet gimbal bulunmaktadır: bunları isimsel olarak “Kamera Gimbalı” ve “Lazer Gimbalı” olarak adlandırdık. Bu isimlendirme tamamen isimseldir, her iki gimbal da hem kameraya hem lazere takılabilecek şekilde üretilmiştir, aralarında fiziksel/yapısal bir fark yoktur. Aşağıdaki bölümlerde bahsedilen “kamera gimbalı” ve “lazer gimbalı” ifadeleri, yapılan hassasiyet testlerinde kullanılan iki ayrı fiziksel örneği tanımlamak için kullanılmıştır.

Gimbalin temel görevi, lazeri (veya kamerayı) sağa-sola taşımak değil, baktığı yönü değiştirmektir. Bu ayrım kritik önemdedir: bir eksen düz bir ray üzerinde 5 mm kaydırılırsa gövde 5 mm yer değiştirir fakat ışının doğrultusu aynı kalır; oysa gövde bir eksen etrafında döndürülürse konumu neredeyse sabit kalır ama ışının yönü değişir. Uzak bir hedefte asıl etkiyi yaratan bu *küçük açısal* değişimdir.

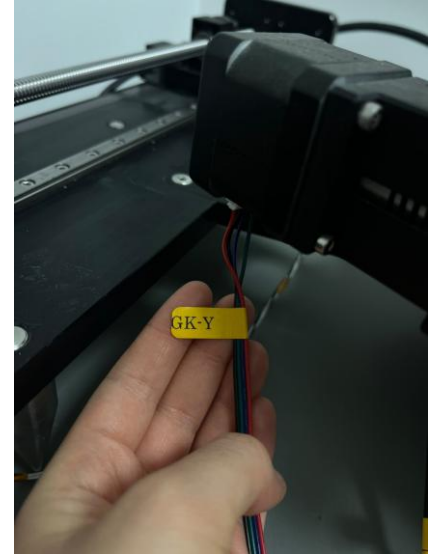
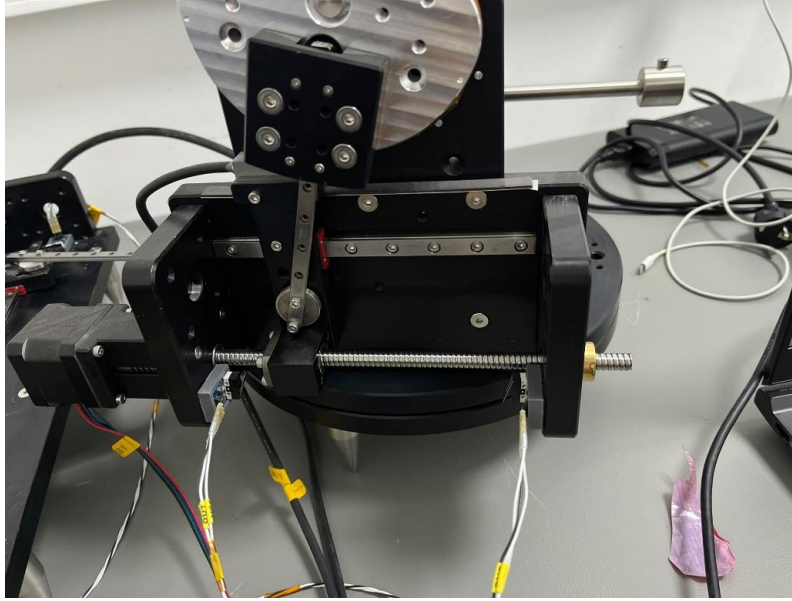
Sistemde kızak (lineer step motor + vidalı mil) ve yuvarlak tabla (dönen gövde) birlikte kullanılır: kızak çok küçük, kontrollü lineer hareketler üretir; bu hareket bağlantı kolu üzerinden büyük dairesel gövdeyi merkezi etrafında döndürür. Böylece girişte milimetre cinsinden doğrusal hareket, çıkışta derece/arcsecond cinsinden açısal harekete dönüşür.



(Resim 1.0. Kamera Gimballi)



(Resim 1.1. GK-X: Gimbal Kamera X eksenli hareket mekanizması, Azimut hareketi sağlar)



(Resim 1.2. GK-Y: Gimbal Kamera Y eksen hareket mekanizması, Elevasyon hareketi sağlar)

3.2 İki Gimbal ve “Kamera Gimbalı”nin Seçimi

Her iki gimbal için de azimut ve elevasyon eksenlerinde, otokollimatör ile sekiz ayrı ölçüm kampanyası yürütülmüştür (bkz. [Gimbal Hassasiyet Ölçüm Raporu](#)). Sonuçlar günün sonunda Kamera Gimbalı lehine karar verilmesine yol açmıştır. Kısaca:

- Azimut ekseninde Kamera Gimbalı teorik değere (2.880") en yakın ve en dengeli sonucu vermiştir: $+2.923'' \pm 0.447''$ / $-2.866'' \pm 0.382''$ (doğruluk hatası %1,5 altında).
- Lazer Gimbalı azimutta belirgin bir “yavaş-başlangıç” (run-in) sorunu göstermiştir: ilk 9–11 hareket teorik adımın çok altında kalmış, ancak sonrasında nominal değere oturmuştur. Kamera Gimbalinde bu bölge yalnızca 1 harekettir.
- Elevasyon ekseninde her iki gimbalde de benzer büyüklükte (%5–7) sistematik aşırı-dönme görülmüştür; bu nedenle bu eksen kıyaslamada belirleyici olmamıştır.

Özetle, azimut ekseninde daha hızlı kararlı rejime geçme ve daha yüksek doğruluk gösteren “**Kamera Gimbalı**” günlük kullanım için tercih edilmiştir.

3.3 Hassasiyet Parametrelerinin Yorumlanması

Bu kısım, hassasiyet raporundaki sayıların ne anlama geldiğini kısaca hatırlatmak amacıyla eklenmiştir. (Parametreler kamera gimbalı içindir.)

- **Çözünürlük:** Sistemin güvenilir şekilde üretebildiği en küçük açısal hareket.
(Azimutta ≈ 3 arcsec ($\sim 0.0008^\circ$), Elevasyonda ≈ 7.6 arcsec ($\sim 0.0021^\circ$) — minimum lineer komut olan 0.004 mm (azimut) ve 0.005 mm (elevasyon) karşılığı gerçek açısal hareket.)
- **Tekrarlanabilirlik:** Aynı komut tekrar tekrar verildiğinde sonuçların birbirine ne kadar yakın çıktığı (standart sapma ile ölçülür).
(Azimutta ± 0.38 – 0.45 arcsec (pozitif yönde $\pm 0.447''$, negatif yönde $\pm 0.382''$), Elevasyonda ± 0.76 – 1.18 arcsec (pozitif yönde $\pm 0.760''$, negatif yönde $\pm 1.181''$))
- **Doğruluk:** Ortalama ölçülen hareketin, hedeflenen teorik açığa ne kadar yakın olduğu (kalibrasyon kaynaklı sapma).
(Azimutta $+1.48$ (pozitif yön) / -0.48 (negatif yön) — teorik 2.880" hedefine göre. Elevasyonda $+5.71$ (pozitif yön) / $+5.17$ (negatif yön) — teorik 7.200" hedefine göre.)

- **Çapraz-eksen kuplajı:** Bir eksende hareket ederken diğer eksende istenmeden oluşan açısal kayma. (~%7–9 (her iki gimbalde ve her iki eksende benzer büyüklükte görülmüş; kaynağının gimbal mekanizmasından çok ayna-otokollimatör dizilim hizasızlığı olduğu düşünülüyor).)

Kısacası: çözünürlük “ne kadar küçük hareket edebiliyorum”, tekrarlanabilirlik “aynı komutu tekrarlıysam ne kadar tutarlıyım”, doğruluk ise “gerçek sonucum hedefe ne kadar yakın” sorularına cevap verir.

4. Sistem Mimarisi ve Donanım

Deney düzeneği üç ana alt sistemden oluşur: (i) görüntüleme için kullanılan endüstriyel kamera, (ii) lazer kaynağını iki eksende hassas biçimde yönlendiren gimbal ve hareket kontrol elektroniği, (iii) karanlık ortamda yalnızca lazer noktasının görünür olduğu bir ölçüm ortamı.

4.1 Kamera ve Lens: Prosilica GT2050 + Basler C11-1220-12M

Görüntüleme birimi olarak Allied Vision Prosilica GT2050 GigE Vision kamerası kullanılmıştır. Kamera, bilgisayara standart Ethernet (GigE) arayüzü üzerinden bağlanmakta ve Allied Vision'ın Vimba yazılım paketi ile (Python tarafında vmbpy kütüphanesi aracılığıyla) kontrol edilmektedir. Üretici veri sayfasına göre kamera, CMOSIS/ams CMV4000 model CMOS sensörünü kullanmakta; sensör çözünürlüğü 2048×2048 piksel (4.20 MP), piksel boyutu 5.50 µm × 5.50 µm ve sensör aktif alanı 11.26 × 11.26 mm'dir (optik format 1 inç). Sensör 12-bit derinlikte global shutter ile çalışmakta ve GigE arayüzü üzerinden 28 fps'e kadar kare hızı sunmaktadır. 2048×2048'lik çözünürlükte, ideal (kayma içermeyen) bir sistemde optik eksenin (cx, cy) ≈ (1024, 1024) piksel civarında düşmesi beklenir. Kameranın CMOS tabanlı olması, her pikselin bağımsız olarak okunabildiği, düşük güç tüketen ve yüksek kare hızına imkân veren bir mimari sağlar (bkz. Bölüm 5.5).

Kamera üzerinde kullanılan lens Basler C11-1220-12M (C-mount, Basler Premium serisi) sabit odaklı bir lenstir. Üretici veri sayfasına göre nominal odak uzaklığı $f = 12$ mm, diyafram aralığı F2.0–F16 (manuel), maksimum görüntü dairesi 1.1 inç (kameranın 1 inç'lik sensörüyle uyumlu) ve minimum çalışma mesafesi (min. working distance) 100 mm'dir. Bu son değer, Bölüm 10.5'te tartışılan lazer-kamera mesafesi meselesi açısından doğrudan önem taşımaktadır: lens, üretici tarafından yalnızca 100 mm ve ötesindeki mesafeler için net görüntü verecek şekilde tasarlanmıştır.

4.2 Gimbal ve Hareket Kontrol Sistemi

Lazer kaynağı, iki eksende (azimut / X ve elevasyon / Y) hareket edebilen bir gimbal üzerine sabitlenmiştir. Hareket kontrolü, kökeni 3D yazıcı elektroniğine dayanan bir denetleyici kart (Marlin tabanlı firmware çalıştıran bir MKS serisi kart) üzerinden, G-code komutlarıyla sağlanmaktadır. Python tarafında seri haberleşme (USB üzerinden sanal COM portu, 250000 baud) ile kart arasında G-code komutları gönderilip pozisyon geri bildirimleri (M114) okunmaktadır.

Kullanılan başlıca G-code komutları:

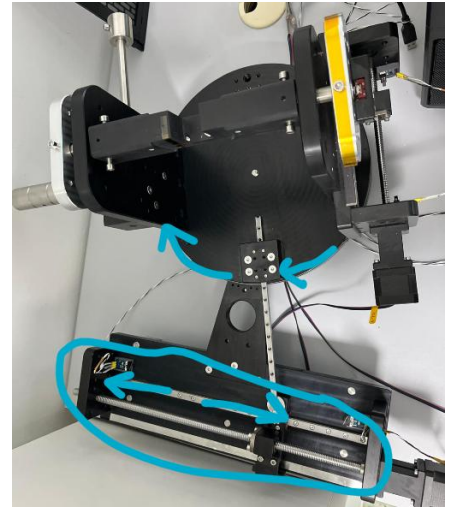
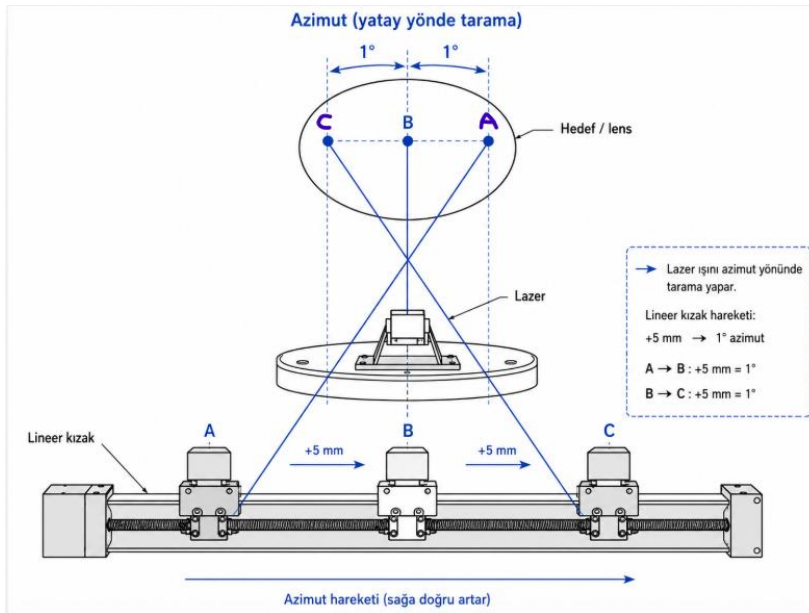
- G28 — Homing: eksenleri fiziksel limit switch'lere göre referans (sıfır) konumuna götürür.
- G90 / G91 — Mutlak / görelî konumlandırma modu.
- G1 X.. Y.. F.. — Belirtilen hızda (F, mm/dk) hedef konuma doğrusal hareket.
- M400 — Hareket kuyruğunun tamamen bitmesini bekle (senkronizasyon).
- M114 — Denetleyicinin bildirdiği güncel eksen konumunu oku (doğrulama).

Her step motorun bir G1 komutunu ne kadar doğru uyguladığı, motorun mekanik olarak "adım kaybetme" (lost steps) riskine açıktır: yazılım motorun kaç adım attığını varsayar, ancak motor bir engelle (örn. limit switch, mekanik sürtünme) karşılaşırsa gerçek adım sayısı farklı olabilir. Bu belirsizliği azaltmak için sistemde manyetik cetvel / rotary encoder tabanlı konum geri beslemesi ile kapalı çevrim (closed-loop) kontrol ilkesi değerlendirilmiştir: sensör gerçek konumu okur, denetleyici hedef ile gerçek konum arasındaki farkı (hata sinyalini) hesaplar ve motoru bu farkı kapatacak şekilde sürer. Bu, yalnızca komut gönderip "gitmiş olması gerekir" varsayımına dayanan açık çevrim (open-loop) kontrole kıyasla çok daha güvenilir bir konumlandırma sağlar.

4.2.1 Azimut ve Elevasyon: Tanımlar ve Teorik mm → Açık Formülleri

Azimut Nedir?

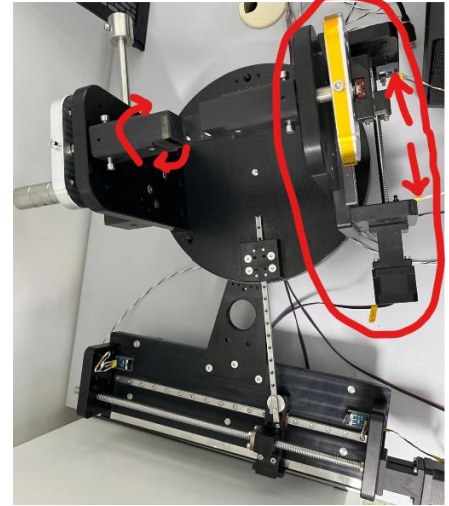
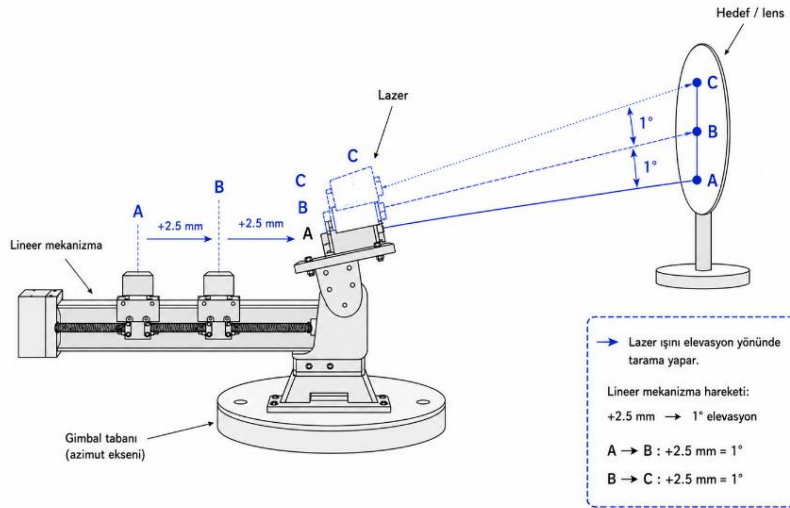
Azimut, lazerin/kameranın sağa-sola baktığı yönün değişimidir. Alttaki lineer kızak hareket eder, bağlantı kolu büyük yuvarlak gövdeyi döndürür; ilk doğrultu ile yeni doğrultu arasındaki yatay açı farkı azimut değişimidir.



Elevasyon Nedir?

Elevasyon, lazerin/kameranın yukarı-aşağı baktığı yönün değişimidir. İkinci döner mekanizma lazeri/kamerayı taşıyan kolu eğerek ışının yukarı veya aşağı yönelmesini sağlar.

Elevasyon (düşey yönde tarama)



Eksen Harfleri

Azimut ve elevasyon *fiziksel hareket isimleridir*. X, Y, Z ve A ise yazılımda eksenlere verilmiş etiketlerdir.

Kamera gimballi: X = azimut, Y = elevasyon.

Lazer gimballi: Z = azimut, A = elevasyon.

Teorik Formüller

AZİMUT

$$1^\circ / 5 \text{ mm} = 0.2^\circ/\text{mm} = 720 \text{ arcsec}/\text{mm}$$

$$\text{Açı (}^\circ\text{)} = \text{lineer hareket (mm)} \times 0.2$$

$$\text{Açı (arcsec)} = \text{lineer hareket (mm)} \times 720$$

ELEVASYON

$$1^\circ / 2.5 \text{ mm} = 0.4^\circ/\text{mm} = 1440 \text{ arcsec}/\text{mm}$$

$$\text{Açı (}^\circ\text{)} = \text{lineer hareket (mm)} \times 0.4$$

$$\text{Açı (arcsec)} = \text{lineer hareket (mm)} \times 1440$$

Raporda kullanılan iki temel kontrol hesabı: $0.004 \text{ mm} \times 720 = 2.880 \text{ arcsec}$ (azimut); $0.005 \text{ mm} \times 1440 = 7.200 \text{ arcsec}$ (elevasyon). Kamera Gimbalinin ölçülen değerleri bu teorik hedeflere en yakın sonuçlardır (bkz. Bölüm 3.2).

4.2.2 Doğrusal Hareketten Açısal Harekete Dönüşüm (DeneySEL Katsayılar)

Gimbal eksenleri motor sürücüleri tarafından doğrusal (mm) birimde konumlandırılmaktadır; ancak lazer ışınının kamera üzerindeki etkisi açısalıdır. Bu nedenle sistemde her eksen için ayrı bir deneysel dönüşüm katsayısı (DEG_PER_MM) tanımlanmıştır:

$$\theta_x = mm_x \cdot \text{DEG_PER_MM_X}, \quad \theta_y = mm_y \cdot \text{DEG_PER_MM_Y}$$

Kullanılan katsayılar $\text{DEG_PER_MM_X} = 1/5.0$ (yani 1 mm hareket $\approx 0.2^\circ$) ve $\text{DEG_PER_MM_Y} = 1/2.37$ (1 mm hareket $\approx 0.4219^\circ$) şeklindedir. Bu katsayılar, gimbal mekanizmasının kaldırıcı/pivot geometrisinden veya ayrı bir hassasiyet ölçümünden elde edilmiş sabitlerdir ve iki eksenin farklı mekanik oranlara sahip olabileceğini göstermektedir.

Bu iki katsayının birbirinden farklı olmasının önemli, ilk bakışta gözden kaçabilecek bir sonucu vardır: X ve Y eksenlerinde eşit mm mesafe taramak, eşit derece taramak anlamına gelmez. Erken geliştirme aşamasında bu ayrım gözetilmeden X ve Y için aynı adım mesafesi ($\text{step}_x = \text{step}_y$) kullanılmış; bu durum ilerleyen bölümde açıklanan dikdörtgen/kare-olmayan tarama alanı sorununa yol açmıştır. Düzeltme olarak her eksene kendi DEG_PER_MM katsayısına göre ayrı adım parametreleri atanmıştır.

Bunu somut bir örnekle görelim: X ekseninde 160 mm'lik bir toplam tarama mesafesi (span) ile Y ekseninde 60 mm'lik bir tarama mesafesi kullanıldığını varsayalım. Bu iki mesafe mm cinsinden birbirine yakın görünse de, açısal karşılıkları çok farklıdır:

$$\theta_x = \text{span}_x \times \text{DEG_PER_MM_X} = 160 \times 0.2 = 32^\circ$$

$$\theta_y = \text{span}_y \times \text{DEG_PER_MM_Y} = \frac{60}{2.37} \approx 25.32^\circ$$

Yani X ekseni 32° 'lik bir açısal alanı tararken, Y ekseni yalnızca 25.32° 'lik bir alanı taramaktadır — kameranın gördüğü tarama deseni, mm cinsinden "kare" olsa bile açısal olarak kare değildir ve dikdörtgen bir görüş alanına karşılık gelir. Bu, sistemin ilk denemelerinde gözlenen dikdörtgen (kareye benzemeyen) ölçüm desenlerinin başlıca nedenlerinden biridir (bkz. Bölüm 8.1).

Burada dikkat edilmesi gereken bir nokta, Bölüm 4.2.1'deki nominal/teorik elevasyon oranı ($1^\circ/2.5 \text{ mm}$) ile bu deneyde fiilen kullanılan deneysel DEG_PER_MM_Y katsayısının ($1/2.37$, yani $1^\circ/2.37 \text{ mm}$) birbirinden hafifçe farklı olmasıdır. Bu fark beklenmedik değildir: teorik oran gimbalin nominal kaldıraç geometrisinden hesaplanırken, deneysel katsayı sistemin gerçek/ölçülmüş tepkisine göre ayrıca kalibre edilmiştir ve bu raporun geri kalanında (grid tarama ve distorsiyon hesaplarında) kullanılan değer bu deneysel katsayıdır.

4.2.3 G-code Komut Seti ve Kontrol Zinciri

Yukarıdaki beş temel komuta (G28, G90/G91, G1, M400, M114) ek olarak, gimbal kontrol kodunda kullanılan diğer G-code/M-code komutları şunlardır:

- **F (feedrate)** — G1 komutunun içinde belirtilen hız değeridir; birim mm/dakikadır.
- **M503** — Kartın mevcut ayarlarını (firmware parametrelerini) rapor etmesini ister; hareket üretmez, bilgi amaçlıdır.
- **M17** — Step motor sürücülerini etkinleştirir (motorları enerjilendirir/kilitler).
- **M84 S0** — Motorların belirli bir süre hareketsizlik sonrası otomatik olarak kapanmasını (deaktif olmasını) engeller; S0 “asla kapanma” anlamındadır.

Bu komutların pratikte nasıl bir araya geldiğini göstermek amacıyla, gimballı HOME konumuna aldıktan sonra tanımlı bir eksenle tekrarlı hareket ettiren örnek bir Python betiği aşağıda verilmiştir:

```
import serial
import time
import math

# --- Seri port ayarları ---
PORT = 'COM10'      # Doğru port bilgisayarına göre değişecektir
BAUD = 250000        # Marlin çoğunlukla 250000, sende 115200 ise onu yaz
```

```

# Seri portu aç
ser = serial.Serial(PORT, BAUD, timeout=1)

ser.write(b'M503\n') #kartın firmware parametrelerini raporlar

#cihaz HOMEa geldikten sonra, komutları uygulamadan önce 40sn bekler
time.sleep(40)

#tekrardan G91 komutunu (göreceli hareket modu) cihaza hatırlatır! gözden kaçırmış olabilir
ser.write(b'G91\n')

#gönderme bufferındaki verilerin temizce USBye girmesini BEKLER
ser.flush()

# --> KODU girme vakti
for x in range(5):
    ser.write(b'G1 X10 F1000\n')
    #bir sonraki 0.8 kayma için HER SINGULAR 0.8mm kayışların bitmesini bekle
    ser.write(b'M400\n')
    ser.flush()

    #her adımı yaptım diye yaz
    print("%", x, " completed.")
    #each 8mm hareket sonrası 1 sn bekler
    time.sleep(1)

#5 adet 0.8mm kayma hareketinin TAMAMEN bitmesini bekle
ser.write(b'M400\n')
ser.flush()
#-----

time.sleep(2)

#bir başka hareket
ser.write(b'G1 X-10 F1050\n')
ser.write(b'M400\n')
ser.flush()
# Motorları aktif tut
ser.write(b'M17\n')

# Motorların otomatik olarak kapanmasını engelle
ser.write(b'M84 S0\n')
ser.flush()

#time.sleep(10) #10 sn sonra homea geri döner (kartın yazılımında var)

print("Gimbal son konumda bekliyor.")
print("Programı kapatmak için CTRL + C tuşlarına bas.")

try:
    while True:
        time.sleep(1)

except KeyboardInterrupt:
    print("Program kapatılıyor...")

finally:
    ser.close()

```

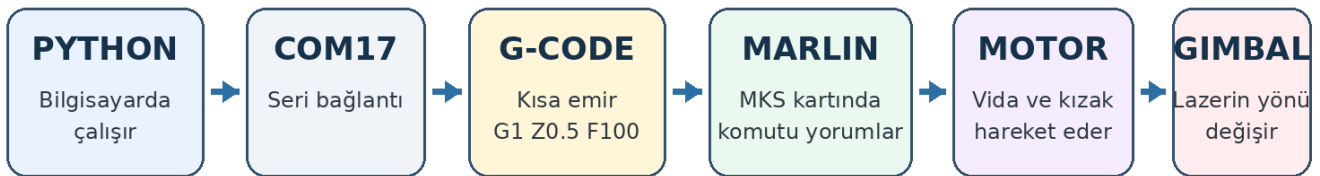
Bu betiğin işleyişi adım adım şu şekildedir:

- Bağlantı ve hazırlık: PORT ve BAUD tanımları ile serial.Serial(...) satırı, bilgisayar ile MKS kartı arasında seri bağlantıyı açar. time.sleep(#) kart resetlenip hazır olsun diye kısa bir bekleme sağlar.
- İlk mod ayarı: İlk G91 komutu göreceli hareket modunu ayarlar; hemen altındaki yorum satırındaki örnek G1 komutları (X, Y, Z, A) farklı eksenlerin nasıl hareket ettirileceğini gösteren, şu an devre dışı bırakılmış (# ile yorumlanmış) örneklerdir.
- HOME / bekleme kısmı: time.sleep(40) satırı, cihazın HOME konumuna gelmesi ve komutları uygulamaya başlamadan önce sistemin oturması için konan 40 saniyelik bekleme süresidir. Hemen ardından G91 tekrar gönderilerek göreceli mod hatırlatılır (kart gözden kaçırmış olabilir diye).
- Asıl hareket döngüsü: for x in range(10) döngüsü, X ekseninde 8 birimlik hareketi 10 kez tekrarlar; her hareketten sonra M400 ile o hareketin tam olarak bitmesi beklenir, sonra 1 saniye ara verilir ve ilerleme ekrana yazdırılır.
- Son konumlama: Döngü bittikten sonra M400 ile tüm hareketlerin kesin olarak tamamlanması beklenir, 2 saniye ara verilir, ardından X ve Y eksenlerinde tek seferlik ek bir hareket (G1 X15 Y20 F1050) gönderilir.
- Motorları tutma: M17 motorları aktif eder, M84 S0 motorların otomatik kapanmasını engelleyerek gimballi son konumunda sabit tutar.
- Sonsuz bekleme: try/while True/except KeyboardInterrupt bloğu, kullanıcı CTRL+C ile programı sonlandırana kadar bağlantıyı açık tutar; sonunda ser.close() ile port düzgünce kapatılır.

Marlin arcsecond hesabı yapmaz; sadece milimetre cinsinden verilen komutu motor hareketine çevirir. Zincir şu şekilde işler:

- PYTHON: Bilgisayarda çalışır. Motoru doğrudan döndürmez; G-code metnini hazırlar, seri porta yollar, kullanıcıdan veri alabilir ve akışı yönetir.
- COM PORTU: Bilgisayar ile MKS kontrol kartı arasındaki seri haberleşme kapısıdır (BAUD=250000 veri iletişim hızıdır). Port yanlışsa komut ya başka bir cihaza gider ya da bağlantı hiç açılmaz.
- G-CODE: Kartın anlayacağı kısa emir dilidir (G91, G1 X8 F1000, M400 gibi). Python kodunun tamamı G-code'a dönüşmez; Python yalnızca bu kısa metin komutlarını üretip gönderir.
- MARLIN: MKS kartı üzerinde çalışan firmware'dir. Gelen G-code'u okur, hangi eksenin ne kadar ve hangi hızda hareket etmesi gerektiğini çıkarır, motor sürücüsüne STEP/DIR darbeleri verir.
- MOTOR VE MEKANİK DÜZEN: Motor vidalı mili döndürür, kızak milimetre cinsinden ilerler, bağlantı kolu gimballi döndürür.
- GİMBAL: Sonuçta lazerin/kameranin yönü derece/arcsecond cinsinden değişir.

Marlin arcsecond hesabı yapmaz; milimetre komutunu motor hareketine çevirir.



Önemli Not — COM Portu Değişebilir

Bu kodda PORT = 'COM10' olarak tanımlanmıştır; ekran görüntülerindeki örnekte ise COM17 kullanılmıştır. COM port numarası, cihazın bağlandığı bilgisayara, USB girişine ve işletim sistemine göre değişebilir; her kurulumda Aygıt Yöneticisi'nden (Device Manager) doğru port numarası kontrol edilip koda göre güncellenmelidir.

Birimler karışması: Python ve G-code tarafında komut edilen lineer eksen değeri mm'dir; F değeri mm/dakika hızdır. Marlin'in çıkışı elektriksel motor adımlarıdır. Gimbalin fiziksel sonucu ise açıdır (derece/arcsecond). Bunlar aynı büyüklük değildir; zincirin farklı aşamalarında farklı birimler devreye girer.

4.2.4 Güvenli Hareket Sınırları (Safe Limits)

Gimbalin fiziksel olarak gidebileceği en uç konumlar, denetleyici kartın homing sırasında bulduğu referans (home) koordinatlarıyla sınırlıdır. Pronterface üzerinden gönderilen M114 komutuyla okunan home koordinatı $X \approx 135.17$ mm, $Y \approx 50.61$ mm olarak kaydedilmiştir (farklı oturumlarda ölçülen ondalık basamaklardaki küçük farklılıklar okuma/yuvarlama kaynaklıdır ve mekanik olarak aynı referans noktasını temsil eder). G91 (görelî konumlandırma) komutu verilerek bu fiziksel referans, yazılımsal olarak (0, 0) merkez kabul edilmiştir; böylece sonraki tüm hareketler bu merkeze göre + veya – yönde tanımlanır.

Tarama sırasında kullanılan güvenli hareket sınırı (SAFE_LIMIT), doğrudan fiziksel maksimum limitin kullanılmasıyla değil, kasıtlı olarak daha dar bir aralıkla belirlenmiştir. İlk uygulamada SAFE_LIMIT_X = 134, SAFE_LIMIT_Y = 50 (yani fiziksel sınıra oldukça yakın) kullanılmış; ancak bu değerlerin limit switch'lere yakın bölgelerde titreşim yarattığı ve lazer noktasının kameranın görüş alanından (FOV) çıkmasına neden olduğu gözlemlenmiştir. Bu nedenle sınırlar SAFE_LIMIT_X = 80, SAFE_LIMIT_Y = 30 değerlerine düşürülmüştür:

`SAFE_LIMIT_X = 80.0 # Güvenli sınır ±80 mm`

`SAFE_LIMIT_Y = 30.0 # Güvenli sınır ±30 mm`

Buradan toplam tarama mesafesi (span), merkezden her iki yöne gidilebilecek mesafenin iki katı olarak hesaplanır:

$$span_x = limit_x \times 2, \quad span_y = limit_y \times 2$$

SAFE_LIMIT_X = 80 için span_x = 160 mm, SAFE_LIMIT_Y = 30 için span_y = 60 mm elde edilir. Bu toplam mesafe, kullanıcının girdiği grid boyutuna (grid_x, grid_y) bölünerek her bir adımın kaç mm olacağı belirlenir:

$$step_x = \text{round_stepper}\left(\frac{span_x}{grid_x - 1}\right)$$

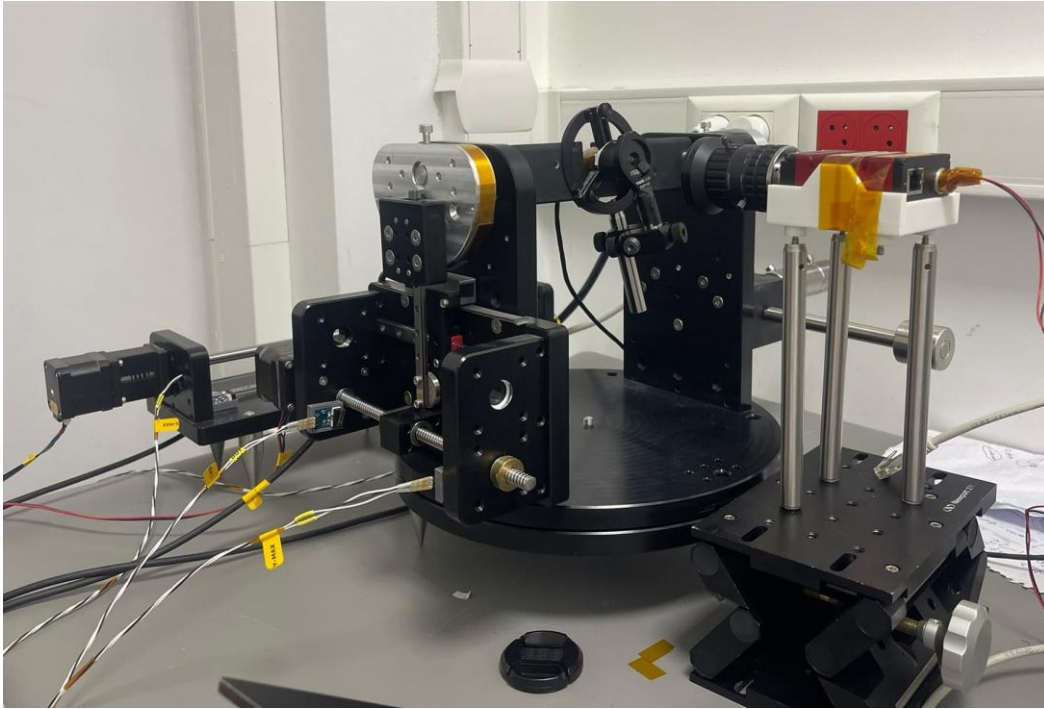
Örneğin 15×15'lik bir grid için step_x = 160/(15–1) ≈ 11.43 mm olur; bu değer daha sonra Bölüm 8.3'teki round_stepper fonksiyonuyla **meanik sistemin uygulayabileceği en yakın 0.02 mm'nin katına yuvarlanır**.

4.3 Lazer Kaynağı ve Karanlık Oda Ölçümü

Ölçümler karanlık bir ortamda, sahnede yalnızca lazer noktasının görünür olduğu koşullarda gerçekleştirilmiştir. Bu, görüntüdeki tek parlak bölgenin lazer noktası olmasını garanti ederek, ilerleyen bölümde açıklanan ağırlık merkezi (centroid) hesaplamasının güvenilirliğini artırır. Gimbal her hedef konuma ulaştıktan sonra, mekanik titreşimin sönmesi için kısa bir bekleme süresi uygulanmış; ardından fotoğraf çekilmiştir. Bu disiplin, hareket bitiminin mekanik olarak tam sükûnet anlamına gelmediği gözlemine dayanmaktadır — titreşim sönmeden alınan görüntüler, lazer merkezinin hatalı hesaplanmasına yol açabilmektedir.

4.4 Fiziksel Düzenek ve Montaj Zorlukları

Deney düzeneğinin fiziksel kurulumu, aşağıda özetlenen bileşenlerden oluşur ve süreç boyunca çözölen çeşitli montaj sorunlarını içermiştir.



Şekil 2.1 — Deney düzeneğinin genel görünümü: gimbal üzerine sabitlenmiş lazer + pinhole tertibatı (solda), lens ve Prosilica GT2050 kamera (sağda), kamera altında yükseklik/stabilite için kullanılan Lab Jack ve metal çubuklar.

Sistem; 1 lazer, 1 pinhole (Thorlabs FMP1/M mekanik pinhole tutucusu, 150 μm çaplı açıklık), gimbal, lens ve kameradan (Prosilica GT2050, CMV4000 CMOS sensörlü) oluşmaktadır. Kamerayı sabitlemek ve yükseklik ayarı yapmak için bir Lab Jack (makaslı kaldırma tablası) ve buna bağlı metal çubuklar kullanılmıştır (tasarımda 4 çubuk öngörölmüş olsa da montaj sırasında 3 çubukla çalışılmıştır).

Karşılaşılan başlıca montaj sorunları ve uygulanan geçici çözümler:

- Lazer sabitleme: Lazerin silindirik gövdesi, adaptörün açık kalan tutma bölümüne göre küçük kalmıştır; geçici çözüm olarak lazer köpökle sarılıp yuvaya oturtulmuştur. Bu çözüm arkadan gelecek bir kuvvetle lazerin yerinden oynamasına açık olduğundan kalıcı bir sabitleme aparatı ihtiyacı not edilmiştir.
- Pinhole hizalama: Pinhole, lazerin önüne takılmış; manuel yakınlaştır-uzaklaştır-kaydır müdahaleleriyle, Vimba yazılımından canlı görüntü izlenerek, yazılımın algılayabileceği parlaklık seviyesine ulaşılan kadar ayarlanmıştır. Bu seviyeye ulaşıldıktan sonra pinhole konumuna bir daha dokunulmamıştır (tekrarlanabilirliği korumak için).
- Yansıma (reflection) sorunu: Çekilen fotoğraflarda, lazer ışığının çevredeki metal parçalardan veya duvardan yansıyarak tekrar kamera lensine ulaştığı fark edilmiştir. Bu, sahnede "tek parlak nokta" varsayımını (Bölüm 4.3) bozan önemli bir güröltü kaynağıdır. Çözüm olarak pinhole çevresine optik ışınları kesen siyah bir bant (elektrik bandı değil, optik açıdan ışık geçirmeyen bir malzeme) sarılmıştır.
- Pinhole-lazer arasındaki boşluk: Pinhole ile lazer arasında kalan küçük boşluktan ışık sızarak ortamdan yansımaya ve güröltüye yol açmıştır; bu boşluk siyah kumaşla kapatılmıştır. Ayrıca pinhole açıklığının kenarında oluşan küçük bir aşınma da ışık sızıntısına neden olduğundan, buraya da siyah bant uygulanmıştır.

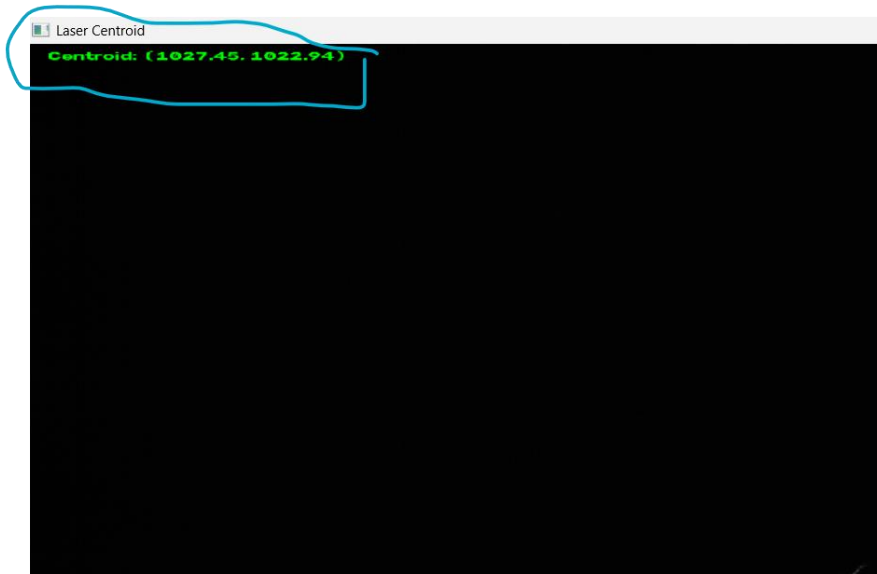


Şekil 2.2 — Thorlabs FMP1/M pinhole tutucusu (150 μm açıklık) ve lazer-pinhole arayüzündeki sızıntıyı önlemek için uygulanan bant/dolgu detayı.

- Gimbal/kamera mekanik sabitlemesi: Lab Jack'in çubuklarını sıkma saat yönünde (CW) çevirmeyi gerektirirken, kamera düzeneğine takılan çivilerin dişi saat yönünün tersine (CCW) sıkılması gerekmiştir ayrıca kamera düzeneğinin alt kısmındaki çivi metriği ile çubukların dişi girişi aynı metrik değildir. Bu uyumsuzluk nedeniyle yalnızca bir taraf (Lab Jack tarafı) tam olarak sıkılabilmiş; kamera tarafı **optimum stabilite gözetilerek kısmen sabitlenmiştir**. Ölçüm sırasında sistem hareketsiz kaldığından bu kısmi sabitleme şimdilik sorun yaratmamış, ancak ileride kalıcı bir düzeltme gerektiği not edilmiştir.

4.5 Lazer Merkezi Bulma (Center-Finding) Aracı

Distorsiyon tarama koduna başlamadan önce, lazer noktasının kamera görüntüsündeki başlangıç konumunun mümkün olduğunca sensör merkezine (1024, 1024) yakın olması istenmiştir. Bunun için ayrı bir canlı izleme aracı ("Laser Centroid") geliştirilmiştir: gimbal manuel olarak sağa-sola/yukarı-aşağı oynatılırken, kod her karede lazer noktasının centroid'ini (Bölüm 8.4'teki ağırlık merkezi yöntemiyle) hesaplayıp ekranın sol üst köşesinde canlı olarak (x, y) piksel koordinatı şeklinde göstermektedir.



Şekil 2.3 — Canlı centroid izleme aracının ekran görüntüsü: lazer noktası piksel (1027.45, 1022.94) konumunda okunmuştur — sensör merkezine (1024, 1024) çok yakın.

Nokta, mümkün olan en yakın konuma (1024, 1024) getirildikten sonra bu araç (kod) kapatılır ve distorsiyon hesaplama kodunu başlatmadan önce, kamerayı bir daha bu konumdan saptırmayız. Bu prosedür, Bölüm 9.4'teki merkez-hızlı ölçümün (Ölçüm 4) doğrudan kaynağıdır ve c_x , c_y 'nin ideal sensör merkezine bu kadar yakın çıkmasının teknik açıklamasıdır.

Center olarak bu noktayı alsak da distortion hesaplama kodu ilk lazer atışını sol üst noktadan yapacak şekilde tasarlanmıştır. Bknz:

```
x_positions = [round_stepper(-half_x + i * step_x) for i in range(grid_x)]
```

```
y_positions = [round_stepper(+half_y - j * step_y) for j in range(grid_y)]
```

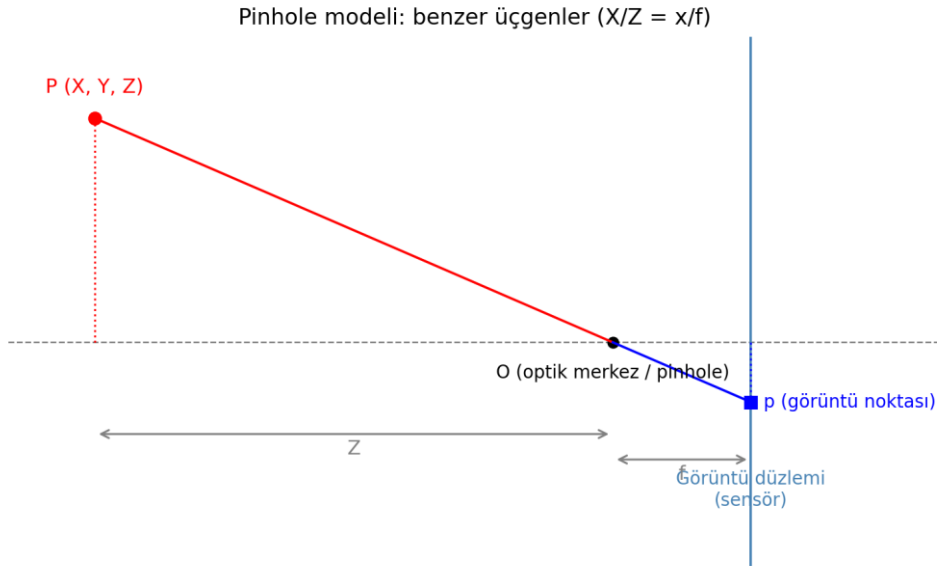
$x_positions[0] = -half_x \rightarrow$ en sol

$y_positions[0] = +half_y \rightarrow$ en üst

5. Optik Temelleri

5.1 Görüntü Oluşumu: Pinhole Modeli

Bir görüntüleme sisteminin en basit modeli, ışığın tek bir noktadan (pinhole / optik merkez) geçerek düz çizgiler halinde bir görüntü düzlemine ulaştığı ideal iğne-deliği (pinhole) modelidir. Bu modelde her ışın, uzaydaki kaynağını ve görüntü düzlemindeki izdüşümünü, optik merkezden geçen tek bir doğru üzerinde birleştirir.



Şekil 3.1 — Pinhole geometrisi: kamera merkezi O , dünyadaki nokta $P(X,Y,Z)$ ve görüntü düzlemindeki izdüşümü p arasındaki benzer üçgen ilişkisi.

Şekilde görüldüğü gibi, optik eksen boyunca uzanan büyük üçgen (yükseklik X , taban Z) ile görüntü düzlemindeki küçük üçgen (yükseklik x , taban f) benzerdir; çünkü ikisi de aynı ışın tarafından, aynı O noktasından kesilmektedir. Benzer üçgenlerde karşılıklı kenar oranları eşit olduğundan:

$$\frac{X}{Z} = \frac{x}{f} \Rightarrow x = f \cdot \frac{X}{Z}$$

Bu ilişki bir ezber kuralı değil, saf geometrik bir sonuçtur. X/Z oranı, ışının optik eksene göre yaptığı açının tanjantına eşittir ve ışının açısal konumunu temsil eder; cisim aynı X 'te sabit tutulup yalnızca Z (derinlik) artırılırsa X/Z küçülür ve görüntü noktası optik eksene yaklaşır.

5.2 Açı Kavramı: $\tan(\theta) = X/Z$

Gimbal-lazer deneyinde, lazer ışınının kamera optik eksenine göre yaptığı açı doğrudan bilinmektedir (Bölüm 4.2.2'deki DEG_PER_MM dönüşümünden). Bu açıyı görüntü düzlemindeki normalize koordinatlara bağlayan ilişki, yatay ve düşey eksenler için ayrı ayrı şu şekilde yazılır:

$$\tan(\theta_x) = \frac{X}{Z} \Rightarrow x' = \frac{X}{Z} = \tan(\theta_x)$$

$$\tan(\theta_y) = \frac{Y}{Z} \Rightarrow y' = \frac{Y}{Z} = \tan(\theta_y)$$

Burada x' ve y' , henüz piksel biriminde olmayan, boyutsuz "**normalize**" koordinatlardır — yalnızca ışının yönünü temsil ederler. Bu iki denklem, açısal referans (gimbalin bildirdiği θ_x, θ_y) ile kameranın göreceği geometrik izdüşüm arasındaki köprüyü kurar ve ilerleyen bölümlerdeki tüm distorsiyon ve kalibrasyon modelinin başlangıç noktasıdır.

5.3 İnce Mercek Denklemi ve Odaklama

Gerçek bir lens için f (odak uzaklığı) sabit bir fiziksel büyüklüktür ve yalnızca lensin kendisine bağlıdır. Buna karşılık, bir cismin lens tarafından oluşturduğu gerçek görüntünün lensin arkasında hangi mesafede odaklandığını gösteren görüntü mesafesi v , cismin uzaklığına (Z) bağlı olarak değişir. Bu ikisi arasındaki ilişki ince mercek denklemiyle verilir:

$$\frac{1}{f} = \frac{1}{Z} + \frac{1}{v}$$

$Z \rightarrow \infty$ olduğunda (çok uzak cisimler, örneğin yıldızlar) $1/Z \rightarrow 0$ olur ve $v \rightarrow f$ olur; yani sonsuzdan gelen ışınlar neredeyse paralel kabul edilebilir ve görüntü tam odak düzleminde oluşur. Bu, star tracker uygulaması için önemli bir sadeleştirmedir: yıldızlar pratik olarak sonsuz uzaklıkta kabul edilebileceğinden, görüntü düzlemi sabit $v \approx f$ 'de sabitlenebilir.

Önemli bir kavramsal ayırım: f ile v farklı şeylerdir. f lens değişmedikçe sabit kalırken, v cismin konumuna göre değişir. Örneğin $f = 10$ cm ve $Z = 30$ cm için $1/10 = 1/30 + 1/v$ denkleminde $v = 15$ cm bulunur; cisim

uzaklaştıkça v azalır f 'ye yaklaşır — ancak bu f 'nin değiştiği anlamına gelmez. Kameralarda sensör düzlemi sabit olduğundan, farklı mesafelerdeki cisimleri netlemek için genellikle objektif içindeki mercek grupları hareket ettirilerek görüntü sabit sensöre getirilir (otomatik odaklama); insan gözünde ise bu işlevi göz merceğinin kasların etkisiyle şekil değiştirmesi (akomodasyon) üstlenir.

5.4 Noktasal Yayılım Fonksiyonu (PSF) ve Netlik

Gerçek bir optik sistemde, tek bir noktasal ışık kaynağından (bu çalışmadaki lazer noktası gibi) gelen ışık, sensörde asla tek bir piksele düşmez; sonlu genişlikte bir yoğunluk dağılımına yayılır. Bu dağılıma noktasal yayılım fonksiyonu (Point Spread Function, PSF) denir. Sensör, odak düzleminden tam olarak f kadar uzaktaysa PSF en dar halini alır (en keskin görüntü); sensör bu konumdan yaklaştırılır veya uzaklaştırılırsa ışınlar tam odaklanamadan (ya fazla yakınsamış ya da ıraksamış halde) sensöre ulaşır ve PSF genişleyerek görüntü bulanıklaşır.

PSF'in sonlu genişliği, bu çalışmadaki centroid (ağırlık merkezi) yönteminin de temel gerekçesidir: lazer noktası birkaç piksele yayıldığından, "en parlak piksel" yaklaşımı yalnızca tam sayı piksel çözünürlüğünde bir sonuç verir; oysa yoğunluk-ağırlıklı ortalama, PSF'in şeklini kullanarak piksel-altı (subpixel) hassasiyette bir merkez konumu üretir (bkz. Bölüm 8.4).

Ayrıca, aşırı küçük bir açıklık (aperture/pinhole) kullanıldığında kırınım (diffraction) etkisiyle ışık artık dümdüz ilerlemeyip saçılmaya başlar; açıklık çok büyük olduğunda ise geometrik bulanıklık artar. Bu değiş tokuşu (trade-off) dengeleyen ideal pinhole çapı için literatürde şu yaklaşık bağıntı kullanılır:

$$d = 1.9 \times \sqrt{f \times \lambda}$$

burada f , açıklık-sensör mesafesi (pinhole kameralarda odak uzaklığına karşılık gelir) ve λ ışığın dalga boyudur. Bu ilişki doğrudan bu çalışmadaki lensli kamerada kullanılsa da, gerçek lenslerin neden "tek bir delik" yerine tercih edildiğini (aynı anda hem yüksek ışık toplama hem de yüksek çözünürlük sağlamak için) kavramsal olarak açıklar.

5.5 CCD / CMOS Sensör Çalışma Prensipleri

Sensör, gelen fotonları elektrik yüküne, ardından bir Analog-Dijital Çevirici (ADC) aracılığıyla sayısal parlaklık değerlerine dönüştüren elektronik bir çiptir. CCD (Charge-Coupled Device) mimarisinde her pikselin biriktirdiği yük, komşu piksellere aktarılarak tek bir çıkıştan sırayla okunur; bu düşük gürültülü ve yüksek kaliteli, ancak nispeten yavaş ve yüksek güç tüketen bir yöntemdir. CMOS (Complementary Metal-Oxide Semiconductor) mimarisinde ise her pikselin kendi transistörü bulunur ve pikseller birbirinden bağımsız olarak okunabilir; bu da yüksek hız, düşük güç tüketimi ve düşük maliyet sağlar. Bu çalışmada kullanılan Prosilica GT2050, CMV4000 model CMOS sensörünü kullanmaktadır ve günümüz endüstriyel kameralarının büyük çoğunluğu gibi CMOS tabanlıdır.

6. Kamera Matematiksel Modeli: İçsel Parametreler ve Distorsiyon

6.1 İçsel Parametreler ve K Matrisi

Gerçek bir kamerada görüntü koordinatları fiziksel mm cinsinden değil, piksel cinsinden ifade edilir. Fiziksel odak uzaklığı f (mm) ile piksel-cinsi odak uzaklığı f_x, f_y arasındaki bağlantı, sensördeki piksel boyutları p_x ve p_y (mm/piksel) aracılığıyla kurulur:

$$f_x = \frac{f}{p_x}, \quad f_y = \frac{f}{p_y}$$

f_x ve f_y , odak uzaklığının piksel ölçeğindeki karşılığıdır ve görüş açısının (perspektifin) ne kadar "güçlü" olduğunu belirler: f_x, f_y büyüdükçe görüş alanı daralır ve cisimler daha büyük (tele/zoom benzeri) görünür; küçüldükçe görüş alanı genişler (geniş açı). Bölüm 5.2'deki $x' = X/Z, y' = Y/Z$ ideal normalize koordinatları, piksel koordinatına şu şekilde ölçeklenir:

$$u = f_x \frac{X}{Z} + c_x, \quad v = f_y \frac{Y}{Z} + c_y$$

Buradaki c_x, c_y terimleri bir toplama sabiti (offset) olarak görünür ve kritik bir fiziksel anlam taşır: optik eksenin görüntü düzleminde hangi piksele düştüğünü gösterirler. Sensörün geometrik merkezi ile optik eksenin düştüğü nokta aynı olmak zorunda değildir — lensin sensöre montajındaki en ufak bir yatay/düşey kayma, ilk bakışta beklenmeyen c_x, c_y değerlerine yol açar. 2048×2048'lik bir sensörde geometrik merkez (1024, 1024) iken, gerçek c_x, c_y bu değerden onlarca-yüzlerce piksel kayabilir; bu durum bir hata değil, doğrudan ölçülmesi gereken bir üretim/montaj gerçeğidir.

Tüm bu ilişkiler, standart bilgisayarla görü literatüründe 3×3'lük içsel (intrinsic) matris K olarak toplu biçimde ifade edilir:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

K matrisinin normalize koordinatlarla ($[x', y', 1]$ vektörü) çarpımı, tam olarak yukarıdaki ölçekleme ilişkisini üretir. İlk satırın açık hâli:

$$x_p = f_x x + c_x$$

ikinci satırın açık hâli:

$$y_p = f_y y + c_y$$

üçüncü satır ise özdeşlik olarak $1 = 1$ verir; bu da homojen koordinat sisteminin (üçüncü bileşenin 1'e normalize edilmesi) matematiksel bir gerekliliğidir. Kısacası K matrisi, "3 boyutlu bir yön vektörünü 2 boyutlu piksel koordinatına taşıyan" tek, kompakt doğrusal dönüşümdür; kalibrasyonun asıl hedefi bu dört sayıyı (f_x , f_y , c_x , c_y) olabildiğince doğru kestirmektir.

6.2 Distorsiyon Modeli

K matrisi yalnızca ideal (distorsiyonsuz) bir lens için geçerlidir. Gerçek lensler, özellikle görüş alanının kenarlarına doğru, görüntüyü sistematik biçimde bükür. Bu bükülme iki bileşene ayrılır: merkeze göre simetrik olan radyal distorsiyon ve lensin sensöre tam paralel/ortalanmış oturmamasından kaynaklanan tanjansiyel distorsiyon.

6.2.1 Radyal Distorsiyon ve Taylor Açılımı

Normalize koordinatların (x' , y') optik eksene olan uzaklığı r ile ifade edilir:

$$r^2 = x'^2 + y'^2 = \left(\frac{X}{Z}\right)^2 + \left(\frac{Y}{Z}\right)^2$$

Burada r hâlâ boyutsuzdur (piksel değildir); merkezde $r = 0$, kenarlara gidildikçe r büyür. Gerçek distorsiyonu, r 'nin bir fonksiyonu $D(r)$ olarak düşünelim. Fiziksel olarak, merkezde ($r = 0$) distorsiyon olmamalıdır, yani $D(0) = 1$. $D(r)$ fonksiyonu, $r = 0$ civarında bir Taylor serisi ile açılabilir:

$$D(r) = D(0) + D'(0)r + \frac{1}{2}D''(0)r^2 + \frac{1}{6}D'''(0)r^3 + \frac{1}{24}D^{(4)}(0)r^4 + \dots$$

Lens ve sistem geometrik olarak (optik eksene göre) simetrik kabul edildiğinden, tek dereceli terimler (r , r^3 , r^5 , ...) fiziksel olarak sıfırlanır — çünkü distorsiyon merkeze göre yön bağımsız (izotropik) olmalıdır ve tek kuvvetler yön değiştiğinde işaret değiştirirdi, bu da simetriyle çelişirdi. Geriye yalnızca çift dereceli terimler kalır:

$$D(r) \approx 1 + k_1 r^2 + k_2 r^4 + k_3 r^6 + \dots$$

Buradaki k_1 , k_2 , k_3 katsayıları, $D(r)$ 'nin $r = 0$ noktasındaki türevlerinin ($D''(0)$, $D^{(4)}(0)$, ...) ölçekli hâlleridir:

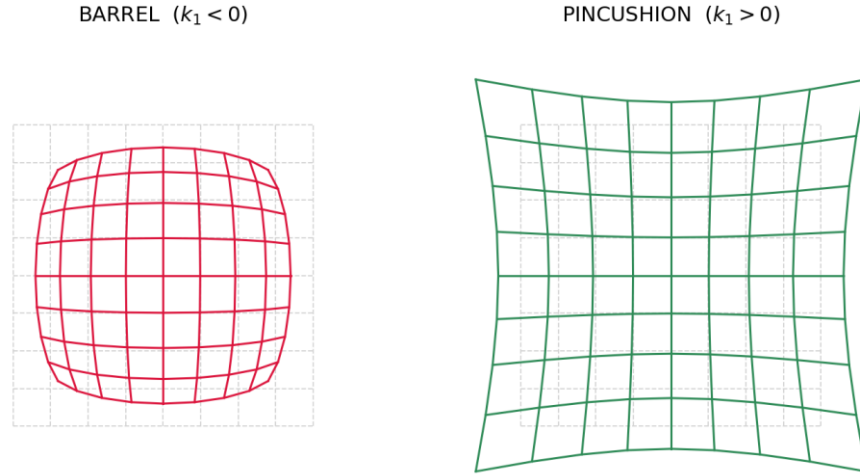
$$k_1 \approx \frac{1}{2}D''(0), \quad k_2 \approx \frac{1}{24}D^{(4)}(0)$$

Pratikte bu katsayılar analitik olarak değil, deneysel ölçümlere en iyi uyan değerler olarak (bkz. Bölüm 8.5, least-squares) bulunur. Bu çalışmada modelin r^2 ve r^4 terimleriyle (k_1 , k_2) sınırlı tutulması yeterli görülmüştür; daha yüksek dereceli terimler (k_3 ve ötesi) yalnızca çok güçlü distorsiyonlu geniş-açı lenslerde gerekli olur. Radyal distorsiyonun normalize koordinatlara uygulanması:

$$x_{radial} = x'(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 + \dots)$$

$$y_{radial} = y'(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 + \dots)$$

k_1 katsayısının işareti, distorsiyonun yönünü doğrudan belirler ve bu çalışmadaki en önemli yorum araçlarından biridir:



Şekil 4.1 — $k_1 < 0$ için barrel (fıçı) distorsiyonu: kenar noktalar merkeze doğru çekilir. $k_1 > 0$ için pincushion (yastık) distorsiyonu: kenar noktalar merkezden uzaklaşır.

- $k_1 < 0 \rightarrow (1 + k_1 r^2)$ çarpanı 1'den küçülür $\rightarrow x_{radial}$, y_{radial} merkeze doğru küçülür $\rightarrow$ BARREL (fıçı) distorsiyon.
- $k_1 > 0 \rightarrow (1 + k_1 r^2)$ çarpanı 1'den büyür $\rightarrow$ merkeze göre uzaklık artar $\rightarrow$ PINCUSHION (yastık) distorsiyon.

6.2.2 Tanjansiyel Distorsiyon (Brown–Conrady Modeli)

İdeal durumda distorsiyonun yalnızca radyal (merkeze göre simetrik) olması beklenir. Ancak lens elemanlarının optik eksene göre tam hizalı olmaması (decentering) veya sensöre göre hafif eğik durması (tilt) ek bir yanakayma (tanjansiyel kayma) yaratır. Bu etki, (x', y') 'ye bağlı şu genel polinom terimleriyle modellenir:

$$x_{tangential} = 2p_1 x' y' + p_2 (r^2 + 2x'^2)$$

$$y_{tangential} = p_1 (r^2 + 2y'^2) + 2p_2 x' y'$$

Bu form, Brown–Conrady distorsiyon modelinin klasik ifadesidir ve lens-ofset/eğim hatalarının birinci mertebe etkilerini en iyi yakalayan polinom terimleri olarak literatürde kabul görmüştür. p_1 , p_2 katsayıları da tıpkı k_1 , k_2 gibi deneysel olarak (least-squares ile) kestirilir.

6.2.3 Birleşik Distorsiyon Potansiyeli Yaklaşımı

Radyal ve tanjansiyel distorsiyon formülleri ilk bakışta birbirinden bağımsız, ayrı ayrı "uydurulmuş" ifadeler gibi görünebilir. Oysa ikisi de, optik aberasyon teorisinde standart olan tek bir yaklaşımdan — skaler bir distorsiyon potansiyeli $\Phi(x', y')$ 'nin gradyanından — türetilir. Bu bakış açısı hem formüllerin neden bu şekilde yazıldığını hem de distorsiyonun neden K matrisinin bir parçası olamayacağını netleştirir.

Genel fikir şudur: bir noktanın ideal (x', y') konumundan gerçek (bozulmuş) konumuna kayması ($\delta x, \delta y$), bir potansiyel fonksiyonun gradyanı olarak yazılabilir:

$$(\delta x, \delta y) = \nabla \Phi(x', y') = \left(\frac{\partial \Phi}{\partial x'}, \frac{\partial \Phi}{\partial y'} \right)$$

Radyal distorsiyon için Φ , dönele simetrik olmalıdır (her yöne aynı davranmalı) — yani yalnızca $r = \sqrt{x'^2 + y'^2}$ 'ye bağlı olmalıdır. En düşük dereceden simetrik terimler $r^4, r^6, \dots$ olduğundan:

$$\Phi_{radial}(x', y') = \frac{k_1}{4} r^4 + \frac{k_2}{6} r^6 + \dots$$

Bu potansiyelin x' 'e göre kısmi türevi (zincir kuralıyla) alındığında, Bölüm 6.2.1'deki radyal distorsiyon formülü doğrudan elde edilir:

$$\frac{\partial \Phi_{radial}}{\partial x'} = k_1 x' r^2 + k_2 x' r^4 + \dots = x' (k_1 r^2 + k_2 r^4 + \dots)$$

Tanjansiyel distorsiyon için ise soru şudur: "dönele simetriyi bozan, ama en düşük dereceden (en baskın) terim nedir?" Radyal terimler r 'nin çift kuvvetleriydi (simetri gereği); simetriyi bozan en düşük derece üçüncü derece (kübik) bir terimdir ve genel biçimi şöyledir:

$$\Phi_{tangential}(x', y') = r^2 (p_2 x' + p_1 y')$$

Burada (p_1, p_2), potansiyeldeki "tercih edilen yönü" belirleyen bir çift sabittir. Bu potansiyelin x' ve y' 'ye göre kısmi türevleri alındığında:

$$\frac{\partial \Phi_{tangential}}{\partial x'} = 3p_2 x'^2 + 2p_1 x' y' + p_2 y'^2 = 2p_1 x' y' + p_2 (r^2 + 2x'^2)$$

$$\frac{\partial \Phi_{tangential}}{\partial y'} = p_1 x'^2 + 2p_2 x' y' + 3p_1 y'^2 = p_1 (r^2 + 2y'^2) + 2p_2 x' y'$$

elde edilir — bu da tam olarak Bölüm 6.2.2'deki Brown–Conrady tanjansiyel distorsiyon formülleridir. Yani p_1, p_2 katsayılı ifadedeki "2" çarpanları ve " $r^2 + 2x'^2$ " gibi terimler rastgele seçilmiş değildir; $r^2(p_2 x' + p_1 y')$ potansiyelinin gradyanından matematiksel olarak zorunlu biçimde çıkar.

Bu yaklaşımın fiziksel yorumu da açıktır: radyal potansiyel dönele simetrik olduğundan (yalnızca r 'ye bağlı), ideal bir lensin optik eksene göre mükemmel simetrik olduğu varsayımıyla örtüşür. Tanjansiyel potansiyel ise dönele simetrik değildir — (p_1, p_2) çifti bir yön belirtir ve bu matematiksel asimetri, fiziksel olarak lensin optik eksene göre kayık durmasına (decentering) veya eğik oturmasına (tilt) karşılık gelir; yani sistemde artık "tercih edilen bir yön" vardır ve bu yön (p_1, p_2) vektörüyle kodlanmıştır.

Son olarak, bu potansiyel yaklaşımı K matrisiyle olan ilişkiyi de netleştirir: K matrisi saf doğrusal (lineer) bir dönüşümdür — yalnızca ölçekleme (f_x, f_y) ve öteleme (c_x, c_y) yapar. Distorsiyon ise $\nabla \Phi$ olarak, konuma (r 'ye)

bağlı, doğrusal olmayan bir bükülmedir; bir matris çarpımı bunu asla üretemez. Bu yüzden distorsiyon her zaman K'den önce, normalize koordinatlar (x' , y') üzerinde uygulanır (Bölüm 6.3); K yalnızca son adımda, zaten bozulmuş koordinatları piksele ölçekleyip kaydırır. Bu, Brown–Conrady modelinin 1966'da Duane C. Brown tarafından geliştirilmesinden (kökeni 1919'da A.E. Conrady'nin fotogrametrideki "decentering distortion" tanımına dayanır) bu yana neredeyse tüm modern kalibrasyon kütüphanelerinde (OpenCV, MATLAB Camera Calibration Toolbox dahil) standart olarak kullanılan formülasyondur.

6.3 Normalize → Distorsiyon → Piksel Dönüşüm Akışı

Yukarıdaki tüm adımlar bir araya getirildiğinde, bir gimbal açısından (θ_x , θ_y) nihai piksel koordinatına (x_u , y_u) giden tam işlem zinciri şu sırayla işler:

- 1) Normalize et: $x' = \tan(\theta_x)$, $y' = \tan(\theta_y)$ — açısal bilgi, boyutsuz yön vektörüne çevrilir.
- 2) Distorsiyon uygula: $(x', y') \rightarrow (x_{dist}, y_{dist})$ — radyal (k_1, k_2) ve tanjansiyel (p_1, p_2) terimler eklenir.
- 3) Piksele dönüştür: K matrisi (f_x, f_y, c_x, c_y) ile ölçekle ve kaydır.

$$x_u = f_x x_{dist} + c_x, \quad y_u = f_y y_{dist} + c_y$$

Bu akış, kalibrasyonun "ileri modelini" (forward model) tanımlar: bilinen bir gimbal açısından, modelin tahmin ettiği piksel koordinatını ($x_{u,pred}$, $y_{u,pred}$) üretir. Kalibrasyonun amacı, bu tahminin gerçek ölçülen piksel koordinatına (x_{meas} , y_{meas} — fotoğraftan centroid ile ölçülen) mümkün olduğunca yakın olmasını sağlayacak sekiz parametreyi ($f_x, f_y, c_x, c_y, k_1, k_2, p_1, p_2$) bulmaktır. Burada dikkat edilmesi gereken kritik bir nokta, x_u (ideal/model tahmini) ile x_{meas} 'in (ölçülen gerçek değer) kavramsal olarak farklı büyüklükler olduğudur; residual (kalan hata), bu ikisi arasındaki farktır.

7. İki Aşamalı Kalibrasyon Stratejisi

Bu çalışmada kasıtlı olarak iki farklı, birbirini doğrulayan yöntem art arda uygulanmıştır. Önemle belirtilmelidir ki bu iki aşama birbirinin sonucunu "miras almaz": chessboard ölçümünde belirli bir köşenin ne kadar kaydığı gözlemi, lazer taramasında aynı miktarda bir kaymanın olacağı anlamına gelmez. Bunun nedeni, iki yöntemin farklı fiziksel örnekleme stratejileri kullanmasıdır — chessboard yönteminde kamera farklı açı/mesafelerden sabit bir düzlemsel örüntüye bakarken, lazer yönteminde tek bir nokta kaynağı, kamera sabit dururken kontrollü açısal konumlara taşınır. Bu nedenle lazer taraması için ayrı, bağımsız bir kod ve model kurulumu (grid tarama + centroid + least-squares) kullanılmıştır.

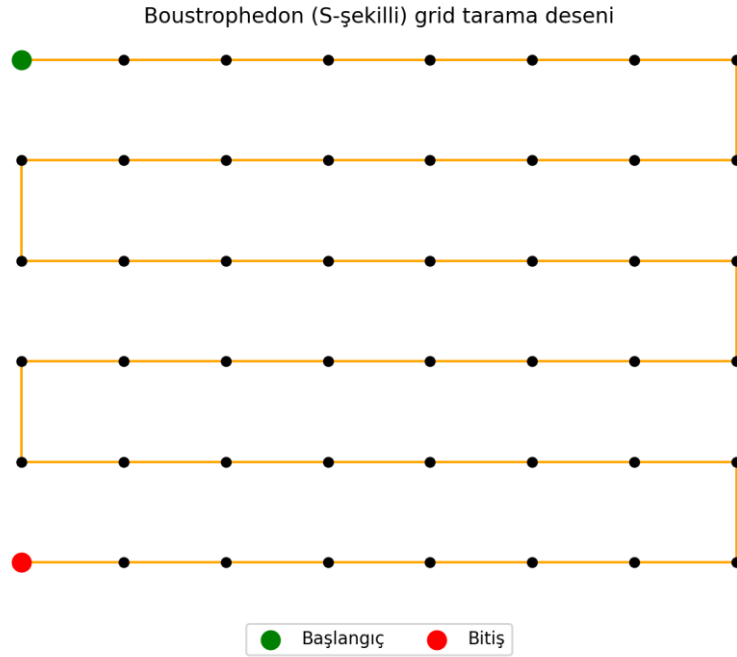
7.1 Aşama 2 — Gimbal + Lazer Tabanlı Distorsiyon Haritalama

İkinci aşamada, chessboard yerine tek noktasal bir lazer kaynağı kullanılmıştır. Bu yöntemin temel avantajı, her ölçüm noktasının konumunun (gimbal açısı olarak) çok yüksek hassasiyetle ve doğrudan bilinmesidir — bir baskılı desendeki köşe konumlarına kıyasla çok daha az belirsizlik içerir. Ayrıca gimbal, kameranın tüm görüş alanını sistematik bir ızgara (grid) deseniyle tarayarak distorsiyonun uzaysal dağılımını (merkeze yakın mı, kenarlarda mı güçlü) doğrudan görünür kılar. Bu yöntemin ayrıntıları Bölüm 8'de sunulmaktadır.

8. Deneysel Yöntem: Gimbal-Lazer Grid Taraması

8.1 Grid Tarama Deseni: Boustrophedon Yaklaşımı

Tarama, gimbalin X (azimut) ve Y (elevasyon) eksenlerinde $N \times N$ 'lik bir ızgara (örneğin 7×7 , 10×10 , 15×15 , 30×30) üzerinde hareket etmesiyle gerçekleştirilir. İlk uygulamada, gereksiz geri-dönüş hareketini azaltmak amacıyla klasik bir boustrophedon ("öküzün tarlayı sürmesi" deseninden esinlenen S-şekilli) tarama deseni kullanılmıştır: bir satır soldan sağa taranır, ardından bir alt satıra geçilip sağdan sola taranır — yön her satırda tersine döner ve satır başına dönerken fotoğraf çekilmez.



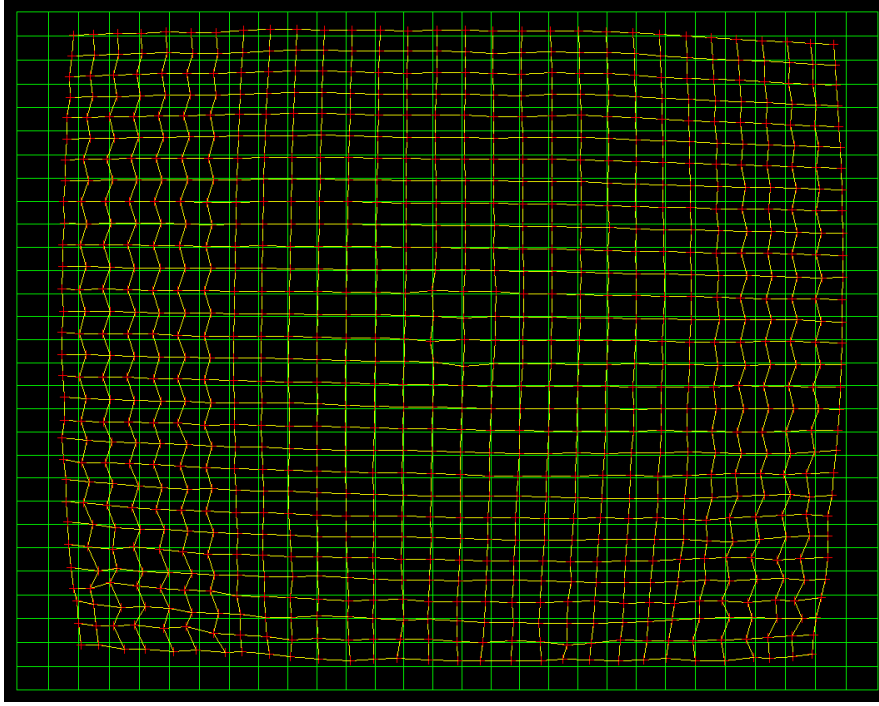
Şekil 6.1 — İlk uygulamadaki S-şekilli (boustrophedon) tarama deseni: her satırda yön değişir.

8.2 Zigzag Artefaktı: Neden Oldu, Neden Boustrophedon Yerine Tek Yönlü Taramaya Geçildi

Bu bölüm, ölçümlerdeki en görünür ve en öğretici sorunlardan birini — zigzag (testere dişi) deseni — adım adım, nedenini ve çözümünü sadeleştirerek açıklamaktadır.

8.2.1 Gözlenen Belirti

Ölçüm ızgaralarının orta bölgesi büyük ölçüde düzgün görünürken, özellikle ızgaranın sol ve sağ kenarlarında gözle görülür bir zigzag (satır satır sağa-sola sekme) deseni gözlemlenmiştir. Yani ideal ızgara ile üst üste bindirildiğinde, art arda gelen satırlar birbirine göre küçük yatay sıçramalarla kaymış gibi görünmektedir.



Şekil 6.2 — Zigzag artefaktı: ölçülen ızgara (sarı) çizgileri, ideal ızgaraya (yeşil) göre satır satır testere dişi şeklinde sapmaktadır.

8.2.2 İki Aday Açıklama

Bu belirtinin iki farklı, birbirinden bağımsız nedeni olabilirdi ve ikisi de başlangıçta makul görünüyordu:

- Aday 1 — Kameranın geniş açığı iyi algılayamaması: Lazer noktası kenarlara doğru hareket ettikçe, kameraya gelen ışının açısı büyür (optik eksenden uzaklaşır). Geniş açılı gelen ışınların kamerada gürültüyle birlikte bozulmuş/daha az net görünmesi mümkündür; bu durumda hata, tarama yönünden bağımsız, sadece "kenara ne kadar yakın olduğuna" bağlı bir gürültü olurdu.
- Aday 2 — Mekanik backlash (dişli/vida boşluğu): Gimbalin hareket mekanizmasında (vida-somun veya dişli tahriki), yön her değiştiğinde küçük bir mekanik boşluk kapanana kadar motor döner ama kızak/gimbal fiilen hareket etmez. Bu durumda hata, tarama yönü her değiştiğinde (yani boustrophedon deseninde her satırda) ortaya çıkar ve konumdan bağımsız, yalnızca "o an hangi yöne gidiliyor"a bağlıdır.

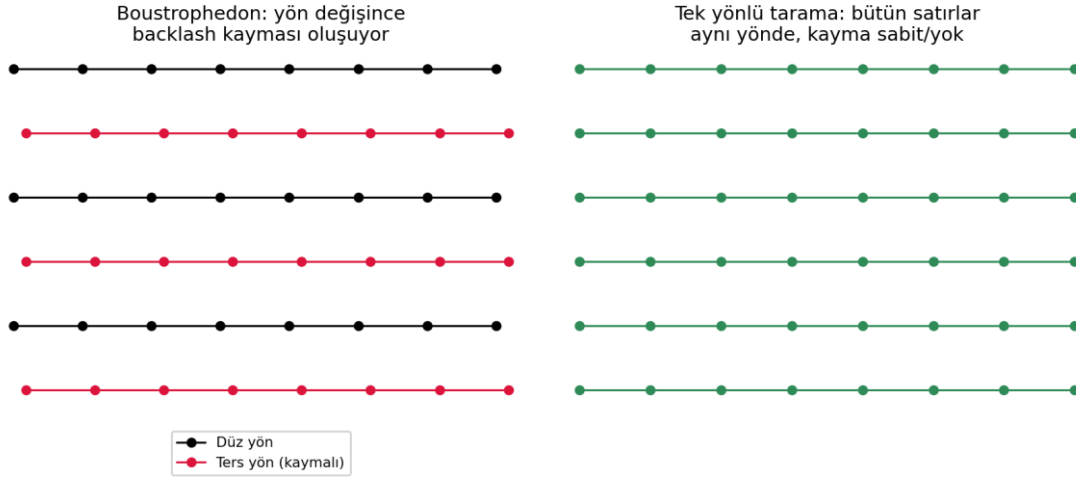
Bu iki hipotezi birbirinden ayırmanın en basit yolu bir kontrollü deney kurmaktır: eğer sorun Aday 1 (geniş açı/gürültü) ise, tarama yönü değiştirilse bile zigzag deseni değişmeden kalmalıdır — çünkü sorun kenara yakınlıkla ilgilidir, yönle değil. Eğer sorun Aday 2 (backlash) ise, tarama deseni yön değiştirmeyecek şekilde ayarlanırsa zigzag tamamen kaybolmalıdır.

8.2.3 Boustrophedon Neden Zigzag Yaratıyor

Orijinal tarama deseni boustrophedon'du: 1. satır soldan sağa taranıyor, 2. satır sağdan sola, 3. satır tekrar soldan sağa — yani gimbal her satırda yön değiştiriyordu (bkz. Şekil 6.1). Şimdi gimbalin hareket mekanizmasını bir vida-somun sistemi gibi düşünün: vidayı bir yöne çevirirken önce dişliler arasındaki ufak boşluk kapanır, sonra somun gerçekten hareket etmeye başlar. Bu boşluk kapanma süresi boyunca motor "döndüğünü" (komut gönderildiğini) sanır ama kızak henüz kıpırdamamıştır.

Soldan sağa giden bir satırda bu boşluk zaten önceki hareketle kapanmış durumdadır, dolayısıyla o satırdaki tüm noktalar güvenilir biçimde konumlanır. Ama bir sonraki satıra geçerken yön tersine döndüğü (sağdan sola) anda, boşluk yeniden açılır ve motor birkaç adım "boşa" döner; bu satırdaki tüm noktalar, komut edilen

konumdan sistematik olarak kaymış (gerçekte biraz daha az hareket etmiş) olarak kaydedilir. Bir sonraki satır tekrar soldan sağa gittiğinde bu kayma büyük ölçüde ortadan kalkar. Sonuç: bir satır doğru, bir sonraki satır kaymış, bir sonraki tekrar doğru — üst üste çizildiğinde bu tam olarak bir zigzag (testere dişi) deseni üretir.

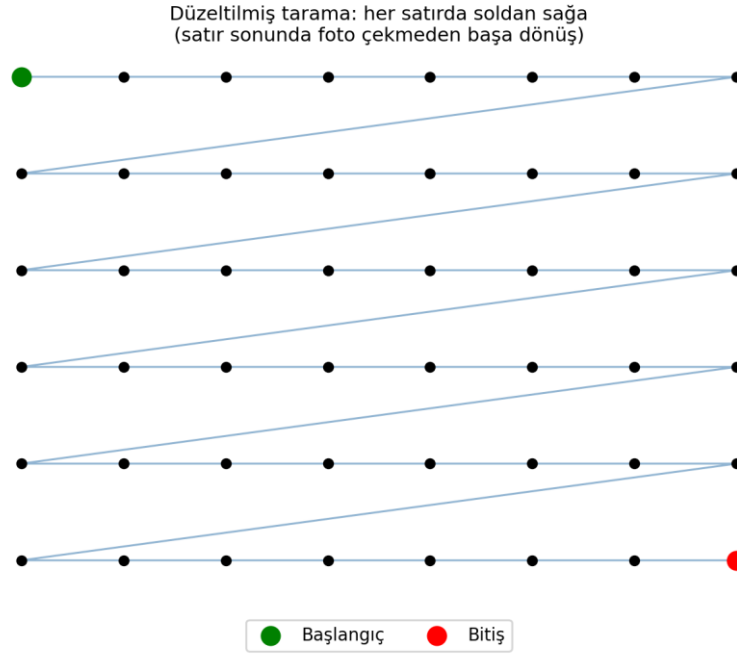


Şekil 6.3 — Solda: boustrophedon taramada yön değiştiren (kırmızı) satırlar backlash nedeniyle sabit bir miktar kaymaktadır — bu, üst üste bakıldığında zigzag deseni yaratır. Sağda: tek yönlü taramada tüm satırlar aynı yönde hareket ettiğinden kayma (varsa) her satırda aynıdır ve zigzag oluşmaz.

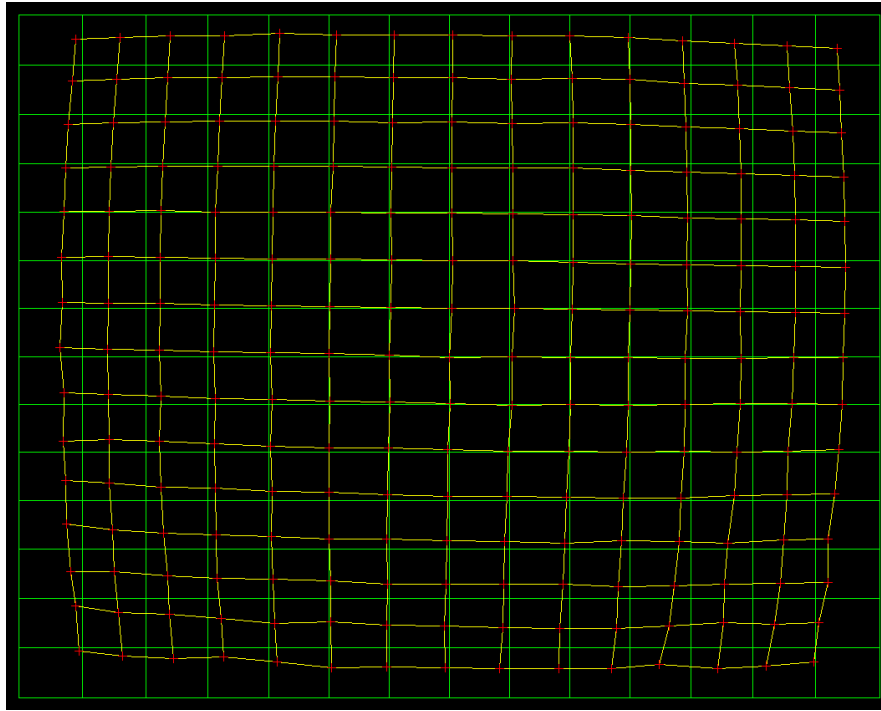
Kenarlarda daha belirgin görünmesinin nedeni ise şudur: satırın başlangıç ve bitiş noktaları (yani ızgaranın sol ve sağ kenarları), yön değişiminin hemen öncesinde veya sonrasındaki noktalardır — dolayısıyla backlash kaymasının etkisi tam da oradaki noktalarda en taze/en az "telafi edilmiş" haldedir. Orta bölgedeki noktalara gelindiğinde ise gimbal zaten stabil biçimde hareket ettiğinden hata görece küçüktür.

8.2.4 Uygulanan Çözüm ve Sonucun Doğrulaması

Çözüm olarak tarama deseni tek yönlü (unidirectional) hâle getirilmiştir: her satır yalnızca soldan sağa taranır; satır sonunda fotoğraf çekilmeden gimbal başlangıç sütununa (en sola) geri döner ve bir alt satırdan yeniden soldan sağa taramaya başlanır. Bu sayede ölçüm anlarının tamamı — istisnasız — aynı hareket yönünde gerçekleşir ve yön-değişimi kaynaklı backlash hatası, ölçülen noktalar arasında bir daha hiç oluşmaz (satır başına dönüş sırasında oluşan backlash ise ölçüm alınmadan önce hareketin tamamlanmasıyla "yutulur", çünkü bir sonraki satırın ilk noktası da yine soldan sağa giderken ölçülür).



Şekil 6.4 — Düzeltilmiş tek yönlü tarama deseni: her satır soldan sağa, satır sonunda ölçüm yapılmadan başa dönülür.



Şekil 6.5 — Tek yönlü tarama sonrası aynı bölgede ölçülen ızgara: zigzag artefaktı tamamen ortadan kalkmış, düzgün bir barrel-tipi distorsiyon eğrisi ortaya çıkmıştır.

Bu düzeltmenin ardından zigzag deseninin tamamen kaybolması ve RMS hatasının ~ 38 kat iyileşmesi (Bölüm 9.3 $\rightarrow$ 9.4, 426.96 px $\rightarrow$ 11.29 px), Bölüm 8.2.2'deki iki adaydan Aday 2'nin (mekanik backlash) baskın neden olduğunu deneysel olarak doğrulamaktadır — çünkü sorun yalnızca tarama yönü sabitlenerek çözülmüştür, kameranın geniş açı algılama performansında herhangi bir değişiklik yapılmamıştır. Bununla birlikte Aday 1'in (geniş açıda azalan ölçüm kalitesi) tamamen etkisiz olduğu da söylenemez; düzeltme sonrası ölçümlerde bile kenar bölgelerdeki noktaların merkez bölgeye göre biraz daha fazla saçılım göstermesi (bkz. Ölçüm 3, Bölüm 9.3), bu ikincil etkinin hâlâ küçük bir katkısı olabileceğini düşündürmektedir; bu, ileride ayrı bir deneyle (örneğin sabit yönde ama farklı ışık şiddetlerinde ölçüm tekrarlanarak) test edilebilir.

8.3 Adım Çözünürlüğü ve Yuvarlama Mantığı

Sistemde kullanılan adım çözünürlüğü STEP_RES = 0.02 mm olarak seçilmiştir. Bu seçimin nedeni, mekanik hassasiyet raporunda azimut eksenini için 0.004 mm, elevasyon eksenini için 0.005 mm'lik lineer adım büyüklükleri verilmiş olmasıdır; iki eksenin ortak (her ikisiyle de uyumlu) çalışabileceği bir çözünürlük olarak 0.02 mm seçilmiştir. Yazılımın hesapladığı teorik hareket miktarı, doğrudan sürücüye gönderilmeden önce bu ortak çözünürlüğe yuvarlanır:

$$x_{yeni} = \text{round}\left(\frac{x}{\Delta}\right) \cdot \Delta$$

Burada x istenen hareket miktarını, $\Delta = 0.02$ mm ise seçilen ortak adım çözünürlüğünü gösterir. Örneğin $x = 0.005$ mm için $x/\Delta = 0.25$ olur ve $\text{round}(0.25) = 0$ sonucunu verir; dolayısıyla teorik olarak planlanan 0.005 mm'lik hareket, mekanik sistemin ortak çözünürlüğü nedeniyle fiilen 0 mm olarak uygulanır. Bu yuvarlama mantığı, yazılım tarafında hesaplanan "ideal" hareketi mekanik sistemin gerçekte uygulayabileceği hareketle uyumlu hâle getirmek için zorunludur; aksi hâlde denetleyiciye gönderilen komutlarla gerçek fiziksel konum arasında sistematik bir tutarsızlık oluşur.

8.4 Subpixel Centroid (Ağırlık Merkezi) Tespiti

Bölüm 5.4'te açıklandığı gibi, sensöre düşen lazer noktası PSF nedeniyle birkaç piksele yayılır ve her pikselde farklı bir parlaklık (yoğunluk) değeri okunur. Lazer noktasının merkezi, yalnızca en parlak pikseli seçmek yerine, tüm ilgili piksellerin konumlarının parlaklıklarına göre ağırlıklandırılmış ortalaması alınarak (centroid yöntemi) hesaplanır:

$$x_c = \frac{\sum_i x_i \cdot I_i}{\sum_i I_i}, \quad y_c = \frac{\sum_i y_i \cdot I_i}{\sum_i I_i}$$

Burada I_i , i . pikseldeki ölçülen yoğunluk (parlaklık) değeridir. Bu yöntem, tam sayı piksel konumlarıyla sınırlı olmayan, ondalıklı (örneğin 248.51 piksel gibi) bir merkez konumu üretir; bu da tek piksellik (± 1 px) belirsizlik yerine çok daha ince bir konum hassasiyeti sağlar. Star tracker gibi açısal ölçüm hassasiyetinin doğrudan sistemin nihai performansını belirlediği bir uygulamada, bu subpixel hassasiyeti kritik önemdedir.

8.5 Model Kurma ve Least-Squares Optimizasyonu

Her grid noktası için, gimbalin bildirdiği açısal konum (θ_x, θ_y) ile o konumda çekilen fotoğraftan centroid yöntemiyle ölçülen piksel konumu (x_{meas}, y_{meas}) bir araya getirilerek bir veri kümesi oluşturulur. Amaç, Bölüm 6.3'teki ileri model kullanılarak tahmin edilen piksel konumu (x_{pred}, y_{pred}) ile gerçek ölçüm arasındaki farkı — kalan hatayı (residual) — tüm veri noktaları üzerinde toplamda minimize eden sekiz parametreyi ($f_x, f_y, c_x, c_y, k_1, k_2, p_1, p_2$) bulmaktır:

$$r(\theta) = [x_{meas} - x_{pred}, y_{meas} - y_{pred}, \dots]$$

Bu residual'lerin karelerinin toplamı, minimize edilecek maliyet fonksiyonunu oluşturur:

$$J(\theta) = \sum_{i=1}^n r_i(\theta)^2$$

Bu doğrusal olmayan en küçük kareler (nonlinear least-squares) problemi, Python'da SciPy kütüphanesinin `scipy.optimize.least_squares` işleviyle sayısal olarak çözülmüştür. Çözüm süreci şu adımlardan oluşur:

- 1) Başlangıç tahmini verilir: $\theta_0 = [f_x, f_y, c_x, c_y, k_1, k_2, p_1, p_2]$.
- 2) Her parametre kombinasyonu için, modelden tüm grid noktaları için (x_u, y_u) tahmini hesaplanır.
- 3) Residual vektörü $[x_{meas} - x_{pred}, y_{meas} - y_{pred}, \dots]$ oluşturulur.
- 4) Amaç fonksiyonu $J(\theta) = \sum r_i(\theta)^2$ minimize edilecek şekilde parametreler yinelemeli olarak güncellenir.
- 5) Çıktı olarak `result.x = [f_x, f_y, c_x, c_y, k_1, k_2, p_1, p_2]` elde edilir.

Optimizasyonun kalitesini değerlendirmek için kök ortalama kare hatası (RMS) hesaplanır:

$$RMS = \sqrt{\frac{1}{2N} \sum_{i=1}^N [(x_{meas,i} - x_{pred,i})^2 + (y_{meas,i} - y_{pred,i})^2]}$$

Burada N , lazer noktası (grid noktası) sayısıdır ve RMS birimi pikseldir (px). RMS ne kadar düşükse, modelin ölçülen verilere o kadar iyi uyduğu, dolayısıyla kestirilen parametrelerin o kadar güvenilir olduğu söylenebilir. Bununla birlikte düşük RMS tek başına yeterli değildir: parametrelerin fiziksel olarak anlamlı olup olmadığı (örneğin f_x, f_y 'nin makul bir odak uzaklığı-piksel boyutu oranına karşılık gelmesi, c_x, c_y 'nin sensör sınırları içinde kalması) ayrıca değerlendirilmelidir; aksi hâlde sayısal olarak "iyi uyan" ama fiziksel olarak anlamsız bir çözüme (overfitting benzeri bir duruma) ulaşılabilir.

8.6 Genel Distorsiyon Parametre Bulma Algoritması (Özet Akış)

Bölüm 8.5'teki optimizasyon sürecini, uygulamadaki en yalın hâliyle şu 7 adımlık akışta özetlemek mümkündür. Gimbal durağan durumdayken bilinen bir (x_i, y_i) açısal konumundan lazer ışığı gönderilir; ışık lensten geçip kameraya ulaşır ve bir fotoğraf çekilir; bu fotoğraftan centroid yöntemiyle gerçek (ve lens nedeniyle doğası gereği distorsiyonlu) piksel koordinatı $(x_{u,i}, y_{u,i})$ ölçülür. Yazılım, henüz bilinmeyen bir parametre seti $(f_x, f_y, c_x, c_y, k_1, k_2, p_1, p_2)$ "uydurur" ve bu parametrelerle, aynı (x_i, y_i) açısal konumu için modelin tahmin ettiği piksel koordinatını $(x_{u,model,i}, y_{u,model,i})$ hesaplar:

$$x_{u,model,i} = f_x \cdot x'_i + c_x, \quad y_{u,model,i} = f_y \cdot y'_i + c_y$$

Buradaki x'_i ve y'_i — radyal ve tanjansiyel distorsiyonun birlikte uygulandığı ara değerler — açıkça yazıldığında:

$$x'_i = x_i (1 + k_1 r_i^2 + k_2 r_i^4) + 2p_1 x_i y_i + p_2 (r_i^2 + 2x_i^2)$$

$$y'_i = y_i(1 + k_1 r_i^2 + k_2 r_i^4) + p_1(r_i^2 + 2y_i^2) + 2p_2 x_i y_i$$

Ardından, ölçülen gerçek piksel koordinatı ile modelin tahmini arasındaki fark (hata) hesaplanır:

$$e_{x,i} = x_{u,i} - x_{u,model,i}, \quad e_{y,i} = y_{u,i} - y_{u,model,i}$$

Tüm grid noktaları için bu hataların kareleri toplamı (toplam hata) minimize edilecek şekilde parametre seti güncellenir:

$$\min_{\theta} \sum_i [(x_{u,i} - x_{u,model,i})^2 + (y_{u,i} - y_{u,model,i})^2]$$

Bu minimizasyon sırasında optimizasyon algoritması, her parametrenin toplam hatayı ne yönde ve ne kadar etkilediğini (kısmi türevlerini) değerlendirir:

$$\frac{\partial E}{\partial f_x}, \quad \frac{\partial E}{\partial c_x}, \quad \dots, \quad \frac{\partial E}{\partial p_1}, \quad \dots$$

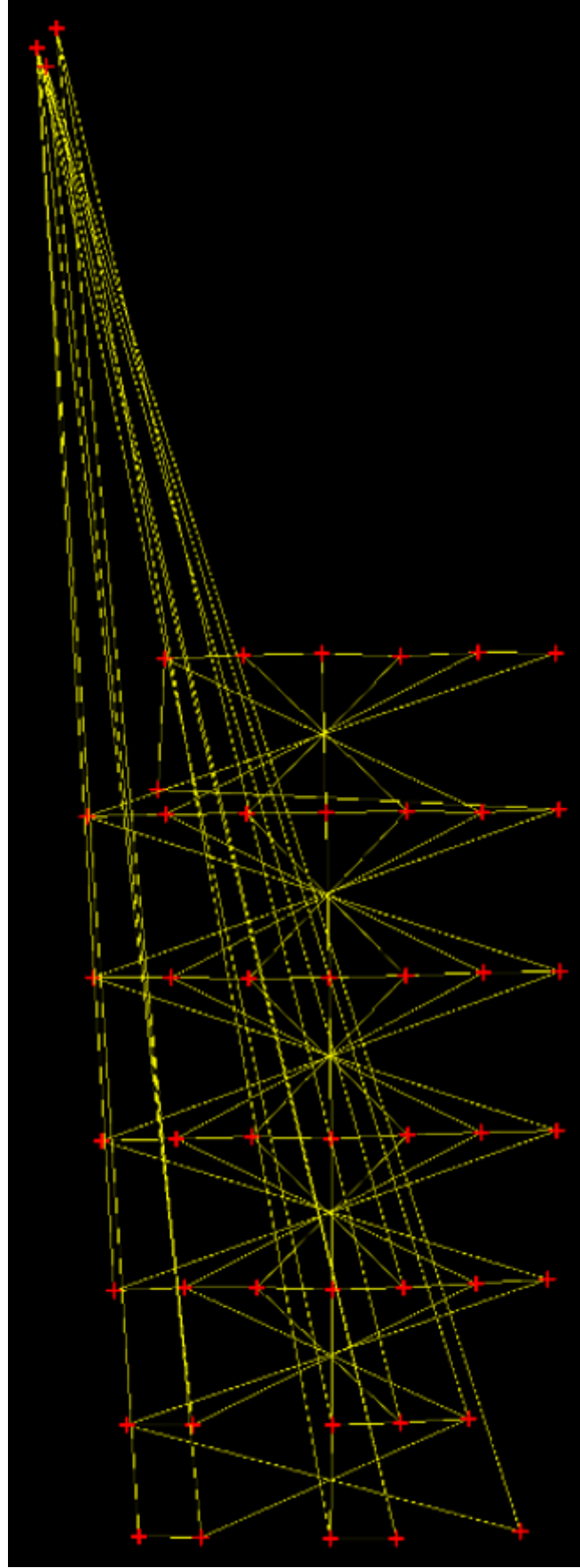
ve f_x , f_y , c_x , c_y , k_1 , k_2 , p_1 , p_2 parametrelerini, toplam hatayı en aza indirecek yönde küçük adımlarla günceller. Bu süreç yakınsayana kadar (hata daha fazla küçülmeyene kadar) tekrarlanır; sonunda elde edilen parametre seti, ölçülen verilere en iyi uyan (en düşük RMS'i veren) kalibrasyon sonucudur. Özetle: yazılım kameranın "nasıl gördüğünü" baştan bilmez; yalnızca bilinen açısal konumlar ile ölçülen piksel konumları arasındaki tutarsızlığı defalarca ölçüp azaltarak, kamerayı tarif eden parametreleri geriye doğru çıkarım yoluyla bulur.

9. Ölçüm Sonuçları: Kronolojik Gelişim

Bu bölüm, çalışma boyunca elde edilen sekiz ölçüm setini, elde edildikleri sıraya göre kronolojik olarak sunmaktadır. Her ölçüm, bir öncekinde tespit edilen belirli bir sorunun (eksen ölçek farkı, yansıma, mekanik sabitleme, kamera açısı, tarama yönü gibi) giderilmesinin doğrudan sonucu olarak ortaya çıkmıştır; bu nedenle ölçümler tek tek sonuçlardan çok, sistemin adım adım olgunlaşmasının bir kaydı olarak okunmalıdır. Her ölçüm için ızgara görselleştirmesinde kırmızı/sarı çizgiler gerçek ölçülen (centroid ile bulunan) piksel konumlarını, beyaz/yeşil çizgiler ise ideal (distorsiyonsuz) referans ızgarayı göstermektedir.

Genel eğilimi özetlemek gerekirse: RMS hata $245.68 \rightarrow 27.28 \rightarrow 426.96 \rightarrow 11.29 \rightarrow 10.64 \rightarrow 4.89 \rightarrow 4.85 \rightarrow 4.94$ piksel şeklinde, dalgalanmalarla birlikte genel olarak düşüş göstermiştir (üçüncü ölçümdeki ani sıçrama, o ölçüme özgü ayrı bir yansıma sorunundan kaynaklanmaktadır ve Bölüm 9.3'te ayrıca açıklanmaktadır). Son üç ölçüm (9.6–9.8), birbirine çok yakın parametrelerle ($f_x \approx 1800$, $k_1 \approx -1.2$) tekrarlanabilir biçimde yakınsayarak sistemin artık yorumlanabilir, olgun bir çalışma bandına ulaştığını göstermektedir.

9.1 Ölçüm 1 — Başlangıç Tarama Geometrisi (Yelpaze Deseni)



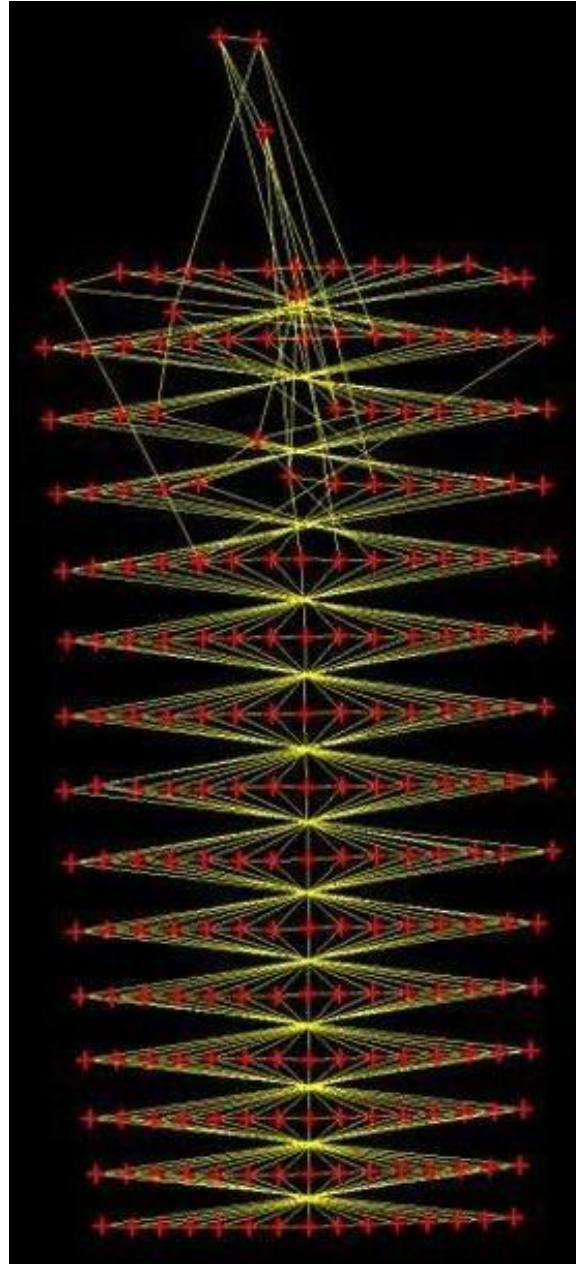
Şekil 7.1 — İlk tarama denemesinde elde edilen, yelpaze (fan) benzeri düzensiz nokta örgüsü.

fx	2821.687	fy	-2567.920
cx	1167.522	cy	1174.282

k1	-26.7277	k2	215.086
p1	0.3886	p2	-0.2931
RMS (px)	245.6768		

Bu ilk deneme, sistemin henüz dengeli bir tarama geometrisi üretmediği başlangıç aşamasını temsil eder. Nokta örgüsünün yelpaze biçiminde açılması, X ve Y eksenlerinin henüz eşdeğer bir açısal ölçekle ele alınmadığını (bkz. Bölüm 4.2.2) ve referans geometrinin daha kurulamadığını göstermektedir. $k1 = -26.73$, $k2 = 215.09$ gibi fiziksel olarak makul olmayan büyüklükteki katsayılar, optimizasyonun burada gerçek bir lens karakteri değil, geometrik olarak tutarsız veriye zorla uydurulmuş anlamsız bir çözüm ürettiğine işaret eder. Bu ölçüm yalnızca metodolojik gelişim sürecini belgelemek amacıyla rapora dahil edilmiştir.

9.2 Ölçüm 2 — Eksenler Arası Açısal Eşitsizliğin Etkisi



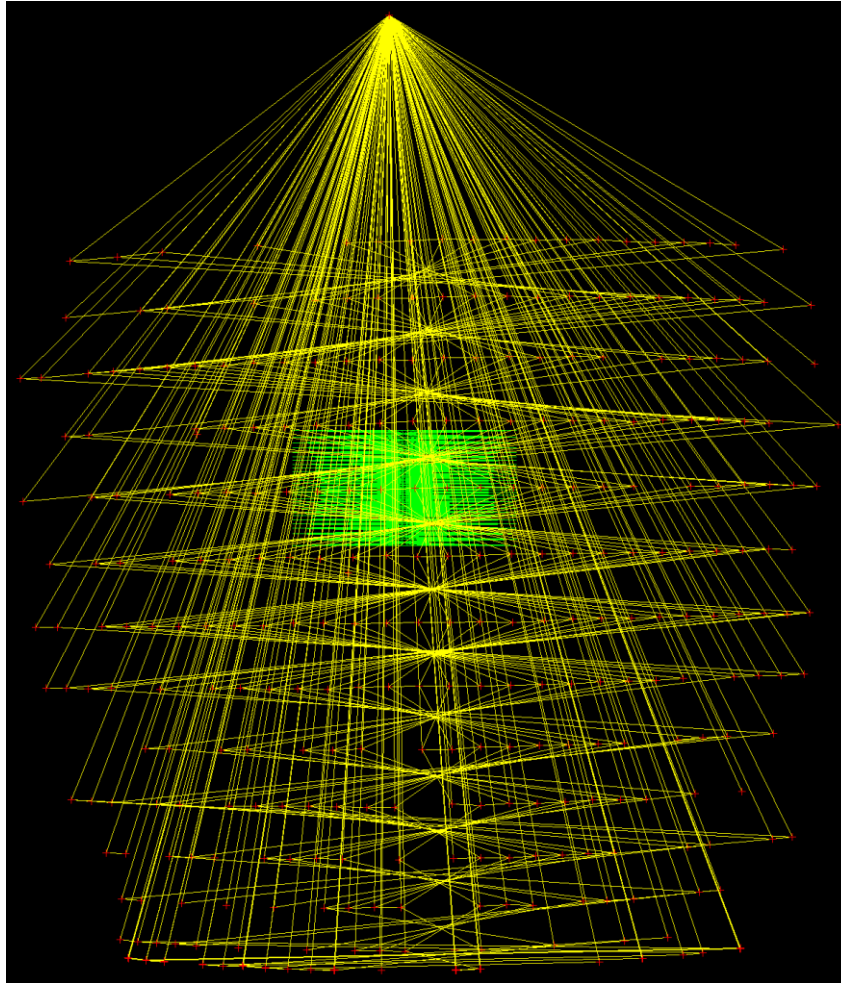
Şekil 7.2 — X ve Y eksenlerinin aynı açısal alanı taramadığı, dikey yönde aşırı uzamış erken dönem çıktı.

fx	1592.349	fy	-1635.001
cx	1145.169	cy	1180.139

k1	2.2957	k2	-48.8065
p1	0.0888	p2	0.0117
RMS (px)	27.2781		

Bu ölçümde X ve Y eksenleri henüz ayrı DEG_PER_MM katsayılarıyla ele alınmadığından (bkz. Bölüm 4.2.2), sistem aynı mm hareketin her iki ekseninde de aynı açısalları tarayacağını varsaymıştır. Gerçekte Y eksenine (DEG_PER_MM_Y = 1/2.37) X eksenine (DEG_PER_MM_X = 1/5.0) göre birim mm başına yaklaşık 2.1 kat daha fazla açısalları ürettiğinden, ızgara dikey yönde aşırı uzamış, dikdörtgen değil neredeyse bir sütun gibi görünmüştür. Bu gözlem doğrudan eksenlere ayrı açısalları ölçek katsayıları atanması kararına yol açmıştır.

9.3 Ölçüm 3 — Yansıma Etkisinin Baskın Olduğu Ölçüm



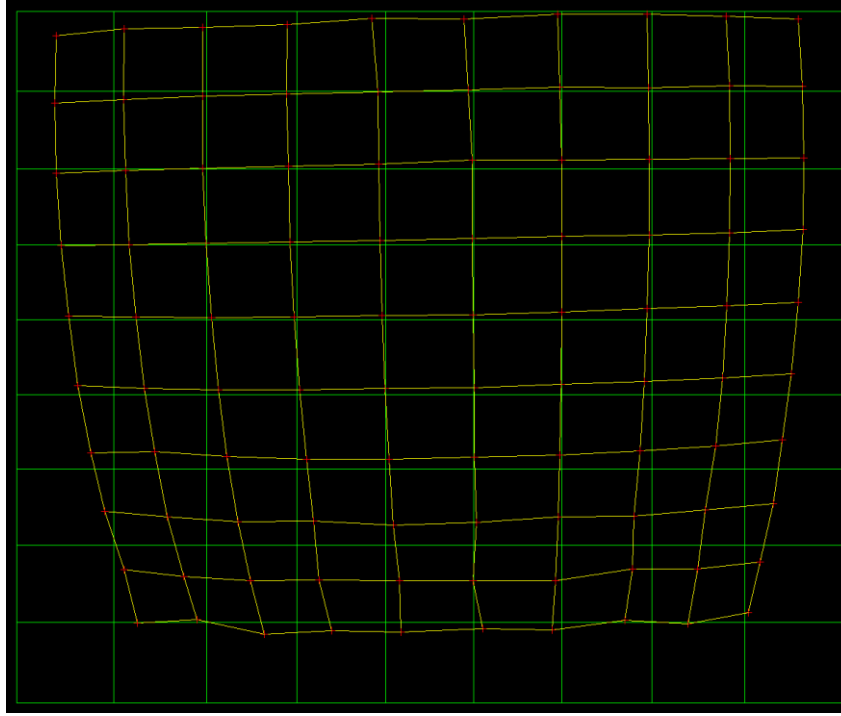
Şekil 7.3 — Noktaların beklenen yerlerden koparak düzensiz biçimde toplandığı, yansıma etkisinin baskın olduğu ölçüm.

fx	373.709	fy	-71.644
cx	883.728	cy	814.710
k1	1.8257	k2	-0.7065
p1	0.4756	p2	-0.0887
RMS (px)	426.9604		

Eksen ölçeği düzeltildikten sonra bile bazı noktaların gelmesi gereken konumdan tamamen kopup alakasız bölgelerde kümелendiği görülmüştür. Bunun nedeni, lazer ışığının çevredeki metal yüzeylerden veya duvardan yansarak kameraya ikinci (istenmeyen) bir yol üzerinden ulaşmasıdır; centroid algoritması bu durumda gerçek lazer noktasını değil, o an daha parlak görünen yansıma bileşenini takip etmeye başlamıştır. RMS =

426.96 px ile bu, veri kümesindeki en yüksek hatadır ve bu ölçüm, Bölüm 4.4'te açıklanan optik yalıtım önlemlerinin (siyah bant, siyah kumaş) neden zorunlu hale geldiğini doğrudan göstermektedir.

9.4 Ölçüm 4 — Mekanik Sabitleme Eksikliği ve Perspektif Etkisi

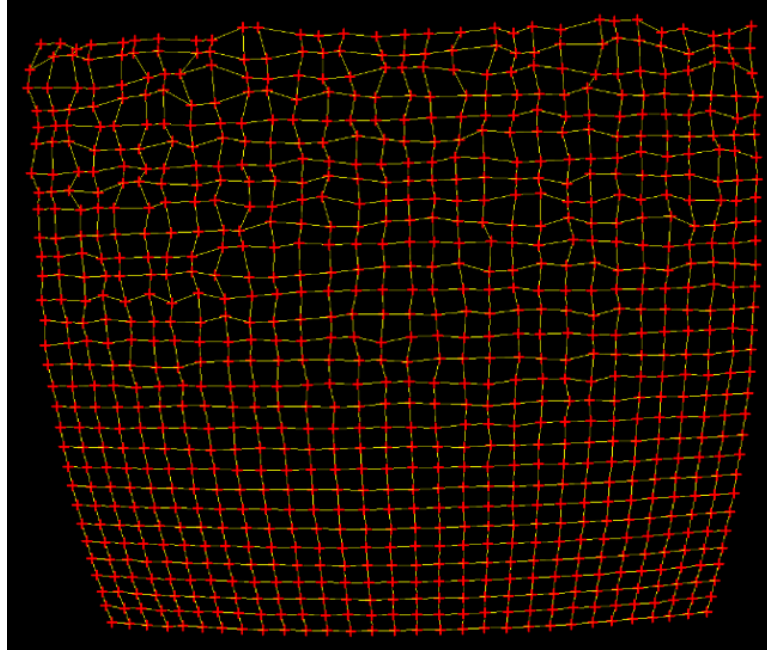


Şekil 7.4 — Elevasyon ekseninin yeterince sabitlenememesi nedeniyle perspektif altında eğilmiş görünen ölçüm ızgarası.

fx	1635.190	fy	-1640.460
cx	912.095	cy	1141.898
k1	-1.1124	k2	1.7044
p1	0.1007	p2	0.0113
RMS (px)	11.2885		

Yansıma bastırıldıktan sonra RMS ~38 kat iyileşerek 11.29 px'e inmiştir. Ancak gimballı ve özellikle kamerayı taşıyan elevasyon ekseninin yeterince sabitlenememesi, ilk ve son satırların aynı düzlemde kalmamasına, yani ızgaranın perspektif altında eğilmiş gibi görünmesine yol açmıştır. Bu gözlem, görüntüdeki bozulmanın bir kısmının lense değil mekaniğe ait olabileceği sorusunu gündeme getirmiş ve sabitleme iyileştirmelerinin yalnızca konfor değil, optik yorumun geçerliliği için zorunlu olduğunu ortaya koymuştur.

9.5 Ölçüm 5 — Kamera Açısından Kaynaklanan Trapezleşme

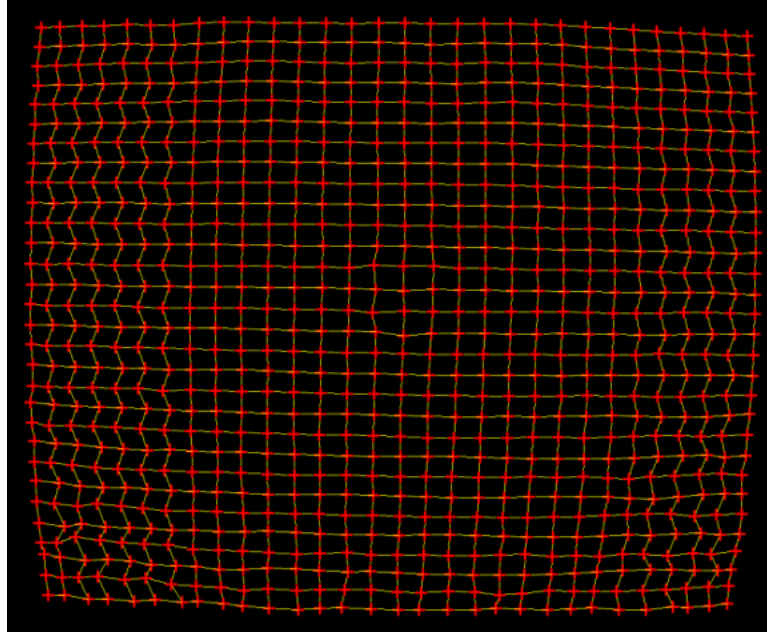


Şekil 7.5 — Kameranin pinhole eksenine tam dik değil, hafifçe yukarıdan bakması nedeniyle aşağı doğru daralan yamuk (trapez) biçimli ölçüm.

fx	1573.936	fy	-1593.595
cx	926.303	cy	1034.351
k1	-1.0235	k2	1.4780
p1	0.0859	p2	0.0076
RMS (px)	10.6413		

Bu ölçümde ızgaranın üstten geniş, alta doğru daralan bir yamuk (trapez) biçimi alması, kameranin pinhole/lazer eksenine tam dik konumlanmadığını, hafifçe yukarıdan baktığını göstermiştir. Yani gözlenen bozulmanın bir kısmı optik distorsiyondan değil, saf kamera perspektifinden (bakış açısından) kaynaklanmaktadır. Alt bölgelerin daha düzenli görünmesi, o bölgeye ulaşan ışınların açısal aralığının daha dar (kameraya göre daha merkezi) olmasıyla açıklanabilir; üst bölgede daha geniş açı bileşenleri devreye girdiğinden saçılma da artmıştır. Bu gözlem, yalnızca lazer-pinhole ekseninin değil, kameranin bakış açısının da düzenekte doğrudan kontrol edilmesi gerektiğini göstermiştir.

9.6 Ölçüm 6 — Kenar Bölgelerde Zigzag Deseni

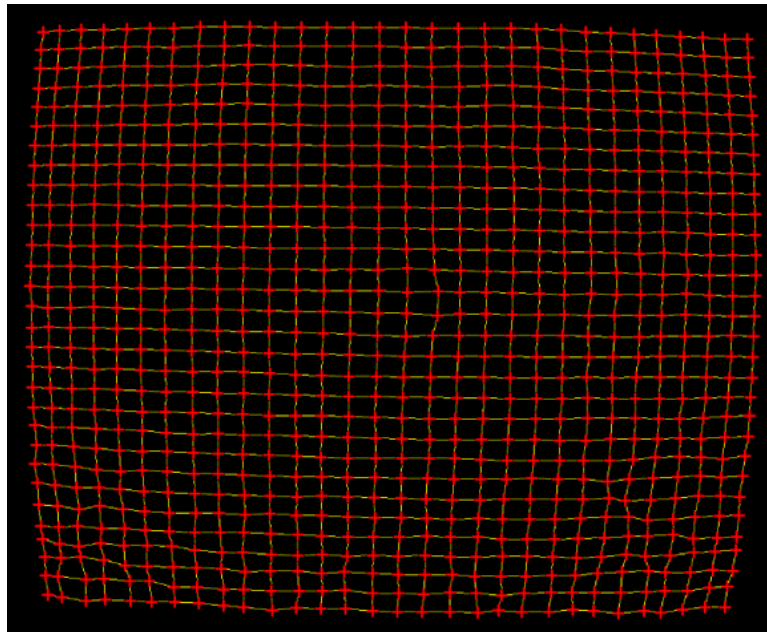


Şekil 7.6 — Orta bölgede düzenli, sağ ve sol kenarlarda ise belirgin zigzag deseni gösteren ölçüm (bkz. Bölüm 8.2).

fx	1799.449	fy	-1822.736
cx	1027.483	cy	1036.063
k1	-1.2485	k2	2.6977
p1	0.0252	p2	0.0068
RMS (px)	4.8928		

Kamera açısı düzeltildikten sonra orta bölgedeki nokta örgüsü büyük ölçüde düzenli hâle gelmiş, ancak sağ ve sol kenarlarda gözle görülür bir zigzag deseni ortaya çıkmıştır. Bölüm 8.2'de ayrıntılı biçimde açıklandığı gibi, bu gözlem boustrophedon tarama deseninin terk edilip her satırın soldan sağa aynı yönde taranmasına geçilmesine yol açmıştır. RMS bu ölçümde zaten 4.89 px'e inmiş olsa da, kenar bölgelerdeki zigzag deseni, verideki hâlâ giderilmemiş bir sistematik hatanın izini taşımaktadır.

9.7 Ölçüm 7 — Kararlı Çözüm Aşaması I

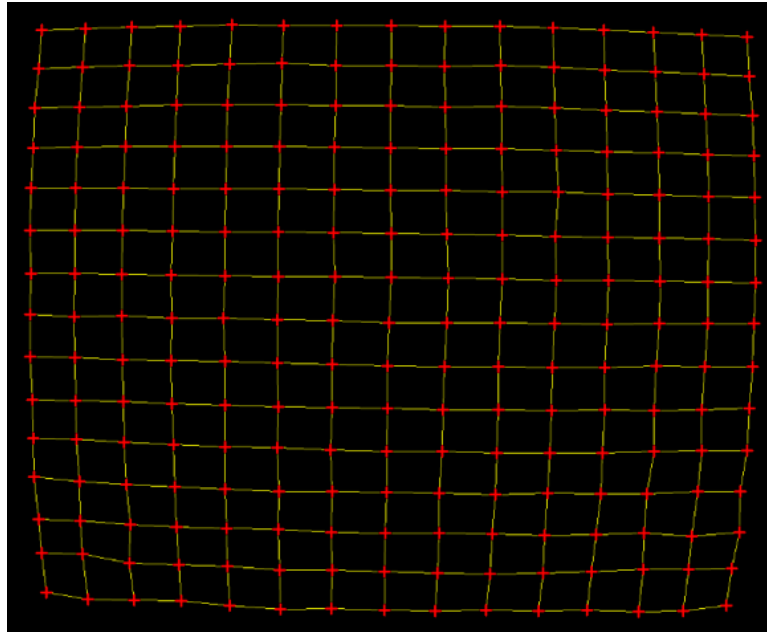


Şekil 7.7 — Tek yönlü tarama ve mekanik iyileştirmeler sonrası elde edilen ilk kararlı ölçüm.

fx	1800.531	fy	-1820.662
cx	1035.105	cy	1037.396
k1	-1.2003	k2	2.2481
p1	0.0254	p2	0.0012
RMS (px)	4.8456		

Tek yönlü tarama deseni ve mekanik sabitleme iyileştirmeleri uygulandıktan sonra elde edilen bu ölçüm, sistemin kararlı çalışma bandına yaklaştığı ilk açık örneklerden biridir. Işgара formu hem merkezde hem de çevresel bölgelerde tutarlı görünmekte; özellikle tanjansiyel distorsiyon bileşenlerini temsil eden p1, p2 değerlerinin önceki ölçümlere kıyasla küçük ve istikrarlı kalması, sabitleme ve hizalama iyileştirmelerinin sayısal çözüme doğrudan olumlu yansıdığını göstermektedir.

9.8 Ölçüm 8 — Kararlı Çözüm Aşaması II (Nihai Referans Ölçüm)



Şekil 7.8 — Çalışmanın en olgun ölçümü; örgü formu ve katsayılar dar bir bantta tekrarlanmıştır.

fx	1799.752	fy	-1822.958
cx	1022.851	cy	1026.270
k1	-1.2067	k2	2.5310
p1	0.0239	p2	0.0078
RMS (px)	4.9372		

Bu son ölçüm, Ölçüm 7 ile neredeyse aynı parametre bandında ($fx \approx 1800$, $fy \approx -1822$, $k1 \approx -1.20$) tekrarlanabilir bir sonuç vermiştir; bu tekrarlanabilirlik, sistemin artık deneme-yanılma aşamasından çıkıp ölçümsel olarak yorumlanabilir bir olgunluk seviyesine ulaştığının en güçlü kanıtıdır. k1'in iki ölçümde de tutarlı biçimde negatif kalması, lens davranışının baskın olarak barrel distortion karakteri taşıdığını doğrulamaktadır; k2'nin pozitif kalması ise daha yüksek dereceli radyal bileşenin de modele küçük ama tutarlı bir katkı verdiğini göstermektedir. Bu iki ölçüm (7.7 ve 7.8), çalışmanın nihai referans sonuçları olarak değerlendirilmelidir.



Şekil 7.9 — Pinhole bölgesinin yakın plan görünümü: hasarlı kenardan sızan ışığı önlemek için uygulanan siyah bant müdahalesi görülmektedir. Bu tür optik yalıtım adımları, Ölçüm 6→7→8 arasındaki iyileşmenin fiziksel kaynaklarından biridir.

10. Tartışma

Sekiz ölçümün karşılaştırmalı incelenmesi, hem sistemin optik karakteri hem de deneysel yöntemin olgunlaşma süreci hakkında önemli bulgular sunmaktadır. Tüm ölçüm çıktıları birlikte değerlendirildiğinde, sistemde gözlenen bozulmaların tek bir nedene indirgenemeyeceği açıkça görülmektedir: erken ölçümlerdeki yelpaze biçimli veya aşırı uzamış örgüler esas olarak eksenler arası açısal eşitsizlik ve yansıma kaynaklıyken, orta aşamadaki yamuklaşma ve satır kaymaları perspektif ve mekanik sabitleme eksikleriyle, geç dönemdeki kenar zigzakları ise tarama stratejisiyle ilişkilendirilmiştir.

10.1 RMS İyileşme Eğilimi

RMS değerleri kronolojik sırayla 245.68 → 27.28 → 426.96 → 11.29 → 10.64 → 4.89 → 4.85 → 4.94 px olarak seyretmiştir. Üçüncü ölçümdeki ani sıçrama (27.28 → 426.96), bu ölçüme özgü, ayrı bir yansıma probleminin devreye girmesinden kaynaklanmaktadır (bkz. Bölüm 9.3) ve genel eğilimi bozan bir istisnadır. Bu istisna dışında bakıldığında, her düzeltme adımı (eksen ölçeği ayırımı, optik yalıtım, mekanik sabitleme, kamera açısı düzeltilmesi, tek yönlü tarama) RMS'te kayda değer bir iyileşmeye karşılık gelmiştir. Son üç ölçümün (9.6–9.8) RMS'inin 4.85–4.94 px dar bandında tekrarlanabilir biçimde kalması, sistematik hataların büyük bölümünün artık kontrol altına alındığını göstermektedir.

10.2 k1 İşareti ve Barrel Distorsiyon Yorumu

Sistem henüz kararsızken (Ölçüm 1, 2) elde edilen k1 değerleri (-26.73, +2.30) fiziksel olarak anlamsız büyüklüktedir ve geometrik olarak tutarsız veriye zorla uydurulmuş bir optimizasyonun işaretidir. Ölçüm 3'ten itibaren k1 tutarlı biçimde negatif bir bantta (-1.02 ile -1.25 arası) yerleşmiş ve son iki ölçümde (-1.2003, -1.2067) neredeyse aynı değere yakınsamıştır. Bu tutarlı negatif işaret, barrel (fıçı tipi) distorsiyona işaret etmektedir — yani görüntünün kenarlarındaki noktalar ideal konumlarına göre merkeze doğru çekilmektedir. Bu, kısa odaklı ($f = 12$ mm) lenslerde sıkça karşılaşılan, beklenen bir davranıştır.

10.3 Dikkat Edilmesi Gereken Bir Gözlem: f_y 'nin İşareti

Sekiz ölçümün tamamında f_y değeri sistematik olarak negatif çıkmıştır, buna karşılık f_x pozitifdir. Fiziksel olarak f_x ve f_y , aynı fiziksel odak uzaklığının (f) iki eksendeki piksel-ölçekli karşılıklarıdır ($f_x = f/p_x$, $f_y = f/p_y$, Bölüm 6.1) ve piksel boyutu fiziksel olarak pozitif bir büyüklük olduğundan, f_x ve f_y 'nin aynı işarete (ikisi de pozitif) sahip olması beklenir. Sistematik olarak negatif çıkan f_y , muhtemelen model kurulumunda piksel v-ekseninin (görüntüde aşağı yön) ile gimbal elevasyon ekseninin yön kuralının (sign convention) birbiriyle ters tanımlanmış olmasından kaynaklanmaktadır — yani optimizasyon, gerçek fiziksel f_y yerine matematiksel olarak eşdeğer ama işareti ters bir çözüme yakınsamış olabilir. Bu, kalibrasyon sonucunun genel geçerliliğini bozmaz (çünkü ileri model hâlâ ölçülen verilere düşük RMS ile uyar) ancak f_y 'nin ham hâliyle "fiziksel odak uzaklığı" olarak yorumlanmaması, bunun yerine $|f_y|$ değerinin kullanılması ve ilerideki çalışmada y-ekseni yön kuralının gözden geçirilmesi önerilir. Bu, raporun bilinen bir sınırlaması olarak açıkça belirtilmektedir.

10.4 Mekanik ve Optik Hata Kaynaklarının Ayrıştırılması

Bu çalışmanın en önemli metodolojik kazanımlarından biri, gözlemlenen sapmaların hangi kısmının optik (lens distorsiyonu) ve hangi kısmının mekanik/geometrik (eksen ölçeği, yansıma, sabitleme, kamera açısı, backlash) kaynaklı olduğunun adım adım ayrıştırılabildiği olmasıdır. Ölçüm 2→3→4→5→6 arasındaki her geçiş, tek bir spesifik mekanik/geometrik sorunun giderilmesiyle açıklanabilirken, tüm ölçümlerde (kararsız olanlar hariç) tutarlı biçimde görülen barrel eğilimli k_1 değerleri gerçek, tekrarlanabilir bir optik özelliktir. Bu ayrıştırma, distortion ölçümünün yalnızca bir görüntü işleme/optimizasyon problemi değil; optik yolun temizliği, eksenler arası kinematik ilişki, taşıyıcı rijitliği ve tarama stratejisinin birlikte ele alınması gereken çok katmanlı bir sistem problemi olduğunu göstermektedir.

10.5 Teorik ve Deneysel f_x Karşılaştırması, Çalışma Mesafesi Üzerine Bir Not

Basler C11-1220-12M lensinin nominal odak uzaklığı $f = 12$ mm ve Prosilica GT2050'nin piksel boyutu 5.50 μm 'dir (Bölüm 4.1). Bu durumda, sonsuz uzaklıktaki bir kaynak için beklenen teorik f_x değeri:

$$f_x = \frac{f}{p_x}, \quad f_y = \frac{f}{p_y}$$

ile $f_x = 12000 \mu\text{m} / 5.5 \mu\text{m} \approx 2182$ piksel olarak hesaplanır. Buna karşılık, çalışmanın en olgun ölçümlerinde (Ölçüm 7 ve 8) deneysel olarak bulunan f_x değerleri sırasıyla 1800.53 ve 1799.75 pikseldir — teorik değer yaklaşık %82'si, yani %18 civarında daha düşüktür. Bu fark birkaç olası nedenden kaynaklanabilir: (i) lazer-kamera mesafesinin (Z) sonsuz kabul edilemeyecek kadar küçük olması ve ince mercek denkleminde (Bölüm 5.3) görüntü mesafesinin $v = f$ 'den farklılaşması, (ii) lensin gerçek odak uzaklığının üretici toleransları dahilinde nominal değerden sapması, (iii) doğrusal olmayan optimizasyonda f_x ile k_1 , k_2 arasındaki parametre korelasyonu (bir miktarı diğerinin telafi etmesi) nedeniyle f_x 'in saf geometrik anlamından bir miktar uzaklaşmış olması.

Bu noktada önemli bir uyarı gerekir: Basler C11-1220-12M lensinin üretici veri sayfasında belirtilen minimum çalışma mesafesi (min. working distance) 100 mm'dir (Bölüm 4.1). Deney sırasında lazer-kamera mesafesi kesin olarak ölçülmemiş olup yaklaşık birkaç santimetre (~ 6 cm) olarak tahmin edilmektedir; bu tahmin doğruysa, sistem lensin üretici tarafından garanti edilen odaklama aralığının dışında çalışıyor olabilir. Bu durum hem PSF genişlemesine (Bölüm 5.4) hem de yukarıdaki f_x sapmasına katkı sağlayabilecek önemli bir potansiyel hata kaynağıdır. Kesin sonuca varmadan önce Z mesafesinin bir cetvel/kumpasla doğrudan ölçülmesi ve bu ölçümün

rapora eklenmesi önerilir; bu doğrulama yapılmadan f_x 'teki %18'lik sapmanın ne kadarının bu etkiye, ne kadarının parametre korelasyonuna ait olduğu kesin olarak ayrıştırılamaz.

10.6 Chessboard ve Lazer Yöntemlerinin Çapraz Karşılaştırması

Raporun iki bağımsız kalibrasyon yöntemi de, birbirinden habersiz biçimde, kamera içsel parametrelerinde sistematik bir anomali işaret etmiştir: chessboard tabanlı 11 kalibrasyonun tamamı optik merkezin dikey bileşeninde (cy) görüntü merkezine göre tutarlı ve tek yönlü bir kayma (Δy ortalama ≈ -72 px, bkz. Bölüm 2.4) bulmuş; gimbal-lazer grid taramasının sekiz ölçümü ise fy parametresinin sistematik olarak negatif işaretli çıktığını (bkz. Bölüm 10.3) göstermiştir. Bu iki bulgu doğrudan aynı sayısal büyüklüğü ölçmese de — biri bir ofset (cy), diğeri bir işaret tutarsızlığı (fy) — ikisi de aynı temaya işaret etmektedir: kameranin düşey eksenindeki (v-ekseni) davranışı, yatay eksene (u-ekseni/cx/fx) kıyasla modelde daha kırılkan ve yön kuralına (sign convention) daha duyarlıdır. Bu örtüşme, raporun 7. Bölüm'de öngörülen "chessboard ve lazer-grid sonuçlarının aynı fx, fy, cx, cy uzayında karşılaştırılması" önerisinin ilk, niteliksel adımını oluşturmaktadır; sayısal/istatistiksel karşılaştırma ise Bölüm 11'deki gelecek çalışma önerileri arasında yer almaya devam etmektedir. Chessboard yönteminin cx için bulduğu değer (≈ 1031 px) ile lazer yönteminin son iki ölçümde bulduğu cx değerleri (1035.1 ve 1022.9 px, bkz. Bölüm 9.7–9.8) ise dikkat çekici derecede yakındır; bu yakınlık, en azından yatay eksende iki yöntemin birbirini bağımsız biçimde doğruladığının bir işareti olarak okunabilir.

11. Sonuç ve Öneriler

Bu çalışmada, star tracker optik sisteminin kamera-lens kombinasyonu, biri klasik (chessboard tabanlı) diğeri özgün (gimbal + lazer tabanlı grid tarama) olmak üzere iki bağımsız yöntemle karakterize edilmiştir. Lazer-pinhole-gimbal düzeneği, sekiz ölçümlük bir gelişim sürecinin sonunda, artık tekrarlanabilir ve yorumlanabilir distortion verisi üretebilen işlevsel bir prototip seviyesine ulaşmıştır. Elde edilen bulgular özetle:

- Sistemde tutarlı biçimde barrel tipi ($k_1 < 0$, nihai ölçümlerde $k_1 \approx -1.20$) radyal distorsiyon gözlemlenmiştir.
- Eksenler arası açısal ölçek farkının ($DEG_PER_MM_X \neq DEG_PER_MM_Y$) modele dahil edilmesi, erken dönemdeki dikdörtgen/yelpaze biçimli ölçüm hatalarını gidermiştir.
- Optik yol yalıtımı (pinhole çevresindeki ışık sızıntısının siyah bant/kumaşla kapatılması) yansıma kaynaklı en yüksek hatalı ölçümü ($RMS = 426.96$ px) ortadan kaldırmıştır.
- Kamera açısının pinhole eksenine dik hale getirilmesi, perspektif kaynaklı trapezleşmeyi gidermiştir.
- Mekanik backlash kaynaklı zigzag artefaktı, tek yönlü tarama deseniyle başarıyla giderilmiştir.
- Son iki ölçüm (Ölçüm 7, 8), $fx \approx 1800$, $k_1 \approx -1.20$ civarında birbirine çok yakın parametrelerle yakınsamış ve $RMS \approx 4.85\text{--}4.94$ px seviyesinde kararlı hâle gelmiştir; bu ölçümler nihai referans sonuçları olarak değerlendirilmelidir.
- fy parametresinin işaretiyle ilgili bir yön-kuralı (sign convention) tutarsızlığı tespit edilmiş ve ilerideki çalışma için not edilmiştir.
- Deneysel fx (≈ 1800 px), lens/sensör veri sayfalarından hesaplanan teorik değerden (≈ 2182 px) yaklaşık %18 düşük çıkmıştır; bu farkın lazer-kamera mesafesinin lensin 100 mm'lik minimum çalışma mesafesinin altında kalmış olabileceği ihtimaliyle ilişkili olabileceği belirlenmiş, kesin doğrulama için mesafenin ölçülmesi önerilmiştir (Bölüm 10.5).
- Bağımsız bir hassasiyet karakterizasyonu (otokollimatör ile sekiz ölçüm kampanyası), laboratuvardaki iki nominal gimbal örneğinden "Kamera Gimbalı" olarak adlandırılanın azimut ekseninde teorik değere daha yakın ve daha hızlı kararlı rejime geçen bir davranış gösterdiğini ortaya koymuş ve bu birimin günlük kullanım için seçilmesine yol açmıştır (Bölüm 3.2).

- Chessboard tabanlı 11 bağımsız kalibrasyonun (3 farklı ortam) ortak sonucu olarak, görüntünün geometrik merkezi (1024, 1024) yerine kod içinde kullanılmak üzere $(cx, cy) \approx (1031, 952) \pm (13, 21)$ px optik merkez değeri önerilmiştir; bu değer bağımsız bir lazer/otokollimasyon ölçümüyle doğrulanması ayrıca önerilmektedir (Bölüm 2.4).

Bununla birlikte raporun vurgulanması gereken bir sonucu, bu katsayıların güvenilir biçimde yorumlanabilmesinin yalnızca iyi bir çözüm algoritmasına bağlı olmadığıdır: lazer ışınının pinhole'dan temiz geçmesi, optik eksenin korunması, kamera perspektifinin denetlenmesi ve gimbal hareketinin mm-açı ilişkisiyle birlikte doğru modellenmesi aynı ölçüde belirleyici olmuştur. Distortion ölçümü, yalnızca bir görüntü işleme kalibrasyon problemi değil; optik yolun temizliği, eksenler arası kinematik ilişki, taşıyıcı rijitliği ve tarama stratejisinin birlikte değerlendirilmesi gereken çok katmanlı bir sistem problemidir.

Gelecek çalışmalar için önerilenler:

- Gimbal elevasyon ekseninin mekanik sabitlenmesinin kalıcı olarak iyileştirilmesi ve mümkünse manyetik cetvel/encoder tabanlı kapalı çevrim kontrolün tüm eksenlerde etkinleştirilmesi.
- y-ekseni yön kuralının (piksel v yönü ile elevasyon açısı yönü arasındaki ilişki) modelde açıkça tanımlanarak fy işaretinin fiziksel olarak tutarlı hâle getirilmesi.
- Lazer-kamera mesafesinin (Z) doğrudan ölçülüp lensin minimum çalışma mesafesiyle (100 mm) karşılaştırılması; gerekirse ışık yolunu kolimatörlü bir yapıya dönüştürecek optik eklemelerin değerlendirilmesi (defter notlarında da bu ihtiyaç ayrıca belirtilmiştir).
- Kameranin pinhole eksenine tam dik bağlanamaması durumunda, çözüme dönme ve öteleme (R, T) bileşenlerinin de eklenmesini gerektirecek daha kapsamlı bir dış (extrinsic) kalibrasyonun değerlendirilmesi.
- 30×30 veya daha yoğun bir grid ile, merkez-hizalı ve tek yönlü tarama koşulları birlikte uygulanarak nihai, yayın-kalitesinde bir kalibrasyon setinin üretilmesi.
- Chessboard (Aşama 1) ve lazer-grid (Aşama 2) sonuçlarının aynı fx, fy, cx, cy uzayında sayısal olarak karşılaştırılması ve iki yöntemin ne ölçüde örtüştüğünün istatistiksel olarak raporlanması.
- Elde edilen nihai K matrisi ve distorsiyon katsayılarının, gerçek yıldız görüntüleriyle uçtan uca (piksel → açısız yön vektörü) doğrulanması.

Ek: Bilinen Donanım Kısıtları ve Yapılacaklar Listesi

Çalışma süresince tespit edilen ve henüz kalıcı biçimde çözülmemiş donanımsal ihtiyaçlar, şeffaflık amacıyla burada listelenmiştir:

- Kamera altı için özel tasarlanmış, 3D yazıcıdan üretilecek bir sabitleme düzeneği (mevcut geçici Lab Jack + çubuk montajının yerini alacak).
- Yeterli sayıda ve çeşitli metrik/dış yapısında çivi/vida stoğu (Lab Jack ve kamera düzeneği arasındaki dış uyumsuzluğunu gidermek için).
- Lazeri kalıcı olarak tutacak, köpük dolgu gerektirmeyecek, ölçülü bir sabitleme aparatı.
- Kabloları düzenli tutacak kablo yönetim aparatları.
- Pinhole'u kalıcı ve titremeden tutacak, hizalaması korunacak bir aparat.
- Lazer-kamera mesafesini sonsuza yakın bir kaynağa dönüştürecek kolimatörlü bir optik yapı (defter notlarında ayrıca belirtilmiştir, bkz. Bölüm 10.5).

Bu listedeki eksikliklerin giderilmesi, özellikle Bölüm 10.4'te tartışılan mekanik kaynaklı hata bileşenlerinin daha da azaltılması ve gelecekteki ölçümlerin tekrarlanabilirliğinin artırılması açısından öncelikli görülmektedir.

Kaynakça / Referanslar

Bu raporda kullanılan kamera modeli ve distorsiyon formülasyonu, bilgisayarla görüde (computer vision) yaygın kabul gören standart pinhole kamera modeli ve Brown–Conrady distorsiyon modeline dayanmaktadır (OpenCV kütüphanesinin camera calibration modülünde kullanılan formülasyonla uyumludur). Optimizasyon, SciPy `scipy.optimize.least_squares` (Levenberg–Marquardt / Trust Region Reflective) çözücüsü ile gerçekleştirilmiştir. Donanım tarafında hareket kontrolü Marlin firmware tabanlı bir denetleyici kart üzerinden standart G-code komut seti ile sağlanmıştır.

14.08.2026

Soru:

Haftalardır üzerinde çalışıyoruz ve hep barrel alıyoruz ama sonra farkettiler ki bu aslında beklenen bir durum hele de lensimizden çıkan ışın sonlu bir ışın, kolimatör lensi yok. biz sonra bunu değiştirdik şu an kolimatör lensi olan eke koyduğum HeNe lazer aygıtını kullanıyoruz ucuna da kolimatör lens taktık. şimdi ışınlar geliyor ve gimbal dairesel oynasa bile istediğimizi şekilde çıktıda kare alıyoruz ama biz doğrusal hareket eden bir sisteme geçmedik hala yaysal hareket ediyoruz bu kolimatör lens bu kadar değişiklik yapabilir mi?

Claude cevap:

Evet, raporu ve kodu okudum — resim net: kamera + pinhole + gimbal + lazer düzeneğiyle, kullandığınız lensin gerçek distorsiyon katsayılarını (k_1 , k_2 , p_1 , p_2) ölçmeye çalışıyorsunuz. Gimbal lazeri açısal olarak (azimut/elevasyon) tarıyor, kod da her açı için $x_{\text{prime}} = \tan(\theta_x)$, $y_{\text{prime}} = \tan(\theta_y)$ varsayıp bunu piksel konumuna $f \cdot x' + c_x$ şeklinde eşliyor — yani standart pinhole modeli, açıya göre kuruluyor. Sorunuzu da doğru anladıysam: "gimbal zaten dairesel/açısal hareket ediyor, doğrusal harekete geçmedik, öyleyse sırf kolimatör lens eklemek nasıl bu kadar köklü bir fark yaratabilir?"

Burada olan şey aslında gimbalin açısal mı doğrusal mı hareket ettiğiyle ilgili değil — kodunuzdaki model zaten baştan beri açısal hareketi ($\tan \theta$) varsayıyor, o yüzden "yaysal hareketten kurtulmadık" demeniz aslında bir sorun değil, olması gereken zaten bu. Asıl kırılma noktası şu: kameranın gördüğü şey lazerin **yönü** mü, yoksa lazer kaynağının **fiziksel konumu** mu?

Kolimasyonsuz (veya zayıf kolimasyonlu) bir kaynaktan gimbal döndükçe, lazerin çıkış noktası pivot eksenine tam oturmuyorsa (ki fiziksel olarak neredeyse hiçbir zaman tam oturmaz — tüp/kafa pivot merkezinden birkaç cm ileride/yanda montajlıdır), bu çıkış noktası dönüş sırasında pivot etrafında küçük bir yay üzerinde de kayar. Eğer kamera bu ışını "nereden geliyor" diye triangüle ediyorsa (yani ışın huzmesi yeterince geniş/iraksak olduğu için kamera aslında kaynağın konumunu görüntülüyorsa), o zaman ölçülen piksel konumu $\tan(\theta)$ değil, kabaca $d \cdot \sin(\theta)/L$ gibi bir terime de bağlı hale gelir (d = pivot–lazer çıkışı arasındaki kol uzunluğu, L = çalışma mesafesi). Modeliniz bunu hesaba katmadığı için, gerçek lens tertemiz olsa bile fit bu farkı "radyal distorsiyon" gibi yorumlar ve elinize sistematik bir barrel çıkar. Raporunuzdaki minimum çalışma mesafesinin 100 mm gibi

oldukça kısa olması da bunu şiddetlendiren bir etken — d/L oranı küçük bir kol uzunluğunda bile hiç de ihmal edilemeyecek kadar büyür.

Kolimatör lens eklediğinizde ise ışın artık gerçek anlamda paralel bir demet (düzlem dalgaya yakın) haline geliyor. Bu durumda kamera artık bir "konum ölçer" gibi değil, bir **otokollimatör** gibi davranır: sonsuza odaklı bir optik sistemde, gelen kolime bir ışının odak düzlemindeki noktası SADECE geliş açısına bağlıdır, ışının hangi lateral noktadan başladığına değil (ışın küçük bir mesafe yana kaysa bile aynı açıyla geliyorsa aynı piksele düşer). Yani gimbal döndükçe lazer çıkışı hâlâ o küçük yayı çiziyor olsa da, artık bunun görüntüye hiçbir katkısı yok — kamera sadece komut verdiğiniz açıyı görüyor, tam da kodunuzun varsaydığı $\tan(\theta)$ modelini birebir sağlıyor. Sonuç: kare çıktı.

Yani evet, kolimatör lens gerçekten bu kadar köklü bir fark yaratabilir, hatta yaratması beklenen bir şey — çünkü değişen "açısal mı doğrusal mı hareket" değil, "kamera açısı mı ölçüyor, konum mu ölçüyor" sorusunun cevabı. Haftalarca ısrarla aynı barrel'i almanızın nedeni de muhtemelen sabit bir kol uzunluğu/mesafe kombinasyonuydu — sistematik olması (rastgele gürültü değil) bu yorumla tam örtüşüyor.

Bunu deneysel olarak doğrulamak isterseniz iki basit test var: (1) eski (kolimatörsüz) kaynakla çalışma mesafesini önemli ölçüde artırıp barrel'in küçülüp küçülmediğine bakmak (d/L küçülünce etki azalmalı), ya da (2) şu anki kolime kaynakla mesafeyi kasıtlı kısaltıp hafif bir barrel'in geri gelip gelmediğine bakmak. İkisi de olursa hipotez fazladan doğrulanmış olur.

tamam önceki konuşmalarımızdan şunu da biliyorsun ki bizim gimbalimiz çembersel döndüğü için aldığımız görüntüde çıkan şekil de üst kısımları daha düzlemsel yandan ise şişen bir barrel görünümü yaratıyor. çünkü sonuçta yaysal hareket eden bir cihazın linear davranmasını bekleyemeyiz. x eksenini üzerinde elbette nonlinearities oluyor, her 2 nokta arası aynı genişlikte değil çünkü burda arcsec işin içine giriyor. bunu kabullendiğimizi ve sistemimizi x-y açısından linear hareket eden bir sisteme geçiremeyeceğimizi biliyoruz. gimbal ile hareket etmek zorundayız. şimdi biz kodumuza R ve T matrislerini eklemeyi düşündük ve lazerimizi normal bir lazerden HeNe lazere geçirdik. bu lazer fiberoptik kablolarla ilerleyip en sonunda kolime lensten geçerek ışık çıkartıyor. yani artık değişen şey şu ki, lazerimiz kolime bir ışık kaynağı yani sonsuzdan geliyor gibi düşünebiliriz. önünde ise pinhole var. pinholeun ışığı yine bir miktar kırdığını biliyoruz belki paralelliğe zarar veriyordur ama o da değiştirebileceğimiz bir şey değil. şimdi lazeri açıp kameranın tam ortasına gelmesini sağlayacak bir kod yaptık yani burda gimbal değil de kamerayı fiziksel olarak ileri geri oynatıyoruz ve 1024x1024 noktasına lazeri getiriyoruz. burdan atış yapan R ve T matrislerini içeren kodu başlatacağız. şimdi burda sorun gimbalin yaysal taraması, bu konuda başka ne yapabiliriz yani olay şu, benim lensimin distortion sayısı -3.1% olarak sitesinde verilmiş, yani barrel distortion yapan bir lens. biz de barrel görüyoruz ama gördüğümüz barrel hem gimbalin yaysal hareketinden gelen hem de distortiondan gelenin birleşimi olduğu için doğru bir ölçüm yapamıyoruz. bu durumda R ve T yi eklessek kodumuza? ve şunu biliyoruz ki $\text{distortion} = (\text{gerçek görüntü yüksekliği} - \text{ideal görüntü yüksekliği}) * 100 / \text{ideal görüntü yüksekliği}$. bu durumda ideal görüntü yüksekliğini bulmak için de kameranın içsel parametreleri olan f_x, f_y ve c_x, c_y yi bulmamız gerekir. bu durumda da kafam çok karışıyor çünkü f_x ve f_y siteden dökümanlarına baktığımda f_x/f_y den $f_y = f_x = 2251.37$ çıkıyor yani biz ölçümlerimizdeki sonuçlarımızı hep bu değerle ykaslayarak kötü yad a iyi sonuç diyoruz. aynı zamanda 2048lik bi kameranın $c_x = c_y = 1024$ olmasını bekleriz, gelen c_x ve c_y için de kötü/iyi diye burda karaar veriyoruz. ama şimdi şu kafamı karıştırıyor biz bunları yazılıma bulduruyoruz ve yazılım farklı buluyor ama biz acaba koda f_x, f_y, c_x, c_y değerlerini girsek ve burdan ideal görüntü yüksekliği her nokta için değişen ama sabit çarpanlara sahip bir fonksiyon olsa şöyle $= x' * 2252.37 + 1025$? mesela her nokta için bu bulunsa ve sonra da piksel açısından hani hangi pikselde olduğu-düşmesi gereken yapıyı disortron bulunsa? burada kafam çok karışıyor çünkü f_x, f_y, c_x, c_y yi bulurken yaotığı hatalar da distortion'a girer mi çünkü biz başından beri least square ile f_x, f_y, c_x, c_y arıyoruz.. eski makaleleri okuyarak acaba zhang vs tarzı insanlar da bu şekilde mi yapmış yoksa kameranın intrinsiclerini koda girmişler mi? bütün paragraftaki kafa karışıklıklarına yardımcı olur msuun

Kafa karışıklığın aslında birbirine bağlı üç ayrı soruya dağılmış durumda — gimbalin yaysal hareketi, R/T eklemek, ve $f_x/f_y/c_x/c_y$ 'yi sabitleyip sabitlememek. Bunları tek tek ayırayım çünkü hepsi aynı köke bağlanıyor.

Önce gimbal meselesi: tek eksenle yaysal (açısal) hareket etmesi zaten bir sorun değil, kodun zaten beklediği şey bu — $x' = \tan(\theta x)$ dediğinizde, θx 'in kendisi zaten açısal bir büyüklük, dolayısıyla iki nokta arası eşit piksel aralığı beklememeniz doğru ve modelin öngördüğü de tam olarak bu. Asıl incelenmesi gereken yer, iki eksenin AYNI ANDA döndüğü durum. İki eksenli bir gimbalde (biri diğerinin üstüne oturan, seri bağlı iki dönme mekanizması), azimut ve elevasyon birbirinden bağımsız iki $\tan(\theta)$ üretmez — hangi eksen "dışta" (referans çerçeveyi taşıyan) hangisi "içte" (diğerinin üzerinde dönen) olduğuna bağlı olarak aralarında bir kuplaj terimi çıkar. Örneğin elevasyon içte, azimut dışta ise (yani önce elevasyon uygulanıp sonra o eğik gövde azimutta döndürülüyorsa), üç boyutlu ışın yönünü $v_0 = (0,0,1)$ 'den başlayıp önce elevasyon sonra azimut rotasyonu ile taşıdığınızda:

$$x' = \frac{y' \cdot \cos(\theta_{az})}{\tan(\theta_{el})} = \tan(\theta_{az})$$

çıkarmak — yani x eksenini "temiz" kalırsın ama y eksenini azimut arttıkça (kenarlara gidildikçe) fazladan bir $1/\cos(\theta_{az})$ çarpanıyla şişer. Eksenlerin sırası tersse (azimut içte, elevasyon dışta) bu sefer x' azimut yerine elevasyona göre şişer. Bu, tam olarak sizin tarif ettiğiniz "bir eksen düzlemsel kalıyor, diğeri kenarlarda şişiyor" asimetrisiyle örtüşüyor ve bu gerçek lens $k1$ 'inden tamamen bağımsız, saf geometrik bir kuplaj etkisi. Bunu kontrol etmenin yolu, gimbalinizin fiziksel olarak hangi eksenin diğerinin üstünde oturduğunu tespit edip, $\tan(\theta_x)$, $\tan(\theta_y)$ 'yi bağımsız değil, gerçek rotasyon kompozisyonundan türetmek (yukarıdaki gibi, ama sizin gerçek dizilişinize göre) ve modeli buna göre güncellemek. Fit sonrası eğer $k1$ daha küçük/daha simetrik bir değere düşerse, kuplajın gerçekten baskın bir bileşen olduğunu doğrulamış olursunuz.

Bununla birlikte, aynı asimetriyi çok daha sıradan bir sebep de üretebilir: x ekseninde taranan açısal aralık y' 'den hâlâ genişse (raporda 32° vs 25.32° örneği vardı), simetrik gerçek bir barrel distorsiyonu bile, x tarafında daha büyük r değerlerine ulaştığı için görsel olarak "yanlarda daha şişkin" görünür — kuplaj hiç olmasa bile. Bu yüzden önce kuplaj modeline geçmeden, taranan açısal aralığı (mm değil, gerçek derece cinsinden) iki eksende eşitleyip aynı asimetri devam ediyor mu diye bakmanızı öneririm; devam ediyorsa kuplaj gerçek bir etken demektir.

R ve T eklemek fikri hakkında

Zhang'ın checkerboard yönteminde R,T her fotoğrafta ayrı ayrı çözülüyor çünkü tahtanın kameraya göre pozunu her karede bilinmiyor, o yüzden dışsal parametre olarak çözülmesi gerekiyor — raporunuzun 2.7 bölümü bunu zaten "yan ürün" olarak tarif ediyor. Sizin gimbal-lazer yönteminizde ise pozisyon bilinmiyor değil, tam tersine gimbal komutundan zaten biliniyor; ihtiyacınız olan Zhang tarzı "her nokta için ayrı R,T" değil, TEK BİR sabit R — gimbalin sıfır referans eksenini ile kameranın gerçek optik eksenini arasındaki küçük fiziksel bağlanma hatasını (boresight misalignment) temsil eden, tüm ölçüm boyunca sabit kalan global bir dönme parametresi. Bunu modele global bir parametre olarak eklemek mantıklı olabilir. T (öteleme) tarafına gelince — kolimatöre geçmeniz bütün amacı zaten bunu gereksiz kılmaktı: kolime ışın artık "nereden çıktığı" değil yalnızca "hangi açıyla geldiği" ile karakterize ediliyor, o yüzden T'yi geri eklemek aslında kolimasyonun kazandırdığı avantajı bozmak olur. **T'siz devam edin.**

Şimdi asıl can alıcı soru: f_x, f_y, c_x, c_y 'yi datasheet'ten sabitleyip kodda sabit girmek mi, yoksa şu anki gibi least-squares'e bulduruyor olmak mı doğru. Cevap net: sabitlemeyin, şu an yaptığınız (birlikte/joint çözmek) doğru olan ve literatürün standart yaklaşımı. Zhang'de, OpenCV'de, klasik fotogrametri'de nominal/teorik f değerini yalnızca optimizasyonun başlangıç tahmini (θ_0) olarak kullanır, gerçek değer olarak değil — çünkü üretim toleransları, gerçek çalışma mesafesi (sonsuzdan farklı olması), lens-sensör montaj sapması gibi etkenler yüzünden belirli bir kopyanın gerçek f_x, f_y, c_x, c_y 'si datasheet'teki nominal sayıdan sistematik olarak sapabilir. Zaten bunu siz kendi checkerboard deneyinizle bizzat kanıtladınız: $c_x=c_y=1024$ varsayımı yanlış çıktı, gerçek optik merkez $\sim(1031, 952)$ — yani ~ 72 piksellik gerçek, tekrarlanabilir bir donanım kayması var. Aynı mantıkla $f_x=f_y=2251.37$ teorik değerini "ideal görüntü yüksekliği" hesabına sabit olarak koyarsanız, gerçek lensinizin kendine özgü sapmasını (ki tam da ölçmeye çalıştığınız şey bu) baştan yanlış bir sabitle maskeleyiş olursunuz

— ortaya çıkan "distorsiyon" sayısı, gerçek lens bükülmesiyle "benim f tahminim yanlış" hatasının karışımı olur, ayrıştırılamaz hale gelir.

f_x, f_y, c_x, c_y 'yi bulurken yapılan hatanın distorsiyona "sızması" endişeniz haklı ve bu bilinen bir sorun — raporunuzun 10.5 bölümünde zaten " f_x ile k_1, k_2 arasındaki parametre korelasyonu" olarak not edilmiş. Ama bunun çözümü f 'yi dışarıdan (özellikle yanlış olabilecek bir nominal değerle) sabitlemek değil, veriyi bu korelasyonu kırarak kadar çeşitlendirmek: geniş ve yoğun açısal kapsama, farklı bölgelerden örnekleme (tam olarak raporun 6.1 bölümündeki "tek fotoğraf neden yetmiyor" mantığının burada da geçerli hali — orada da tahta-mesafe/ f_x telafisi tek pozda ayırt edilemiyordu, çok pozla çözülüyordu). Yani mevcut yönteminiz ($f_x, f_y, c_x, c_y, k_1, k_2, p_1, p_2$ 'yi birlikte, çok noktalı grid üzerinden least-squares ile bulmak) metodolojik olarak doğru; teorik/datasheet değerleri sadece "makul mü" diye bir sağlama (sanity check) ve başlangıç tahmini olarak kullanın, sabit girdi olarak değil.

Son olarak, "%distortion = (gerçek - ideal)/ideal $\times$ 100" formülünü doğru uygulamak için: ideal ve gerçek tarafın İKİSİNDE DE aynı, kendi fit ettiğiniz K matrisini (f_x, f_y, c_x, c_y — Ölçüm 7/8'deki gibi olgun değerler) kullanmanız gerekir. "İdeal" = $K \cdot [x', y', 1]$ (distorsiyonsuz, saf doğrusal projeksiyon), "gerçek/model" = $K \cdot [x_{\text{dist}}, y_{\text{dist}}, 1]$ (k_1, k_2, p_1, p_2 uygulanmış hali). İkisinin farkını r cinsinden alıp ideal r 'ye oranlarsanız, kendi içinde tutarlı, gerçekten yalnızca k_1, k_2, p_1, p_2 'yi yansıtan bir yüzde elde edersiniz. Önerdiğiniz gibi $x' \cdot 2252.37 + 1025$ formülünü "ideal" için kullanıp gerçek ölçümle kıyaslarsanız, ideal tarafa yanlış (nominal) f_x ve yanlış (1024 varsayımlı) c_x enjekte etmiş olursunuz — bu da lensin gerçek -3.1%'lik spesifikasyonu ile anlamlı biçimde kıyaslanamayacak, sistematik olarak çarpık bir sayı üretir. Datasheet'teki -3.1% değeri de zaten üreticinin KENDİ ölçtüğü gerçek f ile hesaplanmıştır, nominal f ile değil — o yüzden adil bir kıyas için siz de kendi fit ettiğiniz gerçek f_x, f_y, c_x, c_y 'yi kullanmalısınız, üreticinin kataloğundakini değil.

Rx ve Ry hakkında

Birincisi: gimbalinizin yazılım/G-code eksen etiketleri. Rapor bunu net tanımlamış: "Kamera gimbalı: X = azimut, Y = elevasyon." Bu sadece motor kartına hangi komutu (G1 X.. veya G1 Y..) gönderdiğinizin ismi — fiziksel/geometrik bir anlamı yok, MKS kartı öyle etiketlemiş diye öyle.

İkincisi: 3B döndürme matrislerini yazarken kullanılan dünya/optik eksenleri — X = yatay (sağ), Y = dikey (yukarı), Z = ileri (bakış/boresight yönü). Rx, Ry, Rz derken kastedilen, "hangi eksenin ETRAFINDA dönülüyor" — yani $R_y(\alpha)$, dünyanın DİKEY eksen(i) etrafında dönmek demek. Sağa-sola bakmak (azimut) tam olarak bu: dikey eksen etrafında dönersiniz. Yukarı-aşağı bakmak (elevasyon) ise yatay eksen etrafında dönmektir, yani $R_x(\beta)$.

Yani " $R_y(\text{azimut})$ " derken "Y" gimbalinizin G-code etiketi değil, dünya uzayının dikey eksenidir — azimut fiziksel olarak dikey eksen etrafında bir dönme olduğu için matematiksel karşılığı R_y 'dir, gimbal kartındaki "Y=elevasyon" etiketiyle hiçbir ilgisi yok, saf bir isim tesadüf/çakışması.

Aslında kendi el yazısı notebook'unda da tam olarak bunu yapmışsın: " $R_y(\alpha) \rightarrow y$ eksen etrafında dönme" diye not almışsın ve α 'yı "yatay" (sağ-sol, yani azimut) olarak tanımlamışsın; $R_x(\beta)$ 'yi de "dikey" (yukarı-aşağı, elevasyon) β için kullanmışsın. Yani benim yazdığım ile senin notebook'un birebir aynı kuralı kullanıyor —

R_y =azimut, R_x =elevasyon. Kafa karışıklığı sadece gimbal kartının "Y" dediği şeyle (elevasyon motoru) matematikteki "Y eksen etrafında dönme"nin (azimut) aynı harfi paylaşıp farklı şeylere işaret etmesinden kaynaklanıyor.

Çerçeve tanımı: Dünya/optik koordinat sistemi X = yatay (sağ), Y = dikey (yukarı), Z = ileri (boresight, $\alpha=\beta=0$ iken bakış yönü). Işının başlangıç (bozulmamış, gimbal sıfır konumundayken) yönü:

$$d_0 = [0, 0, 1]^T$$

Neden $z=1$: $d_0=(0,0,1)$ bir fiziksel ölçüm değil, saf bir tanım/kural. d_0 bir POZİSYON değil, bir YÖN vektörü — yalnızca "hangi yöne baktığını" temsil ediyor, büyüklüğünün (uzunluğunun) hiçbir önemi yok, çünkü sonunda $x'=dx/dz$, $y'=dy/dz$ diye z 'ye bölerek normalize ediyoruz; bu bölme işlemi ölçeği (skalayı) otomatik iptal eder. $d_0=(0,0,5)$ ya da $(0,0,100)$ seçseydik de x',y' aynı çıkardı — cebiri sadeleştirmek için en basit seçenek olan "1"i aldım. Yani " $z=1$ " demek " $\alpha=\beta=0$ iken, ışın tanım gereği yalnızca $+Z$ yönünde, hiç yana kaymadan gidiyor" demek; birim yok, sadece bir referans.

Gimbal ($\alpha=0$, $\beta=0$) neresi, nasıl biliyoruz — bu asıl önemli soru. Burada iki farklı "sıfır" var ve birbirine karıştırılmaması lazım:

Birincisi, mekanik home (G28) — raporun 4.2.4'ünde tarif edilen, limit switch'lere göre bulunan fiziksel referans ($X \approx 135.17\text{mm}$, $Y \approx 50.61\text{mm}$). Bu, sistemin gün be gün tekrar üretebildiği (repeatability spec'iniz arcsec seviyesinde olduğu için) sabit bir mekanik nokta ama kameranın optik eksenine HİÇBİR garanti ilişkisi yok — sadece "gimbal her zaman aynı yere dönebiliyor" garantisi veriyor, "o nokta kameraya tam bakıyor" garantisi vermiyor.

İkincisi, ve modelin gerçekten kullandığı sıfır: raporun 4.5'inde tarif ettiğiniz Center-Finding prosedürü — home'dan sonra gimballı elle oynatıp lazer noktasını pikselde mümkün olduğunca $(1024,1024)$ 'e yaklaştırdığınız, sonra dokunmadığınız o pozisyon. $x_positions[0]=-half_x$, $y_positions[0]=+half_y$ gibi grid matematiğinizin merkezi $(0,0)$ kabul ettiği yer burası — yani modeldeki " $\alpha=0, \beta=0$ " gerçekte "lazerin, sizin elle merkezlediğiniz o anki konumu" demek.

NOT: Şimdi asıl can alıcı nokta: bu elle-merkezleme kameranın GERÇEK optik eksenine (cx,cy) tam oturmak ZORUNDA değil, ve bu bir sorun değil. Çünkü modeldeki cx,cy zaten tam olarak bunun için var: cx,cy , " $(\alpha=0, \beta=0)$ " komutu verildiğinde ışının hangi piksele düştüğünü temsil eden serbest (fit edilen) parametreler. Siz lazeri elle $(1024,1024)$ 'e ne kadar yakın getirirseniz getirin, kalan küçük fark (ki checkerboard deneyinizde bile gerçek optik merkezin $(1024,1024)$ 'ten $\sim 72\text{px}$ saptığını bulmuştunuz) otomatik olarak least-squares'in bulduğu cx,cy içine sızar — model bunu "hata" olarak değil, tam olarak ölçmesi gereken bir şey olarak görür. Yani gimbal sıfırının kamera eksenine milimetrik/piksel hassasiyette hizalı olması gerekmiyor; gerekli olan tek şey, bu sıfırın BİR ölçüm/fit oturumu boyunca sabit kalması — yani taramanın ortasında gimballı yeniden home'lamamanız ya da sıfırı kaydırmamanız. Kızağın "hangi mm'de bırakıldığını" bilmenize de gerek yok; G91 (görelî mod) kullandığınız için tüm sonraki hareketler o anki (ne olduğu tam bilinmeyen ama sabit olan) referansa göre tanımlanıyor, mutlak konumun kendisi modele hiç girmiyor.

Rotasyon matrisleri:

Elevasyon (β) — yatay eksen (X) etrafında dönme:

$$R_x(\beta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \beta & \sin \beta \\ 0 & -\sin \beta & \cos \beta \end{bmatrix}$$

$$\begin{bmatrix} 0 & \cos\beta & -\sin\beta \end{bmatrix}$$

$$\begin{bmatrix} 0 & \sin\beta & \cos\beta \end{bmatrix}$$

Azimet (α) — düşey eksen (Y) etrafında dönme:

$$R_y(\alpha) = \begin{bmatrix} \cos\alpha & 0 & \sin\alpha \\ 0 & 1 & 0 \\ -\sin\alpha & 0 & \cos\alpha \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -\sin\alpha & 0 & \cos\alpha \end{bmatrix}$$

Durum 1 — benim önerdiğim sıra: $d = R_y(\alpha) \cdot R_x(\beta) \cdot d_0$

Önce $R_x(\beta) \cdot d_0$:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\beta & -\sin\beta \\ 0 & \sin\beta & \cos\beta \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \cdot \cos\beta - 1 \cdot \sin\beta \\ 0 \cdot \sin\beta + 1 \cdot \cos\beta \\ \cos\beta \end{bmatrix} = \begin{bmatrix} -\sin\beta \\ \cos\beta \end{bmatrix}$$

$$\rightarrow d_1 = (0, -\sin\beta, \cos\beta)$$

$$\rightarrow d_1 = (0, -\sin\beta, \cos\beta)$$

$$\rightarrow d_1 = (0, -\sin\beta, \cos\beta)$$

Şimdi $R_y(\alpha) \cdot d_1$:

$$\begin{bmatrix} \cos\alpha & 0 & \sin\alpha \\ 0 & 1 & 0 \\ -\sin\alpha & 0 & \cos\alpha \end{bmatrix} \cdot \begin{bmatrix} 0 \\ -\sin\beta \\ \cos\beta \end{bmatrix} = \begin{bmatrix} \cos\alpha \cdot 0 + \sin\alpha \cdot \cos\beta \\ -\sin\beta \\ -\sin\alpha \cdot 0 + \cos\alpha \cdot \cos\beta \end{bmatrix} = \begin{bmatrix} \sin\alpha \cdot \cos\beta \\ -\sin\beta \\ \cos\alpha \cdot \cos\beta \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} \sin\alpha \cdot \cos\beta \\ -\sin\beta \\ \cos\alpha \cdot \cos\beta \end{bmatrix} = \begin{bmatrix} -\sin\beta \end{bmatrix} = \begin{bmatrix} -\sin\beta \end{bmatrix}$$

$$\begin{bmatrix} -\sin\alpha & 0 & \cos\alpha \end{bmatrix} \cdot \begin{bmatrix} \sin\alpha \cdot \cos\beta \\ -\sin\beta \\ \cos\alpha \cdot \cos\beta \end{bmatrix} = \begin{bmatrix} -\sin\alpha \cdot 0 + \cos\alpha \cdot \cos\beta \\ \cos\alpha \cdot \cos\beta \end{bmatrix} = \begin{bmatrix} \cos\alpha \cdot \cos\beta \end{bmatrix}$$

$$\rightarrow d_{\text{final}} = (\sin\alpha \cdot \cos\beta, -\sin\beta, \cos\alpha \cdot \cos\beta)$$

Normalize koordinatlar (pinhole: $x' = dx/dz$, $y' = dy/dz$):

$$x' = (\sin\alpha \cdot \cos\beta) / (\cos\alpha \cdot \cos\beta) = \sin\alpha / \cos\alpha = \tan\alpha \rightarrow \beta \text{ yok, TEMİZ}$$

$$y' = (-\sin\beta) / (\cos\alpha \cdot \cos\beta) = -\tan\beta / \cos\alpha \rightarrow \text{hem } \alpha \text{ hem } \beta \text{ var, KUPLAJLI}$$

$d_{\text{final}} = (dx, dy, dz) = (\sin\alpha \cdot \cos\beta, -\sin\beta, \cos\alpha \cdot \cos\beta)$. Buradaki $dz = \cos\alpha \cdot \cos\beta$, biz seçmedik, rotasyon matrislerinin çarpımından kendiliğinden çıktı — bu vektörün "derinlik" (Z) bileşeni. Normalize görüntü koordinatına geçmek için, x ve y bileşenlerinin İKİSİNİ DE aynı dz'ye böleriz:

$$x' = dx/dz = (\sin\alpha \cdot \cos\beta) / (\cos\alpha \cdot \cos\beta)$$

$$y' = dy/dz = (-\sin\beta) / (\cos\alpha \cdot \cos\beta)$$

ÇÖZÜM: KOD DEĞİŞİMİ

```
theta_x = np.array([r["mm_x"] * DEG_PER_MM_X for r in valid])
```

```
theta_y = np.array([r["mm_y"] * DEG_PER_MM_Y for r in valid])
```

```
alpha = np.radians(theta_x) # azimet açısı, radyan
```

```
beta = np.radians(theta_y) # elevasyon açısı, radyan
```

xprime = np.tan(alpha)

yprime = -np.tan(beta) / np.cos(alpha),

X ve Ynin aynı derece taramaması hakkında

Kilit gözlem şu: k1, k2 gibi radyal distorsiyon katsayıları yalnızca $r = \sqrt{x'^2 + y'^2}$ 'ye bağlı, yani "hangi eksen" büyük r'ye ulaştığınız önemli değil, önemli olan büyük r'ye ULAŞMIŞ olmanız. Bu da şu anlama geliyor: y eksenini tek başına ($x=0$ hattında) yalnızca 12-13°'ye kadar gidebilse bile, gridin köşe noktaları (hem x hem y aynı anda maksimuma yakinken) çok daha büyük bir r'ye ulaşır — örneğin $x=16^\circ$, $y=12.66^\circ$ köşesinde $r=\sqrt{16^2+12.66^2}\approx 20.4^\circ$, ki bu x'in tek başına ulaştığı 16°'den bile büyük. Yani mevcut dikdörtgen grid zaten köşelerinde y'nin "tek başına göremediği" büyüklükte r'leri örnekliyor — sadece bunlar eksen üzerinde değil, çapraz yönlerde yoğunlaşmış durumda.

elinizdeki "elle görülen" dört sınır değeri zaten işimizi görüyor, onlardan ortak simetrik bir derece çıkaralım.

Daha önce hesapladığımız dört sınır (dereceye çevrilmiş hâliyle):

X(+):14.00°

X(-):19.98°

Y(+):21.68°

Y (-) : 13.59° ← en dar olan, darboğaz burada

Simetrik ve ortak bir aralık istiyorsanız, dört yönün de İÇİNDE kalan en büyük değeri seçmeniz gerekir — yani en dar yönü (Y'nin aşağısı, 13.59°) belirleyici olur. Biraz güvenlik payı bırakarak (tam sınırdan değil, sınırın hafif içinde kalmak için):

Ortak simetrik açısal yarı-genişlik: 13.0° (yani her iki eksen de home'dan $\pm 13.0^\circ$, toplam $26^\circ \times 26^\circ$ lik kare bir tarama alanı)

Bunu mm'ye çevirirsek (DEG_PER_MM_X=1/5.0, DEG_PER_MM_Y=1/2.37 ile):

python

SAFE_LIMIT_X = 65.0 # mm (13.0° / 0.2)

SAFE_LIMIT_Y = 30.8 # mm (13.0° / 0.4219)

Bu iki değeri kodunuzdaki SAFE_LIMIT_X, SAFE_LIMIT_Y sabitlerinin yerine koyup gridinizi (ve varsa spoke taramanızı) bunlara göre kurun — böylece X ve Y artık tam olarak aynı açısal aralığı ($\pm 13^\circ$) tarıyor olacak, dört yönün de gerçekten görebildiğiniz sınırların içinde kalacak, ve az önce konuştuğumuz "x daha büyük r'ye ulaştığı için görsel şişkinlik" etkisi devre dışı kalacak — kalan asimetri varsa artık gerçekten kuplaj ya da lens kaynaklı demektir.

Not: bu, home'un X(+) tarafında (14° mevcutken 13° kullanıyorsunuz) ve Y(+) tarafında (21.68° mevcutken 13° kullanıyorsunuz) elinizdeki kullanılabilir alanın bir kısmını bilerek harcamıyor bırakıyor — ama amaç şu an maksimum kapsama değil, temiz ve adil bir karşılaştırma, o yüzden doğru tercih bu.

SPOKE önerisi: şu an daha denenmedi

Buradan çıkan pratik öneri: gridi yalnızca dikdörtgen (satır-sütun) mantığıyla değil, ayrıca kutupsal (radial/spoke) bir örnekleme deseniyle de tarayın — merkezden dışarıya doğru birçok farklı ϕ açısında (0° , 22.5° , 45° , 67.5° , 90° gibi), her yönde o yöndeki iki eksenin ayrı ayrı izin verdiği maksimum r'ye kadar ($r_{\max}(\phi) = \min(x_{\text{limit}}/|\cos\phi|, y_{\text{limit}}/|\sin\phi|)$) ile hesaplanır) noktalar toplayın. Böylece sadece 4 köşede değil, birçok açısal yönde büyük-r verisi birikir; bu hem k1,k2'nin gerçekten izotropik (yöne bağlı olmayan, saf radyal) olup olmadığını sınamanıza hem de fitin büyük-r bölgesindeki güvenilirliğini artırmanıza yarar — y ekseninin kendi başına dar kalması bu stratejiyle önemli ölçüde telafi edilir.

Fit sonrasında da mutlaka şunu yapın: residual'i (ölçülen - model tahmini) r'ye karşı değil, $\phi = \text{atan2}(y', x')$ açısına karşı çizin (r'yi yaklaşık sabit tutan bir bant seçip, o bant içindeki noktaların residual'ini açıya göre dağıtın). Eğer residual ϕ 'den bağımsız, rastgele dağılıyorsa distorsiyon gerçekten izotropik demektir ve gördüğünüz "üstte düz, yanda şişkin" görüntü, büyük ihtimalle sadece x'in daha büyük r'ye ulaşmasından kaynaklanan bir görsel algı meselesidir — katsayılar zaten doğru. Ama residual'de ϕ 'ye bağlı sistematik bir dalga (örneğin $\phi=0^\circ$ 'de -yani saf x yönünde- residual hep bir yönde, $\phi=90^\circ$ 'de -saf y yönünde- hep başka bir yönde) görülüyorsa, bu gerçek bir izotropik-olmayan etkendir — ya önceki mesajda konuştuğumuz gimbal kuplajı ya da lensin/sensörün gerçekten eksenler arası asimetrik davranması. Bu grafik, elinizdeki mevcut veriyle (yeni ölçüm yapmadan) bugün çizilebilir ve hangi teoriyi öncelemeniz gerektiğine dair en ucuz/en hızlı kanıtı verir.

Mekanik sınırı gerçekten genişletmek isterseniz, raporun "Ek: Bilinen Donanım Kısıtları" bölümünde zaten not edilmiş — elevasyon kazağının travel mesafesini artırmak (vida boyu, limit switch konumu) kalıcı çözüm ama kısa vadede pratik değil; kutupsal örnekleme + ϕ -residual analizi, elinizdeki donanım ile bu hafta yapılabilecek, mekanik değişiklik gerektirmeyen bir ara adım olarak öneriyorum.

Neden büyük r'ye ihtiyacımız var? Lens bükülmesi (distorsiyon) merkeze yakın neredeyse hiç fark edilmez, merkezden uzaklaştıkça (r büyüdükçe) belirginleşir — tıpkı bir gölete taş attığınızda dalgaların merkeze yakın küçük, kıyıya doğru büyük olması gibi. k1, k2 katsayılarını güvenilir bulabilmek için, elinizde büyük r'ye sahip (yani merkeze uzak) bol nokta olması lazım — az sayıda ya da hiç büyük-r noktanız yoksa, "dalga'nın kıyıda ne kadar büyüdüğünü" tahmin etmek zorlaşır, optimizasyon kafadan atar.

Sorun neydi? Y ekseniniz (elevasyon) mekanik olarak fazla gidemiyor, yani SADECE Y yönünde ($\phi=90^\circ$, tam yukarı ya da tam aşağı) hareket ederseniz büyük r'ye ulaşamıyorsunuz — $12-13^\circ$ 'de tıkanıyor. X ekseni (azimut) ise 16° 'ye kadar rahat gidiyor.

Spoke (ışın demeti) fikri ne işe yarıyor? Eğer sadece X yönünde ya da sadece Y yönünde değil, ÇAPRAZ yönlerde ($\phi=45^\circ$ gibi) hareket ederseniz, r'nin bir kısmını X'ten bir kısmını Y'den alırsınız — mesela $r=18^\circ$ 'lik bir noktaya çapraz giderken X ekseni sadece $18^\circ \times \cos(45^\circ) \approx 12.7^\circ$, Y ekseni de sadece $18^\circ \times \sin(45^\circ) \approx 12.7^\circ$ kadar gitmiş olur; ikisi de kendi limitinin (16° ve 12.7°) içinde kalır ama ULAŞILAN TOPLAM r (18°), Y'nin tek başına hiç ulaşamayacağı bir büyüklük. Yani çapraz gidince, Y'nin dar limitini "X'in yardımıyla" telafi edip daha büyük r'lere ulaşabiliyorsunuz — sadece 4 grid köşesinde değil, aradaki birçok ϕ açısında da (22.5° , 67.5° gibi) bunu

tekrarlayarak, merkezden her yöne doğru "ışınlar" halinde uzanan ek noktalar topluyorsunuz. Bu yüzden "kutupsal/spoke tarama" diyorum — pergelle çember çizer gibi değil, tekerlek jantından çıkan ıspitler gibi merkezden dışa doğru birçok yönde uzanan çizgiler.

Şimdi residual/ ϕ grafiği ne işe yarıyor, bunu da sıfırdan anlatayım. Fit'i yaptıktan sonra ($f_x, f_y, c_x, c_y, k_1, k_2, p_1, p_2$ elinizde), her ölçüm noktası için modelin "burada olmalı" dediği piksel konumu ile fotoğraftan gerçekten ölçtüğünüz piksel konumu arasında küçük bir fark kalır — buna residual (kalan hata) diyoruz, RMS de zaten bu farkların toplu ortalaması. Şimdi bu farkı, her noktanın r 'sine göre değil, ϕ 'sine (hangi yönde olduğuna) göre bir grafiğe dökün: yatay eksen ϕ (0° 'den 360° 'ye), dikey eksen o noktadaki residual.

İki farklı sonuç çıkabilir:

Birincisi — grafikte hiçbir düzen yok, noktalar rastgele saçılmış, $\phi=0^\circ$ 'de de $\phi=90^\circ$ 'de de residual ortalama sıfır civarında rastgele dağılıyor. Bu, modelin (yani "distorsiyon sadece r 'ye bağlıdır, yöne bağlı değildir" varsayımının) doğru olduğu anlamına gelir — gölete taş attığınızda dalgaların her yöne eşit yayılması gibi. Bu durumda gördüğünüz "üst düz, yan şişkin" görüntü sadece X 'in Y 'den daha uzağa (büyük r 'ye) ulaşmasından kaynaklanan bir GÖRSEL yanılısma — katsayılarınız zaten doğru, düzeltmenize gerek yok.

İkincisi — grafikte bir düzen var: mesela $\phi=0^\circ$ ve $\phi=180^\circ$ civarında (yani X eksenini boyunca) residual hep aynı yönde (+), $\phi=90^\circ$ ve $\phi=270^\circ$ civarında (Y eksenini boyunca) hep ters yönde (–). Bu, distorsiyonun gerçekten YÖNE BAĞLI olduğunu, yani gölete atılan taşın dalgalarının bir yönde daha güçlü diğer yönde daha zayıf yayıldığını gösterir — bu da ya konuştuğumuz gimbal kuplaj etkisinin (ki onu zaten kodunuzda düzelttiniz) tam giderilmediğine ya da lensin/sensörün gerçekten eksenler arası farklı davrandığına işaret eder.

Bunu bugün, yeni bir ölçüm yapmadan, elinizdeki Ölçüm 7 veya 8'in kayıtlı ham verisiyle (her noktanın $mm_x, mm_y, x_{meas}, y_{meas}$ 'i ve fit'ten çıkan $f_x, f_y, c_x, c_y, k_1, k_2, p_1, p_2$) çizebilirsiniz — spoke taramasını fiziksel olarak yapmadan önce, elinizdeki veriyle bu grafiği çizip "asimetri gerçek mi yoksa sadece r farkından mı" sorusuna ucuz ve hızlı bir ilk cevap alabilirsiniz. Spoke tarama, bu ilk bakışın ötesinde daha kesin/yoğun kanıt toplamak istediğinizde devreye giren ikinci adım.

ÖZET

1. Gimbal kuplaj modeli — Kodunuz x' ve y' 'yi birbirinden bağımsız ($\tan(\theta_x), \tan(\theta_y)$) hesaplıyordu, oysa iki eksenli gimbalinizde (azimut dış/tabana, elevasyon onun üstünde taşınan iç eksen) bu ikisi bağımsız değil. Formülü $x' = \tan(\alpha)$, $y' = -\tan(\beta)/\cos(\alpha)$ olarak değiştirdik — bu, gimbalin gerçek 3B rotasyon geometrisinden ($R_y(\alpha) \cdot R_x(\beta)$) türetilmişti. Bu, önceden yanlışlıkla k_1, k_2 'ye yüklenen sahte/geometrik büyümeyi modelden ayırıyor.

2. Simetrik açısal tarama aralığı — X ve Y eksenlerinin gerçek görülebilir sınırlarını ölçüp (asimetrik çıktı: $X -19.98^\circ/+14.00^\circ$, $Y -13.59^\circ/+21.68^\circ$), ikisinin de ortak, simetrik bir aralıkta ($\pm 13^\circ$) tarama yapmasını sağladık. $SAFE_LIMIT_X=65.0\text{mm}$, $SAFE_LIMIT_Y=30.8\text{mm}$ olarak güncellendi — böylece " x daha büyük r 'ye ulaştığı için görsel şişkinlik" ihtimali devre dışı kaldı.

3. save_distortion_comparison bug'u — Bu fonksiyon "ideal" (distorsiyonsuz) referans noktalarını hesaplarken hâlâ eski, kuplajsız formülü kullanıyordu; fit'in kendisiyle tutarsızdı. Aynı kuplajlı modeli kullanacak şekilde düzeltildi, barrel_pincushion.png artık gerçek fit sonucunu doğru yansıtıyor.

4. ϕ -residual izotropi kontrolü eklendi — Fit'ten sonra otomatik çalışan plot_phi_residual fonksiyonu, kalan hatanın (residual) noktanın açısal yönüne (ϕ) bağlı olup olmadığını gösteriyor — distorsiyonun gerçekten izotropik mi yoksa hâlâ yöne-bağlı bir sapma mı taşıdığını bu grafikte sınavacaksınız.

Henüz yapılmayan, sonraki adıma bağlı olan tek şey: global boresight R (gimbal sıfırı ile kamera eksenindeki sabit hizalanma farkı) — ϕ -residual grafiğinde hâlâ sistematik bir dalga görürseniz ona bakacağız.

İşte bugünkü çalışmanın uçtan uca özeti — üç ana başlıkta topluyorum: kavramsal kafa karışıklıkları ve cevapları, elde ettiğimiz veriler/kararlar, ve koda yapılan somut düzeltmeler.

Kavramsal kısım: Önce raporu okuyup mevcut durumu (checkerboard + gimbal-lazer iki aşamalı kalibrasyon, 8 ölçümlük kronoloji, $k_1 \sim 1.20$ barrel bulgusu) özümstedik. Sonra asıl derin tartışma başladı: gördüğünüz barrel'in ne kadarının gerçek lens distorsiyonu, ne kadarının gimbalin iki eksenli (azimut+elevasyon) hareketinin yanlış modellenmesinden kaynaklandığı sorusu. Bunu çözmek için gimbalin gerçek 3B rotasyon geometrisini (R_x, R_y matrisleri, hangi eksenin "dış/tabana" hangisinin "iç/taşınan" olduğu) adım adım türettik; sizin donanımınızda (azimut disk taban, elevasyon onun üstünde taşınıyor) doğru modelin $x' = \tan \alpha$ (temiz), $y' = -\tan \beta / \cos \alpha$ (kuplajlı) olduğuna ulaştık. Ayrıca f_x, f_y, c_x, c_y 'yi datasheet'ten sabitlemek yerine (Zhang yönteminde olduğu gibi) least-squares'e serbestçe buldurmanın doğru yaklaşım olduğunu, ve lens datasheet'indeki -3.1% ile kıyaslanacak distorsiyon yüzdesinin f_x 'ten bağımsız olarak $(k_1 \cdot r^2 + k_2 \cdot r^4) \times 100$ formülüyle hesaplanabileceğini netleştirdik.

Veri/karar kısmı: Elinizdeki home ve uç nokta mm değerlerinden ($X=135.17/Y=50.61$ home; X sol/sağ, Y alt/üst uçlar) gerçek açısal tarama aralığının simetrik olmadığını ($X: -19.98^\circ/+14.00^\circ$, $Y: -13.59^\circ/+21.68^\circ$) hesapladık; bu asimetriyi ortadan kaldırmak için ortak, simetrik bir $\pm 13.0^\circ$ aralığında karar kıldık ($SAFE_LIMIT_X=65.0\text{mm}$, $SAFE_LIMIT_Y=30.8\text{mm}$) — böylece "x daha büyük r'ye ulaştığı için görsel şişkinlik" ihtimalini devre dışı bıraktık.

Koda yapılan düzeltmeler (script içinde):

- Kuplaj formülü ($x_{\text{prime}} = \tan(\alpha)$, $y_{\text{prime}} = -\tan(\beta)/\cos(\alpha)$) fit_camera_params'a eklendi.
- $SAFE_LIMIT_X/Y$ simetrik $\pm 13^\circ$ değerlerine güncellendi.
- save_distortion_comparison'daki bug düzeltildi — "ideal" nokta hesabı artık fit ile aynı kuplajlı modeli kullanıyor (eskiden tutarsızdı).
- plot_phi_residual eklendi — fit sonrası residual'in açısal yöne (ϕ) bağlı olup olmadığını (izotropi kontrolü) otomatik çiziyor ve kaydediyor.
- plot_distortion_curve eklendi — k_1, k_2 'den hesaplanan distorsiyon(%) eğrisini datasheet'teki -3.1% 'e karşı çizip sayısal farkı terminale yazdırıyor.
- Lazer tespiti tamamen sadeleştirildi: önce sabit $MIN_PEAK/MIN_CONTRAST$ eşikleri kaldırılıp istatistiksel eşığa geçildi, sonra bu da yetmeyince tüm eşik/red mantığı (MIN_AREA , morfolojik opening, kenar/aspect/sıçrama filtreleri, FOUND/NOT_FOUND geri bildirimi) tamamen kaldırıldı —

artık her karede en parlak nokta koşulsuz alınıp subpixel centroid'i hesaplanıyor, hiçbir nokta reddedilmiyor/NaN olmuyor.

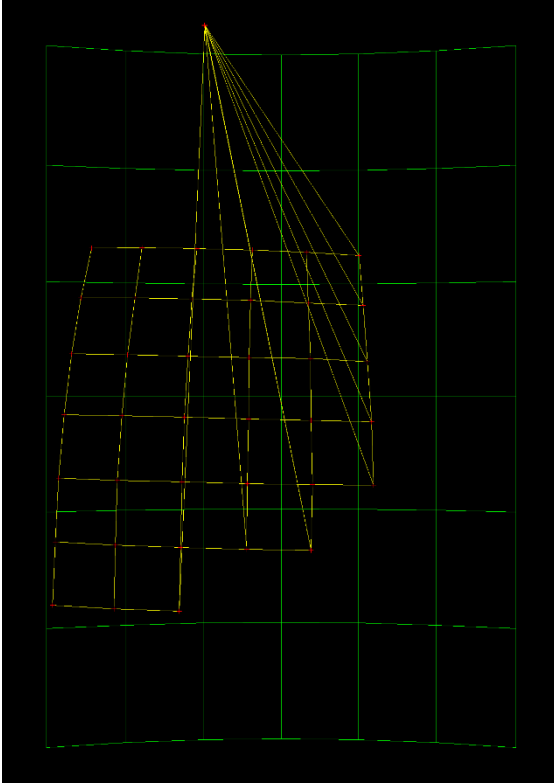
- CSV çıktısı ve run_scan'daki ekran yazıları bu sadeleşmeye göre güncellendi.

NEDEN SPOKE DENENMEDİ

Hayır, spoke (kutupsal) tarama koda hiç eklenmedi — grepladım, dosyada spoke, r_max, N_SPOKES gibi hiçbir iz yok. build_scan_plan hâlâ sadece düz dikdörtgen (satır-sütun) bir grid üretiyor.

Nedeni şu: spoke fikri, X ve Y'nin FARKLI açısal aralık taradığı dönemde (Y mekanik olarak dar kaldığı için) ortaya attığım bir çözümdü — amaç Y'nin tek başına ulaşamadığı büyük r'lere çapraz yönlerden ulaşmaktı. Ama sonra siz gerçek görülebilir sınırları ölçtünüz ve biz bunun yerine daha basit bir çözüme geçtik: X ve Y'yi ortak, simetrik $\pm 13^\circ$ 'ye çektik (SAFE_LIMIT_X=65.0mm, SAFE_LIMIT_Y=30.8mm). Bu, spoke'un çözmeye çalıştığı asıl sorunu (eksenler arası dengesizlik) zaten kökünden hallettiği için spoke'u uygulamaya hiç geçirmedik, konuşma orada kaldı.

Şu anki dikdörtgen grid zaten simetrik olduğu için köşe noktaları ($X=\pm 13^\circ$, $Y=\pm 13^\circ$) diagonal yönde $r=\sqrt{13^2+13^2}\approx 18.4^\circ$ 'ye ulaşıyor — yani bir miktar ϕ çeşitliliği ve büyük-r verisi hâlâ var, sadece spoke kadar yoğun/sistemik değil. İsterseniz ekstra bir izotropi güvencesi olarak spoke taramasını da (grid'e ek olarak) şimdi entegre edebilirim, ya da önce mevcut sadeleşmiş kodla bir tarama yapıp ϕ -residual grafiğine bakıp gerçekten ihtiyaç var mı diye karar verebiliriz — hangisini istersiniz?



f_x ve f_y aslında birer "**derece başına kaç piksel**" çevrim katsayısı — direkt olarak piksel = $f * \tan(\text{açı})$ formülünde açıyı piksele çeviren şey onlar. X ekseninde bu çevrimi f_x yapıyor, Y ekseninde f_y yapıyor. Eğer f_x ile f_y

birbirinden farklıysa, aynı açıyı (mesela 10°) taradığında X ekseninde ürettiğin piksel mesafesiyle Y ekseninde ürettiğin piksel mesafesi otomatik olarak farklı çıkar — çünkü iki eksen açıyı piksele çevirirken farklı "büyütme oranı" kullanıyor.

Somut sayılarla göstereyim, $\tan(10^\circ) \approx 0.176$:

- X ekseninde $\pm 10^\circ$ taradığında: piksel kayması $\approx f_x \times 0.176$
- Y ekseninde $\pm 10^\circ$ taradığında: piksel kayması $\approx f_y \times 0.176$

Bozuk fit'teki değerlerle ($f_x=2183$, $f_y=3184$):

- X: $2183 \times 0.176 \approx 384$ piksel (her yöne, yani toplam genişlik ≈ 768 px)
- Y: $3184 \times 0.176 \approx 561$ piksel (her yöne, yani toplam yükseklik ≈ 1122 px)

768 genişliğe karşı 1122 yükseklik — oran $1122/768 \approx 1.46$. Gönderdiğin görüntüdeki yeşil grid'in en/boy oranına bak, neredeyse birebir bu oranla eşleşiyor. Yani açı eşit ($\pm 10^\circ = \pm 10^\circ$) ama piksel karşılığı eşit değil, çünkü $f_x \neq f_y$ "çevirici" farklı çalışıyor.

Neden $f_x \neq f_y$ olur diye sorarsan iki ihtimal var: (1) gerçekten kameranın/lensin X ve Y ekseninde piksel başına düşen açısal görüş alanı farklı olabilir (piksel pitch'i tam kare olmayabilir, ya da lensin odak uzaklığı eksenler arasında ufak farklılık gösterebilir — bu normaldir, birçok gerçek kamerada f_x/f_y %1-5 civarı farklı çıkar), (2) ya da bizim bu spesifik fit'te olduğu gibi, veri bozuksa (yanlış tespit edilmiş birkaç nokta) optimizasyon f_x/f_y 'yi gerçek değerlerinden çok uzağa savurabilir — %46 gibi bir fark gerçek bir lens için fazla büyük, bu yüzden bu spesifik sonuçta muhtemelen 2. ihtimal geçerli.

Yani kısacası: kare aç taraması $\rightarrow$ kare piksel grid garantisi vermez, çünkü aradaki köprü (f_x, f_y) eşit olmak zorunda değil. Bizim yapmamız gereken, temiz veriyle f_x/f_y 'nin gerçekte ne kadar birbirine yakın çıktığını görmek — eğer temiz veride de böyle büyük bir fark çıkarsa o zaman gerçek bir sensör/lens özelliği, ama muhtemelen çok daha küçük bir farka (belki %2-5) düşecek.

1. **Piksel uzayındaki ideal grid'in şekli** — bu, kameranın f_x ve f_y değerlerine (piksel cinsinden odak uzaklığı, X ve Y eksenlerinde ayrı ayrı) bağlı. İdeal nokta hesaplanırken $x_{\text{pixel}} = f_x * \tan(\alpha) + c_x$, $y_{\text{pixel}} = f_y * \tan(\beta) / \cos(\alpha) + c_y$ şeklinde bir formül kullanılıyor. Eğer $f_x \neq f_y$ ise (ki bozuk fit'te $f_x=2183$, $f_y=3184$ çıkmıştı, yani aralarında $\sim$ %46 fark var), **aynı açısal aralık bile piksel uzayında farklı büyüklükte bir aralığa karşılık gelir** — açı eşit olsa da f_y büyük olduğu için Y eksenini piksel cinsinden "daha fazla gerilir" ve grid dikdörtgen görünür.

Yani yeşil grid'in kare olmaması bir hata ya da tasarım sorunu değil — sensörün/lensin f_x, f_y değerleri birbirinden farklıysa (ki bu gerçek kameralarda oldukça yaygındır, piksel pitch'i kare olsa bile fit sürecinde küçük farklar çıkabilir) bu tamamen beklenen bir durum. Bizim $\pm 10^\circ$ simetrisiyle amaçladığımız şey, grid'in *pikselde kare görünmesi* değildi — amaç, distorsiyonun (sarı ile yeşil arasındaki fark) X ve Y'de **farklı derecede örneklenmiş olmaktan** kaynaklanan yapay bir asimetri göstermemesiydi. Grid'in kendisi dikdörtgen olabilir, önemli olan sarı noktaların yeşil grid'e göre simetrik/tutarlı sapması.

Şimdi asıl yapmamız gereken: yeni $\pm 10^\circ$ limitleriyle temiz bir 7×7 tarama çekip f_x/f_y 'nin bu sefer ne çıktığına bakmak. Eğer o zaman f_x ve f_y birbirine yakınsa (mesela %5-10 fark), grid de zaten görsel olarak kareye çok daha yakın duracak — çünkü asıl "dikdörtgenlik" bu bozuk fit'teki f_x/f_y uçurumundan geliyordu, senin tarama tasarımından değil.

Yeni taramayı çekip sonuçları paylaşabilir misin, o zaman gerçek f_x/f_y oranına ve grid şekline birlikte bakalım.